

Virtual Machine Specification

Java Card™ Platform, Version 2.2.2

Sun Microsystems, Inc.
www.sun.com

3-15-06

Copyright © 2005 Sun Microsystems, Inc., 4150 Network Circle, Santa Clara, California 95054, U.S.A. All rights reserved.

Sun Microsystems, Inc. has intellectual property rights relating to technology embodied in the product that is described in this document. In particular, and without limitation, these intellectual property rights may include one or more of the U.S. patents listed at http://www.sun.com/patents and one or more additional patents or pending patent applications in the U.S. and in other countries.

U.S. Government Rights - Commercial software. Government users are subject to the Sun Microsystems, Inc. standard license agreement and applicable provisions of the FAR and its supplements. This distribution may include materials developed by third parties.

Sun, Sun Microsystems, the Sun logo, Java, Solaris, Java Card, Java Developer Connection, Javadoc, JDK, JVM, J2ME, NetBeans and J2SE are trademarks or registered trademarks of Sun Microsystems, Inc. in the U.S. and other countries.

UNIX is a registered trademark in the U.S. and other countries, exclusively licensed through X/Open Company, Ltd.

Products covered by and information contained in this service manual are controlled by U.S. Export Control laws and may be subject to the export or import laws in other countries. Nuclear, missile, chemical biological weapons or nuclear maritime end uses or end users, whether direct or indirect, are strictly prohibited. Export or reexport to countries subject to U.S. embargo or to entities identified on U.S. export exclusion lists, including, but not limited to, the denied persons and specially designated nationals lists is strictly prohibited.

DOCUMENTATION IS PROVIDED "AS IS" AND ALL EXPRESS OR IMPLIED CONDITIONS, REPRESENTATIONS AND WARRANTIES, INCLUDING ANY IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT, ARE DISCLAIMED, EXCEPT TO THE EXTENT THAT SUCH DISCLAIMERS ARE HELD TO BE LEGALLY INVALID.



Please
Recycle



Contents

Preface xvii

1. Introduction 1–1

1.1 Motivation 1-1

1.2 The Java Card Virtual Machine 1-2

1.3 Java Language Security 1-4

1.4 Java Card Runtime Environment Security 1-5

2. A Subset of the Java Virtual Machine 2–1

2.1 Why a Subset is Needed 2-1

2.2 Java Card Platform Language Subset 2-1

2.2.1 Unsupported Items 2–2

2.2.1.1 Unsupported Features 2–2

2.2.1.2 Keywords 2–4

2.2.1.3 Unsupported Types 2–4

2.2.1.4 Classes 2–5

2.2.2 Supported Items 2–5

2.2.2.1 Features 2–5

2.2.2.2 Keywords 2–7

2.2.2.3 Types 2–7

2.2.2.4 Classes 2–7
2.2.3 Optionally Supported Items 2–8
2.2.3.1 Integer Data Type 2–8
2.2.3.2 Object Deletion Mechanism 2–8
2.2.4 Limitations of the Java Card Virtual Machine 2–8
2.2.4.1 Packages 2–9
2.2.4.2 Classes 2–9
2.2.4.3 Objects 2–10
2.2.4.4 Methods 2–10
2.2.4.5 Switch Statements 2–11
2.2.4.6 Class Initialization 2–11
2.2.5 Multiselectable Applets Restrictions 2–12
2.2.6 Java Card Platform Remote Method Invocation (RMI) Restrictions 2–12
2.2.6.1 Remote Classes and Remote Interfaces 2–12
2.2.6.2 Access Control of Remote Interfaces 2–12
2.2.6.3 Parameters and Return Values 2–13
2.3 Java Card VM Subset 2-13
2.3.1 class File Subset 2–13
2.3.1.1 Not Supported in Class Files 2–14
2.3.1.2 Supported in Class Files 2–15
2.3.2 Bytecode Subset 2–16
2.3.2.1 Unsupported Bytecodes 2–17
2.3.2.2 Supported Bytecodes 2–17
2.3.2.3 Static Restrictions on Bytecodes 2–18
2.3.3 Exceptions 2–19
2.3.3.1 Uncaught and Uncatchable Exceptions 2–19
2.3.3.2 Checked Exceptions 2–20
2.3.3.3 Runtime Exceptions 2–21

2.3.3.4 Errors 2–21

3. Structure of the Java Card Virtual Machine 3–1

3.1 Data Types and Values 3-1

3.2 Words 3-2

3.3 Runtime Data Areas 3-2

3.4 Contexts 3-2

3.5 Frames 3-3

3.6 Representation of Objects 3-3

3.7 Special Initialization Methods 3-3

3.8 Exceptions 3-4

3.9 Binary File Formats 3-4

3.10 Instruction Set Summary 3-4

3.10.1 Types and the Java Card Virtual Machine 3–5

4. Binary Representation 4–1

4.1 Java Card Platform File Formats 4-1

4.1.1 Export File Format 4–2

4.1.2 CAP File Format 4–2

4.1.3 JAR File Container 4–2

4.2 AID-based Naming 4-3

4.2.1 The AID Format 4–3

4.2.2 AID Usage 4–4

4.3 Token-based Linking 4-5

4.3.1 Externally Visible Items 4–5

4.3.2 Private Tokens 4–5

4.3.3 The Export File and Conversion 4–6

4.3.4 References – External and Internal 4–6

4.3.5 Installation and Linking 4–7

4.3.6 Token Assignment 4–7
4.3.7 Token Details 4–7
4.3.7.1 Package 4–8
4.3.7.2 Classes and Interfaces 4–8
4.3.7.3 Static Fields 4–8
4.3.7.4 Static Methods and Constructors 4–8
4.3.7.5 Instance Fields 4–9
4.3.7.6 Virtual Methods 4–9
4.3.7.7 Interface Methods 4–10
4.4 Binary Compatibility 4-11
4.5 Package Versions 4-13
4.5.1 Assigning 4–13
4.5.2 Linking 4–13

5. The Export File Format 5–1
5.1 Export File Name 5-2
5.2 Containment in a JAR File 5-2
5.3 Ownership 5-2
5.4 Hierarchies Represented 5-3
5.5 Export File 5-3
5.6 Constant Pool 5-4
5.6.1 CONSTANT_Package 5–5
5.6.2 CONSTANT_Classref 5–6
5.6.3 CONSTANT_Integer 5–7
5.6.4 CONSTANT_Utf8 5–7
5.7 Classes and Interfaces 5-8
5.8 Fields 5-11
5.9 Methods 5-13
5.10 Attributes 5-15

5.10.1 ConstantValue Attribute 5–15

6. The CAP File Format 6–1

6.1 Component Model 6-2

6.1.1 Containment in a JAR File 6–3

6.1.2 Defining New Components 6–4

6.2 Installation 6-5

6.3 Header Component 6-6

6.4 Directory Component 6-9

6.5 Applet Component 6-12

6.6 Import Component 6-13

6.7 Constant Pool Component 6-14

6.7.1 CONSTANT_Classref 6–16

6.7.2 CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref 6–18

6.7.3 CONSTANT_StaticFieldref and CONSTANT_StaticMethodref 6–19

6.8 Class Component 6-21

6.8.1 type_descriptor 6–23

6.8.2 interface_info and class_info 6–26

6.8.2.1 interface_info and class_info shared Items 6–26

6.8.2.2 interface_info Items 6–28

6.8.2.3 class_info Items 6–28

6.8.2.4 implemented_interface_info 6–32

6.8.2.5 remote_interface_info 6–32

6.9 Method Component 6-35

6.9.1 exception_handler_info 6–37

6.9.2 method_info 6–38

6.10 Static Field Component 6-41

6.11 Reference Location Component 6-44

6.12 Export Component 6-47
6.13 Descriptor Component 6-49
6.13.1 class_descriptor_info 6–50
6.13.2 field_descriptor_info 6–52
6.13.3 method_descriptor_info 6–54
6.13.4 type_descriptor_info 6–56
6.14 Debug Component 6-57
6.14.1 The class_debug_info Structure 6–58
6.14.1.1 The field_debug_info Structure 6–61
6.14.1.2 The method_debug_info Structure 6–62

7. Java Card Virtual Machine Instruction Set 7–1
7.1 Assumptions: The Meaning of "Must" 7-1
7.2 Reserved Opcodes 7-2
7.3 Virtual Machine Errors 7-2
7.4 Security Exceptions 7-3
7.5 The Java Card Virtual Machine Instruction Set 7-3
7.5.1 aaload 7–5
7.5.2 aastore 7–6
7.5.3 aconst_null 7–8
7.5.4 aload 7–8
7.5.5 aload_<n> 7–9
7.5.6 anewarray 7–10
7.5.7 areturn 7–10
7.5.8 arraylength 7–11
7.5.9 astore 7–12
7.5.10 astore_<n> 7–12
7.5.11 athrow 7–13
7.5.12 baload 7–14

7.5.13 bastore 7–15
7.5.14 bipush 7–16
7.5.15 bspush 7–16
7.5.16 checkcast 7–17
7.5.17 dup 7–19
7.5.18 dup_x 7–20
7.5.19 dup2 7–21
7.5.20 getfield_<t> 7–22
7.5.21 getfield_<t>_this 7–23
7.5.22 getfield_<t>_w 7–25
7.5.23 getstatic_<t> 7–26
7.5.24 goto 7–27
7.5.25 goto_w 7–28
7.5.26 i2b 7–29
7.5.27 i2s 7–29
7.5.28 iadd 7–30
7.5.29 iaload 7–31
7.5.30 iand 7–32
7.5.31 iastore 7–32
7.5.32 icmp 7–33
7.5.33 iconst_<i> 7–34
7.5.34 idiv 7–34
7.5.35 if_acmp<cond> 7–35
7.5.36 if_acmp<cond>_w 7–36
7.5.37 if_scmp<cond> 7–37
7.5.38 if_scmp<cond>_w 7–38
7.5.39 if<cond> 7–39
7.5.40 if<cond>_w 7–40

7.5.41 ifnonnull 7–41
7.5.42 ifnonnull_w 7–41
7.5.43 ifnull 7–42
7.5.44 ifnull_w 7–43
7.5.45 iinc 7–43
7.5.46 iinc_w 7–44
7.5.47 iipush 7–45
7.5.48 iload 7–46
7.5.49 iload_<n> 7–46
7.5.50 ilookupswitch 7–47
7.5.51 imul 7–49
7.5.52 ineg 7–49
7.5.53 instanceof 7–50
7.5.54 invokeinterface 7–52
7.5.55 invokespecial 7–54
7.5.56 invokestatic 7–55
7.5.57 invokevirtual 7–56
7.5.58 ior 7–58
7.5.59 irem 7–58
7.5.60 ireturn 7–59
7.5.61 ishl 7–60
7.5.62 ishr 7–61
7.5.63 istore 7–61
7.5.64 istore_<n> 7–62
7.5.65 isub 7–63
7.5.66 itableswitch 7–63
7.5.67 iushr 7–65
7.5.68 ixor 7–66

7.5.69 jsr 7–66
7.5.70 new 7–67
7.5.71 newarray 7–68
7.5.72 nop 7–69
7.5.73 pop 7–69
7.5.74 pop2 7–70
7.5.75 putfield_<t> 7–70
7.5.76 putfield_<t>_this 7–72
7.5.77 putfield_<t>_w 7–74
7.5.78 putstatic_<t> 7–75
7.5.79 ret 7–77
7.5.80 return 7–78
7.5.81 s2b 7–78
7.5.82 s2i 7–79
7.5.83 sadd 7–79
7.5.84 saload 7–80
7.5.85 sand 7–81
7.5.86 sastore 7–81
7.5.87 sconst_<s> 7–82
7.5.88 sdiv 7–83
7.5.89 sinc 7–83
7.5.90 sinc_w 7–84
7.5.91 sipush 7–85
7.5.92 sload 7–85
7.5.93 sload_<n> 7–86
7.5.94 slookupswitch 7–87
7.5.95 smul 7–88
7.5.96 sneg 7–88

7.5.97 sor 7–89

7.5.98 srem 7–90

7.5.99 sreturn 7–90

7.5.100 sshl 7–91

7.5.101 sshr 7–92

7.5.102 sspush 7–93

7.5.103 sstore 7–93

7.5.104 sstore_<n> 7–94

7.5.105 ssub 7–94

7.5.106 stableswitch 7–95

7.5.107 sushr 7–96

7.5.108 swap_x 7–97

7.5.109 sxor 7–98

8. Tables of Instructions 8–1

Glossary Glossary–1

Index Index–1

Figures

FIGURE 1-1 Java Card API Package Conversion 1–3

FIGURE 1-2 Java Card API Package Installation 1–3

FIGURE 4-1 Mapping Package Identifiers To AIDs 4–4

FIGURE 4-2 Binary Compatibility Example 4–11

Tables

TABLE 2-1 Unsupported Keywords 2–4

TABLE 2-2 Supported Keywords 2–7

TABLE 2-3 Unsupported Java Constant Pool Tags 2–14

TABLE 2-4 Supported Java Constant Pool Tags 2–15

TABLE 2-5 Unsupported Bytecodes 2–17

TABLE 2-6 Supported Bytecodes 2–17

TABLE 2-7 Support of Java Checked Exceptions 2–20

TABLE 2-8 Support of Java Runtime Exceptions 2–21

TABLE 2-9 Support of Java Errors 2–21

TABLE 3-1 Type Support in the Java Card Virtual Machine Instruction Set 3–5

TABLE 3-2 Storage Types and Computational Types 3–7

TABLE 4-1 AID Format 4–4

TABLE 4-2 Token Range, Type and Scope 4–7

TABLE 4-3 Tokens For Instance Fields 4–9

TABLE 5-1 Export File Constant Pool Tags 5–5

TABLE 5-2 Export File Package Flags 5–5

TABLE 5-3 Export File Class Access and Modifier Flags 5–9

TABLE 5-4 Export File Field Access and Modifier Flags 5–12

TABLE 5-5 Export File Method Access and Modifier Flags 5–14

TABLE 6-1 CAP File Component Tags 6–2

TABLE 6-2 CAP File Component File Names 6–3
TABLE 6-3 Reference Component Install Order 6–5
TABLE 6-4 CAP File Package Flags 6–7
TABLE 6-5 CAP File Constant Pool Tags 6–15
TABLE 6-6 Type Descriptor Values 6–24
TABLE 6-7 Encoded Reference Type p1.c1 6–24
TABLE 6-8 Encoded Byte Array Type 6–25
TABLE 6-9 Encoded Reference Array Type p1.c1 6–25
TABLE 6-10 Encoded Method Signature ()V 6–25
TABLE 6-11 Encoded Method Signature (Lp1.ci;)S 6–25
TABLE 6-12 CAP File Interface and Class Flags 6–27
TABLE 6-13 CAP File Method Flags 6–39
TABLE 6-14 Segments of a Static Field Image 6–41
TABLE 6-15 Static Field Sizes 6–42
TABLE 6-16 Array Types 6–43
TABLE 6-17 One-byte Reference Location Example 6–46
TABLE 6-18 CAP File Class Descriptor Flags 6–51
TABLE 6-19 CAP File Field Descriptor Flags 6–53
TABLE 6-20 Primitive Type Descriptor Values 6–54
TABLE 6-21 CAP File Method Descriptor Flags 6–55
TABLE 6-22 Class Access and Modifier Flags 6–59
TABLE 6-23 Field Access and Modifier Flags 6–61
TABLE 6-24 Method Modifier Flags 6–63
TABLE 7-1 Example Instruction Page 7–4
TABLE 7-2 Array Values 7–18
TABLE 7-3 Array Values 7–51
TABLE 7-4 Array Values 7–68
TABLE 8-1 Instructions by Opcode Value 8–1
TABLE 8-2 Instructions by Opcode Mnemonic 8–4

Preface

Java Card™ technology combines a subset of the Java™ programming language with a runtime environment optimized for smart cards and similar small-memory embedded devices. The goal of Java Card technology is to bring many of the benefits of Java software programming to the resource-constrained world of devices such as smart cards.

The Java Card platform is defined by three specifications: this *Virtual Machine Specification for the Java Card Platform, Version 2.2.2*, the *Application Programming Interface for the Java Card Platform, Version 2.2.2*, and the *Runtime Environment Specification for the Java Card Platform, Version 2.2.2*.

This specification describes the required behavior of the virtual machine (VM) for the Java Card platform ("Java Card virtual machine" or "Java Card VM"), version 2.2.2, that developers should adhere to when creating an implementation. An implementation within the context of this document refers to a licensee's implementation of the Java Card virtual machine, Application Programming Interface (API), Converter, or other component, based on the Java Card technology specifications. A "reference implementation" is an implementation produced by Sun Microsystems, Inc. Application software written for the Java Card platform is referred to as a Java Card technology-based applet ("Java Card applet" or "card applet").

Who Should Use This Specification

This document is for licensees of the Java Card technology to assist them in creating an implementation, developing a specification to extend the Java Card technology specifications, or in creating an extension to the runtime environment for the Java Card platform ("Java Card Runtime Environment" or "Java Card RE"). This document is also intended for Java Card applet developers who want a more detailed understanding of the Java Card technology specifications.

Before You Read This Specification

Before reading this document, you should be familiar with the Java programming language, the Java Card technology specifications, and smart card technology. A good resource for becoming familiar with Java technology and Java Card technology is the Sun Microsystems, Inc. web site, located at

`http://java.sun.com`

How This Book Is Organized

Chapter 1, “Introduction” provides an overview of the Java Card virtual machine architecture.

Chapter 2, “A Subset of the Java Virtual Machine” describes the subset of the Java programming language and Java Virtual Machine[1] that is supported by the Java Card specification.

Chapter 3, “Structure of the Java Card Virtual Machine” describes the differences between the Java virtual machine and the Java Card virtual machine.

Chapter 4, “Binary Representation” provides information about how programs written for the Java Card platform are represented in binary form.

Chapter 5, “The Export File Format” describes the export file used to link code against another package.

Chapter 6, “The CAP File Format” describes the format of the CAP file.

Chapter 7, “Java Card Virtual Machine Instruction Set” describes the bytecodes (opcodes) that comprise the Java Card virtual machine instruction set.

Chapter 8, “Tables of Instructions” summarizes the Java Card virtual machine instructions in two different tables: one sorted by Opcode Value and the other sorted by Mnemonic.

“Glossary” provides definitions of selected terms in this specification.

1. The terms "Java Virtual Machine" and "JVM" mean a Virtual Machine for the Java platform.

Prerequisites

This specification is not intended to stand on its own; rather it relies heavily on existing documentation of the Java platform. In particular, two books are required for the reader to understand the material presented here.

The Java Language Specification by James Gosling, Bill Joy, and Guy L. Steele (Addison-Wesley, 1996) ISBN 0-201-31008-2, contains the definitive definition of the Java programming language. The Java Card platform, version 2.2.2, language subset defined here is based on the language specified in this book.

The Java Virtual Machine Specification (Second Edition) by Tim Lindholm and Frank Yellin. (Addison-Wesley, 1999) ISBN 0-201-43294-3, defines the standard operation of the Java virtual machine. The Java Card virtual machine presented here is based on the definition specified in this book.

Related Documents

References to various documents or products are made in this manual. You should have the following documents available:

- *Application Programming Interface for the Java Card Platform, Version 2.2.2*.
- *Runtime Environment Specification for the Java Card Platform, Version 2.2.2*.
- *The Java Language Specification* by James Gosling, Bill Joy, and Guy L. Steele (Addison-Wesley, 1996).
- *The Java Virtual Machine Specification* (Second Edition) by Tim Lindholm and Frank Yellin (Addison-Wesley, 1999).
- *The Java Class Libraries: An Annotated Reference, Second Edition (Java Series)* by Patrick Chan, Rosanna Lee and Doug Kramer (Addison-Wesley, 1999).
- *The Java Remote Method Invocation Specification*, Revision 1.7, Sun Microsystems, Inc.
- *ISO 7816 International Standard*, First Edition 1987-07-01, (`http://www.iso.org`).

Typographic Conventions

TABLE P-1

Typeface	Meaning	Examples
`AaBbCc123`	The names of commands, files, and directories; on-screen computer output	Edit your `.login` file. Use `ls -a` to list all files. `% You have mail.`
`AaBbCc123`	What you type, when contrasted with on-screen computer output	`% su` `Password:`
	Procedural steps	**1. Run `cref` in a new window.**
AaBbCc123	Book titles, new words or terms, words to be emphasized	Read Chapter 6 in the *User's Guide*. These are called *class* options. You *must* be superuser to do this.
	Command-line variable; replace with a real name or value	To delete a file, type `rm` *filename*.

Acknowledgements

Java Card technology is based on Java technology. This specification could not exist without all the hard work that went into the development of the Java platform specifications. In particular, this specification is based significantly on *Java Virtual Machine Specification*. In order to maintain consistency with that specification, as well as to make differences easier to notice, we have, where possible, used the words, the style, and even the visual design of that book. Many thanks to Tim Lindholm and Frank Yellin for providing a solid foundation for our work.

Accessing Sun Documentation Online

The Java Developer Connection™ program web site enables you to access Java platform technical documentation on the web at

`http://developer.java.sun.com/developer/infodocs`

Sun Welcomes Your Comments

We are interested in improving our documentation and welcome your comments and suggestions. You can email your comments to us at `docs@java.sun.com`.

CHAPTER 1

Introduction

1.1 Motivation

Java Card technology enables programs written in the Java programming language to be run on smart cards and other small, resource-constrained devices. Developers can build and test programs using standard software development tools and environments, then convert them into a form that can be installed onto a Java Card technology-enabled device. Application software for the Java Card platform is called an applet, or more specifically, a Java Card applet or card applet (to distinguish it from browser applets).

While Java Card technology enables programs written in the Java programming language to run on smart cards, such small devices are far too under-powered to support the full functionality of the Java platform. Therefore, the Java Card platform supports only a carefully chosen, customized subset of the features of the Java platform. This subset provides features that are well-suited for writing programs for small devices and preserves the object-oriented capabilities of the Java programming language.

A simple approach to specifying a Java Card virtual machine would be to describe the subset of the features of the Java virtual machine that must be supported to allow for portability of source code across all Java Card technology enabled devices. Combining that subset specification and the information in *Java Virtual Machine Specification*, smart card manufacturers could construct their own Java Card technology-based implementations ("Java Card implementations"). While that approach is feasible, it has a serious drawback. The resultant platform would be missing the important feature of binary portability of Java Card applets.

The standards that define the Java platform allow for binary portability of Java programs across all Java platform implementations. This "write once, run anywhere" quality of Java programs is perhaps the most significant feature of the

platform. Part of the motivation for the creation of the Java Card platform was to bring just this kind of binary portability to the smart card industry. In a world with hundreds of millions or perhaps even billions of smart cards with varying processors and configurations, the costs of supporting multiple binary formats for software distribution could be overwhelming.

This *Virtual Machine Specification for the Java Card Platform, Version 2.2.2* is the key to providing binary portability. One way of understanding what this specification does is to compare it to its counterpart in the Java platform. The Java virtual machine specification defines a Java virtual machine as an engine that loads Java class files and executes them with a particular set of semantics. The class file is a central piece of the Java architecture, and it is the standard for the binary compatibility of the Java platform. The *Virtual Machine Specification for the Java Card Platform, Version 2.2.2* also defines a file format that is the standard for binary compatibility for the Java Card platform: the CAP file format is the form in which software is loaded onto devices which implement a Java Card virtual machine.

1.2 The Java Card Virtual Machine

The role of the Java Card virtual machine is best understood in the context of the process for production and deployment of software for the Java Card platform. There are several components that make up a Java Card system, including the Java Card virtual machine, the Converter for the Java Card platform ("Java Card Converter"), a terminal installation tool, and an installation program that runs on the device, as shown in FIGURE 1-1 and FIGURE 1-2.

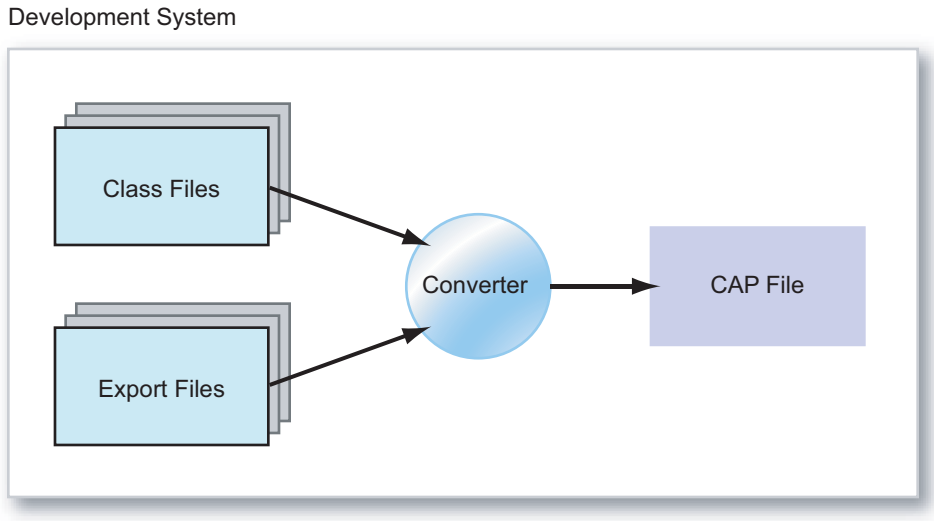


FIGURE 1-1 Java Card API Package Conversion

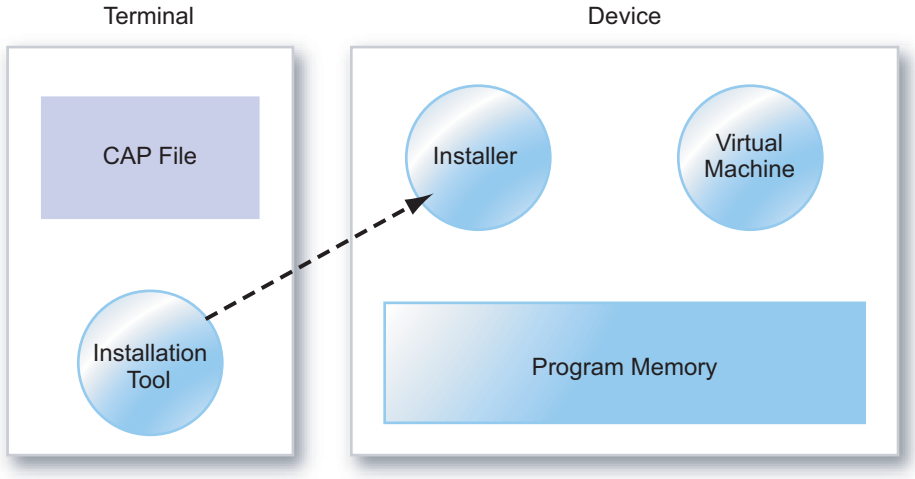


FIGURE 1-2 Java Card API Package Installation

Development of a Java Card applet begins as with any other Java program: a developer writes one or more Java classes, and compiles the source code with a Java compiler, producing one or more class files. The applet is run, tested and debugged on a workstation using simulation tools to emulate the device environment. Then, when an applet is ready to be downloaded to a device, the class files comprising the applet are converted to a CAP (converted applet) file using a Java Card Converter.

The Java Card Converter takes as input all of the class files which make up a Java package. A package that contains one or more non-abstract subclasses, direct or indirect, of the javacard.framework.Applet class is referred to as an applet package. Otherwise the package is referred to as a library package. The Java Card Converter also takes as input one or more export files. An export file contains name and link information for the contents of other packages that are imported by the classes being converted. When an applet or library package is converted, the converter can also produce an export file for that package.

After conversion, the CAP file is copied to a card terminal, such as a desktop computer with a card reader peripheral. Then an installation tool on the terminal loads the CAP file and transmits it to the Java Card technology-enabled device. An installation program on the device receives the contents of the CAP file and prepares the applet to be run by the Java Card virtual machine. The virtual machine itself need not load or manipulate CAP files; it need only execute the applet code found in the CAP file that was loaded onto the device by the installation program.

The division of functionality between the Java Card virtual machine and the installation program keeps both the virtual machine and the installation program small. The installation program may be implemented as a Java program and executed on top of the Java Card virtual machine. Since instructions for the Java Card platform ("Java Card instructions") are denser than typical machine code, this may reduce the size of the installer. The modularity may enable different installers to be used with a single Java Card virtual machine implementation.

1.3 Java Language Security

One of the fundamental features of the Java virtual machine is the strong security provided in part by the class file verifier. Many devices that implement the Java Card platform may be too small to support verification of CAP files on the device itself. This consideration led to a design that enables verification on a device but does not rely on it. The data in a CAP file that is needed only for verification is packaged separately from the data needed for the actual execution of its applet. This allows for flexibility in how security is managed in an implementation.

There are several options for providing language-level security on a Java Card technology-enabled device. The conceptually simplest is to verify the contents of a CAP file on the device as it is downloaded or after it is downloaded. This option might only be feasible in the largest of devices. However, some subset of verification might be possible even on smaller devices. Other options rely on some combination of one or more of: physical security of the installation terminal, a cryptographically enforced chain of trust from the source of the CAP file, and pre-download verification of the contents of a CAP file.

The Java Card platform standards say as little as possible about CAP file installation and security policies. Since smart cards must serve as secure processors in many different systems with different security requirements, it is necessary to allow a great deal of flexibility to meet the needs of smart card issuers and users.

1.4 Java Card Runtime Environment Security

The standard runtime environment for the Java Card platform is the Java Card Runtime Environment. The Java Card RE consists of an implementation of the Java Card virtual machine along with the Java Card API classes. While the Java Card virtual machine has responsibility for ensuring Java language-level security, the Java Card RE imposes additional runtime security requirements on devices that implement the Java Card RE, which results in a need for additional features on the Java Card virtual machine. Throughout this document, these additional features are designated as Java Card RE-specific.

The basic runtime security feature imposed by the Java Card RE enforces isolation of applets using what is called an applet firewall. The applet firewall prevents the objects that were created by one applet from being used by another applet. This prevents unauthorized access to both the fields and methods of class instances, as well as the length and contents of arrays.

Isolation of applets is an important security feature, but it requires a mechanism to allow applets to share objects in situations where there is a need to interoperate. The Java Card RE allows such sharing using the concept of shareable interface objects. These objects provide the only way an applet can make its objects available for use by other applets. For more information about using shareable interface objects, see the description of the interface javacard.framework.Shareable in the *Application Programming Interface, Java Card Platform, Version 2.2.2* specification. Some descriptions of firewall-related features make reference to the Shareable interface.

The applet firewall also protects from unauthorized use the objects owned by the Java Card RE itself. The Java Card RE can use mechanisms not reflected in the Java Card API to make its objects available for use by applets. A full description of the Java Card RE-related isolation and sharing features can be found in the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

CHAPTER 2

A Subset of the Java Virtual Machine

This chapter describes the subset of the Java virtual machine and language that is supported in the Java Card platform, Version 2.2.2.

2.1 Why a Subset is Needed

It would be ideal if programs for smart cards could be written using all of the Java programming language, but a full implementation of the Java virtual machine is far too large to fit on even the most advanced resource-constrained devices available today.

A typical resource-constrained device has on the order of 1.2K of RAM, 16K of non-volatile memory (EEPROM or flash) and 32K-48K of ROM. The code for implementing string manipulation, single and double-precision floating point arithmetic, and thread management would be larger than the ROM space on such a device. Even if it could be made to fit, there would be no space left over for class libraries or application code. RAM resources are also very limited. The only workable option is to implement Java Card technology as a subset of the Java platform.

2.2 Java Card Platform Language Subset

Applets written for the Java Card platform are written in the Java programming language. They are compiled using Java compilers. Java Card technology uses a subset of the Java language, and familiarity with the Java platform is required to understand the Java Card platform.

The items discussed in this section are not described to the level of a language specification. For complete documentation on the Java programming language, see *The Java Language Specification*.

2.2.1 Unsupported Items

The items listed in this section are elements of the Java programming language and platform that are not supported by the Java Card platform.

2.2.1.1 Unsupported Features

Dynamic Class Loading

Dynamic class loading is not supported in the Java Card platform. An implementation of the Java Card platform is not able to load classes dynamically. Classes are either masked into the card during manufacturing or downloaded through an installation process after the card has been issued. Programs executing on the card may only refer to classes that already exist on the card, since there is no way to download classes during the normal execution of application code.

Security Manager

Security management in the Java Card platform differs significantly from that of the Java platform. In the Java platform, there is a Security Manager class (`java.lang.SecurityManager`) responsible for implementing security features. In the Java Card platform, language security policies are implemented by the virtual machine. There is no Security Manager class that makes policy decisions on whether to allow operations.

Finalization

Finalization is also not supported. `finalize()` will not be called automatically by the Java Card virtual machine.

Threads

The Java Card virtual machine does not support multiple threads of control. Programs for the Java Card platform ("Java Card programs") cannot use class `Thread` or any of the thread-related keywords in the Java programming language.

Cloning

The Java Card platform does not support cloning of objects. Java Card API class `Object` does not implement a `clone` method, and there is no `Cloneable` interface provided.

Access Control in Java Packages

The Java Card platform language subset supports the package access control defined in the Java language. However, the cases that are not supported are as follows.

- If a class implements a method with package access visibility, a subclass cannot override the method and change the access visibility of the method to protected or public.
- A public class cannot contain a public or protected field of type reference to a package-visible class.
- A public class cannot contain a public or protected method with a return type of type reference to a package-visible class.
- A public or protected method in a public class cannot contain a formal parameter of type reference to a package-visible class.
- A package-visible class that is extended by a public class cannot define any public or protected methods or fields.
- A package-visible interface that is implemented by a public class cannot define any fields.
- A package-visible interface cannot be extended by an interface with public access visibility.

Typesafe Enums

The Java Card platform language subset does not support the enumerated type facility and the keyword *enum*.

Enhanced for Loop

The Java Card platform language subset does not support the *enhanced for loop* language construct. Support for the *enhanced for loop* construct requires support for array indexing using the integer data type. The Java Card platform only supports array indexing using the short data type.

Varargs

The Java Card platform language subset does not support variable-length argument lists. The variable-length argument construct requires the compiler to generate code that creates a new array object each time a variable-length argument array method is invoked, thereby causing implicit memory allocations in Java Card runtime memory heap.

Runtime Visible Metadata (Annotations)

The Java Card platform does not support this language feature which lets you introduce meta-data information into the runtime environment to be accessed reflectively. The Java Card platform does not support reflection.

Assertions

The Java Card runtime does not provide runtime support for statements in the Java programming language called *assertions* that are used to test assumptions about program functionality.

2.2.1.2 Keywords

The following keywords indicate unsupported options related to native methods, threads, floating point, memory management, and debugging.

TABLE 2-1 Unsupported Keywords

`native`	`synchronized`	`transient`	`volatile`
`strictfp`	`enum`	`assert`	

2.2.1.3 Unsupported Types

The Java Card platform does not support types `char`, `double`, `float` and `long`. It also does not support arrays of more than one dimension.

2.2.1.4 Classes

In general, none of the Java programming language core API classes are supported in the Java Card platform. Some classes from the `java.lang` package are supported (see Section 2.2.2.4, "Classes" on page 2-7), but none of the rest are. For example, classes that are not supported are `String`, `Thread` (and all thread-related classes), wrapper classes such as `Boolean` and `Integer`, and class `Class`.

System

Class `java.lang.System` is not supported. Java Card technology supplies a class `javacard.framework.JCSystem`, which provides an interface to system behavior.

2.2.2 Supported Items

If a language feature is not explicitly described as unsupported, it is part of the supported subset. Notable supported features are described in this section.

2.2.2.1 Features

Packages

Software written for the Java Card platform follows the standard rules for the Java platform packages. Java Card API classes are written as Java source files, which include package designations. Package mechanisms are used to identify and control access to classes, static fields and static methods. Except as noted in "Access Control in Java Packages" (Section 2.2.1.1, "Unsupported Features" on page 2-2), packages in the Java Card platform are used exactly the way they are in the Java platform.

Dynamic Object Creation

The Java Card platform programs supports dynamically created objects, both class instances and arrays. This is done, as usual, by using the `new` operator. Objects are allocated out of the heap.

A Java Card virtual machine will not necessarily garbage collect objects. Any object allocated by a virtual machine may continue to exist and consume resources even after it becomes unreachable. See Section 2.2.3.2, "Object Deletion Mechanism" on page 2-8 for more information regarding support for an optional object deletion mechanism.

Virtual Methods

Since Java Card technology-based objects ("Java Card objects") are Java programming language objects, invoking virtual methods on objects in a program written for the Java Card platform is exactly the same as in a program written for the Java platform. Inheritance is supported, including the use of the `super` keyword.

Interfaces

Java Card API classes may define or implement interfaces as in the Java programming language. Invoking methods on interface types works as expected. Type checking and the `instanceof` operator also work correctly with interfaces.

Exceptions

Java Card programs may define, throw and catch exceptions, as in Java programs. Class `Throwable` and its relevant subclasses are supported. Some `Exception` and `Error` subclasses are omitted, since those exceptions cannot occur in the Java Card platform. See Section 2.3.3, "Exceptions" on page 2-19 for specification of errors and exceptions.

Generics

This Java language facility allows a type or method to operate on objects of various types while providing compile-time type safety. It adds compile-time type safety and eliminates the need for casting.

Static Import

This Java language facility lets you avoid importing an entire class simply to access its static members or qualifying static members with class names each time it is used.

Runtime Invisible Metadata (Annotations)

This language feature lets you avoid writing boilerplate code under many circumstances by enabling tools to generate it from annotations in the source code. The Java Card platform language subset supports the use of annotations which are not visible at runtime. These annotations do not themselves use the runtime visible meta-data annotation `@Retention(RetentionPolicy.RUNTIME)`.

2.2.2.2 Keywords

The following keywords are supported. Their use is the same as in the Java programming language.

TABLE 2-2 Supported Keywords

`abstract`	`default`	`if`	`private`	`this`
`boolean`	`do`	`implements`	`protected`	`throw`
`break`	`else`	`import`	`public`	`throws`
`byte`	`extends`	`instanceof`	`return`	`try`
`case`	`final`	`int`	`short`	`void`
`catch`	`finally`	`interface`	`static`	`while`
`class`	`for`	`new`	`super`	
`continue`	`goto`	`package`	`switch`	

2.2.2.3 Types

Java programming language types `boolean`, `byte`, `short`, and `int` are supported. Objects (class instances and single-dimensional arrays) are also supported. Arrays can contain the supported primitive data types, objects, and other arrays.

Some Java Card implementations might not support use of the `int` data type. (Refer to Section 2.2.3.1, "Integer Data Type" on page 2-8).

2.2.2.4 Classes

Most of the classes in the `java.lang` package are not supported on the Java Card platform. The following classes from `java.lang` are supported on the card in a limited form.

Object

Java Card API classes descend from `java.lang.Object`, just as in the Java programming language. Most of the methods of `Object` are not available in the Java Card API, but the class itself exists to provide a root for the class hierarchy.

Throwable

Class `Throwable` and its subclasses are supported. Most of the methods of `Throwable` are not available in the Java Card API, but the class itself exists to provide a common ancestor for all exceptions.

2.2.3 Optionally Supported Items

This section describes the optional features of the Java Card platform. An optional feature is not required to be supported in a Java Card platform-compatible implementation. However, if an implementation does include support for an optional feature, it must be supported fully, and exactly as specified in this document.

2.2.3.1 Integer Data Type

The `int` keyword and 32-bit integer data type need not be supported in a Java Card implementation. A Java Card virtual machine that does not support the `int` data type will reject programs which use the `int` data type or 32-bit intermediate values.

The result of an arithmetic expression produced by a Java Card virtual machine must be equal to the result produced by a Java virtual machine, regardless of the input values. A Java Card virtual machine that does not support the `int` data type must reject expressions that could produce a different result.

2.2.3.2 Object Deletion Mechanism

Java Card technology, version 2.2.2 offers an optional, object deletion mechanism. Applications designed to run on these implementations can use the facility by invoking the appropriate API. See *Application Programming Interface, Java Card Platform, Version 2.2.2*. But, the facility is best suited for updating large objects such as certificates and keys atomically. Therefore, application programmers should conserve on the allocation of objects.

2.2.4 Limitations of the Java Card Virtual Machine

The limitations of resource-constrained hardware prevent Java Card virtual machines from supporting the full range of functionality of certain Java platform features. The features in question are supported, but a particular virtual machine may limit the range of operation to less than that of the Java platform.

To ensure a level of portability for application code, this section establishes a minimum required level for partial support of these language features.

The limitations here are listed as maximums from the application programmer's perspective. Java packages that do not violate these maximum values can be converted into Java Card technology-based `CAP` files ("Java Card `CAP` files"), and will be portable across all Java Card implementations. From the Java Card virtual machine implementer's perspective, each maximum listed indicates a minimum level of support that will allow portability of applets.

2.2.4.1 Packages

Package References

A package can reference at most 128 other packages.

Package Name

The fully qualified name of a package may contain a maximum of 255 characters. The package name size is further limited if it contains one or more characters which, when represented in UTF-8 format, requires multiple bytes.

2.2.4.2 Classes

Classes in a Package

A package can contain at most 255 classes and interfaces.

Interfaces

A class can implement at most 15 interfaces, including interfaces implemented by superclasses.

An interface can inherit from at most 14 superinterfaces.

Static Fields

A class in an applet package can have at most 256 public or protected static non-final fields. A class in a library package can have at most 255 public or protected static non-final fields. There is no limit on the number of static final fields (constants) declared in a class.

Static Methods

A class in an applet package can have at most 256 public or protected static methods. A class in a library package can have at most 255 public or protected static methods.

2.2.4.3 Objects

Methods

A class can implement a maximum of 128 public or protected instance methods, and a maximum of 128 instance methods with package visibility. These limits include inherited methods.

Class Instances

Class instances can contain a maximum of 255 fields, where an int data type is counted as occupying two fields. These limits include inherited fields.

Arrays

Arrays can hold a maximum of 32767 components.

2.2.4.4 Methods

The maximum number of variables that can be used in a method is 255. This limit includes local variables, method parameters, and, in the case of an instance method invocation, a reference to the object on which the instance method is being invoked (meaning, `this`). An `int` data type is counted as occupying two local variables.

A method can have at most 32767 Java Card virtual machine bytecodes. The number of Java Card technology-based bytecodes ("Java Card bytecodes") may differ from the number of Java bytecodes in the Java virtual machine implementation of that method.

The maximum depth of an operand stack associated with a method is 255 16-bit cells.

2.2.4.5 Switch Statements

The format of the Java Card virtual machine switch instructions limits switch statements to a maximum of 65536 cases. This limit is far greater than the limit imposed by the maximum size of methods (Section 2.2.4.4, "Methods" on page 2-10).

2.2.4.6 Class Initialization

The Java Card virtual machine contains limited support for class initialization because there is no general mechanism for executing `<clinit>` methods. Support for `<clinit>` methods is limited to the initialization of static field values with the following constraints:

- Static fields of applet packages may only be initialized to primitive compile-time constant values, or arrays of primitive compile-time constants.
- Static fields of user libraries may only be initialized to primitive compile-time constant values.
- Only static fields declared in the current class may be initialized in the `<clinit>` method.

Primitive constant data types include `boolean`, `byte`, `short`, and `int`.

Given Java technology source files that adhere to these language-level constraints on static field initialization, it is expected that reasonable Java compilers will:

- Inline constants in the bytecodes that reference static final primitive fields that are initialized in the declaration statement.
- Produce only the following bytecodes:
  - load a value on the stack: `iconst_[m1,0-5]`, `[b|s]ipush`, `ldc`, `ldc_w`, `aconst_null`
  - create an array: `newarray([byte|short|boolean|int])`
  - duplicate items on the stack: `dup`
  - store values in arrays or static fields: `[b|i|s]astore`, `putstatic`
  - return from method: `return`

2.2.5 Multiselectable Applets Restrictions

Applets that implement the `javacard.framework.Multiselectable` interface are called multiselectable applets. For more details on multiselection, please see the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

All applets within a package shall be multiselectable, or none shall be.

2.2.6 Java Card Platform Remote Method Invocation (RMI) Restrictions

This section defines the subset of the RMI system that is supported by Java Card platform RMI ("Java Card RMI").

2.2.6.1 Remote Classes and Remote Interfaces

A class is remote if it or any of its superclasses implements a remote interface.

A remote interface is an interface which satisfies the following requirements:

- The interface name is `java.rmi.Remote` or the interface extends, directly or indirectly, the interface `java.rmi.Remote`.
- Each method declaration in the remote interface or its super-interfaces includes the exception `java.rmi.RemoteException` (or one of its superclasses) in its `throws` clause.
- In a remote method declaration, if a remote object is declared as a return type, it is declared as the remote interface, not the implementation class of that interface.

In addition, Java Card RMI imposes additional constraints on the definition of remote methods. These constraints are as a result of the Java Card platform language subset and other feature limitations. For more information, see Section 2.2.6.2, "Access Control of Remote Interfaces" on page 2-12 and Section 2.2.6.3, "Parameters and Return Values" on page 2-13.

2.2.6.2 Access Control of Remote Interfaces

The Java RMI system supports the package access control defined in the Java language. However, Java Card RMI does not support package-visible remote interfaces.

2.2.6.3 Parameters and Return Values

The parameters of a remote method must only include parameters of the following types:

- any primitive type supported by Java Card technology (`boolean`, `byte`, `short`, `int`),
- any single-dimension array type of an primitive type supported by Java Card technology (`boolean[]`, `byte[]`, `short[]`, `int[]`).

The return type of a remote method must only be one of the following types:

- any primitive type supported by Java Card technology (`boolean`, `byte`, `short`, `int`),
- any single-dimension array type of an primitive type supported by Java Card technology (`boolean[]`, `byte[]`, `short[]`, `int[]`),
- any remote interface type
- type `void`

2.3 Java Card VM Subset

Java Card technology uses a subset of the Java virtual machine, and familiarity with the Java platform is required to understand the Java Card virtual machine.

The items discussed in this section are not described to the level of a virtual machine specification. For complete documentation on the Java virtual machine, refer to the *The Java Virtual Machine Specification*.

2.3.1 class File Subset

The operation of the Java Card virtual machine can be defined in terms of standard Java platform class files. Since the Java Card virtual machine supports only a subset of the behavior of the Java virtual machine, it also supports only a subset of the standard class file format.

2.3.1.1 Not Supported in Class Files

Class access flags

In `class_info` and `interface_info` structures, the access flag `ACC_ENUM` is not supported.

Field Descriptors

Field descriptors may not contain `BaseType` characters `C`, `D`, `F` or `J`. `ArrayType` descriptors for arrays of more than one dimension may not be used.

Constant Pool

Constant pool table entries with the following `tag` values are not supported.

TABLE 2-3 Unsupported Java Constant Pool Tags

Constant Type	Value
`CONSTANT_String`	8
`CONSTANT_Float`	4
`CONSTANT_Long`	5
`CONSTANT_Double`	6

Fields

In `field_info` structures, the access flags `ACC_VOLATILE`, `ACC_TRANSIENT` and `ACC_ENUM` are not supported.

Methods

In `method_info` structures, the access flags `ACC_SYNCHRONIZED`, `ACC_STRICT`, `ACC_NATIVE`, and `ACC_VARARGS` are not supported.

2.3.1.2 Supported in Class Files

ClassFile

All items in the `ClassFile` structure are supported.

Field Descriptors

Field descriptors may contain BaseType characters `B`, `I`, `S` and `Z`, as well as any `ObjectType`. `ArrayType` descriptors for arrays of a single dimension may also be used.

Method Descriptors

All forms of method descriptors are supported.

Constant pool

Constant pool table entry with the following tag values are supported.

TABLE 2-4 Supported Java Constant Pool Tags

Constant Type	Value
`CONSTANT_Class`	7
`CONSTANT_Fieldref`	9
`CONSTANT_Methodref`	10
`CONSTANT_InterfaceMethodref`	11
`CONSTANT_Integer`	3
`CONSTANT_NameAndType`	12
`CONSTANT_Utf8`	1

Fields

In `field_info` structures, the supported access flags are `ACC_PUBLIC`, `ACC_PRIVATE`, `ACC_PROTECTED`, `ACC_STATIC` and `ACC_FINAL`.

The remaining components of `field_info` structures are fully supported.

Methods

In `method_info` structures, the supported access flags are `ACC_PUBLIC`, `ACC_PRIVATE`, `ACC_PROTECTED`, `ACC_STATIC`, `ACC_FINAL` and `ACC_ABSTRACT`.

The remaining components of `method_info` structures are fully supported.

Attributes

The `attribute_info` structure is supported. The `Code`, `ConstantValue`, `Exceptions`, `LocalVariableTable`, `Synthetic`, `InnerClasses`, `RuntimeInvisibleAnnotations`, `RuntimeInvisibleParameterAnnotations` and `Deprecated` attributes are supported.

2.3.2 Bytecode Subset

The following sections detail the bytecodes that are either supported or unsupported in the Java Card platform. For more details, refer to Chapter 7, "Java Card Virtual Machine Instruction Set".

2.3.2.1 Unsupported Bytecodes

TABLE 2-5 Unsupported Bytecodes

lconst_<l>	fconst_<f>	dconst_<d>	ldc2_w2
lload	fload	dload	lload_<n>
fload_<n>	dload_<n>	laload	faload
daload	caload	lstore	fstore
dstore	lstore_<n>	fstore_<n>	dstore_<n>
lastore	fastore	dastore	castore
ladd	fadd	dadd	lsub
fsub	dsub	lmul	fmul
dmul	ldiv	fdiv	ddiv
lrem	frem	drem	lneg
fneg	dneg	lshl	lshr
lushr	land	lor	lxor
i2l	i2f	i2d	l2i
l2f	l2d	f2i	f2d
d2i	d2l	d2f	i2c
lcmp	fcmpl	fcmpg	dcmpl
dcmpg	lreturn	freturn	dreturn
monitorenter	monitorexit	multianewarray	goto_w
jsr_w			

2.3.2.2 Supported Bytecodes

TABLE 2-6 Supported Bytecodes

nop	aconst_null	iconst_<i>	bipush
sipush	ldc	ldc_w	iload
aload	iload_<n>	aload_<n>	iaload
aaload	baload	saload	istore
astore	istore_<n>	astore_<n>	iastore
aastore	bastore	sastore	pop

TABLE 2-6 Supported Bytecodes

pop2	dup	dup_x1	dup_x2
dup2	dup2_x1	dup2_x2	swap
iadd	isub	imul	idiv
irem	ineg	ior	ishl
ishr	iushr	iand	ixor
iinc	i2b	i2s	if<cond>
ificmp_<cond>	ifacmp_<cond>	goto	jsr
ret	tableswitch	lookupswitch	ireturn
areturn	return	getstatic	putstatic
getfield	putfield	invokevirtual	invokespecial
invokestatic	invokeinterface	new	newarray
anewarray	arraylength	athrow	checkcast
instanceof	wide	ifnull	ifnonnull

2.3.2.3 Static Restrictions on Bytecodes

A `class` file must conform to the following restrictions on the static form of bytecodes.

ldc, ldc_w

The `ldc` and `ldc_w` bytecodes can only be used to load integer constants. The constant pool entry at index must be a `CONSTANT_Integer` entry. If a program contains an `ldc` or `ldc_w` instruction that is used to load an integer value less than -32768 or greater than 32767, that program will require the optional int instructions (Section 2.2.3.1, "Integer Data Type" on page 2-8).

lookupswitch

The value of the npairs operand must be less than 65536. This limit is far greater than the limit imposed by the maximum size of methods (Section 2.2.4.4, "Methods" on page 2-10). If a program contains a lookupswitch instruction that uses keys of type int, that program will require the optional int instructions (Section 2.2.3.1, "Integer Data Type" on page 2-8). Otherwise, key values must be in the range -32768 to 32767.

tableswitch

The bytecode can contain at most 65536 cases. This limit is far greater than the limit imposed by the maximum size of methods (Section 2.2.4.4, "Methods" on page 2-10). If a program does not use the optional int instructions (Section 2.2.3.1, "Integer Data Type" on page 2-8), the values of the `high` and `low` operands must both be at least -32768 and at most 32767.

wide

The `wide` bytecode can only be used with an `iinc` instruction.

2.3.3 Exceptions

The Java Card platform provides full support for the Java platform's exception mechanism. Users can define, throw and catch exceptions just as in the Java platform. The Java Card platform also makes use of the exceptions and errors defined in *The Java Language Specification*. An updated list of the Java platform's exceptions is provided in the JDK™ software documentation.

Not all of the Java platform's exceptions are supported in the Java Card platform. Exceptions related to unsupported features are naturally not supported. Class loader exceptions (the bulk of the checked exceptions) are not supported.

Note that some exceptions may be supported to the extent that their error conditions are detected correctly, but classes for those exceptions will not necessarily be present in the API.

The supported subset is described in the tables below.

2.3.3.1 Uncaught and Uncatchable Exceptions

In the Java platform, uncaught exceptions and errors will cause the virtual machine's current thread to exit. As the Java Card virtual machine is single-threaded, uncaught exceptions or errors will cause the virtual machine to halt. Further response to uncaught exceptions or errors after halting the virtual machine is an implementation-specific policy, and is not mandated in this document.

Some error conditions are known to be unrecoverable at the time they are thrown. Throwing a runtime exception or error that cannot be caught will also cause the virtual machine to halt. As with uncaught exceptions, implementations may take further responses after halting the virtual machine. Uncatchable exceptions and

errors which are supported by the Java Card platform may not be reflected in the Java Card API, though the Java Card platform will correctly detect the error condition.

2.3.3.2 Checked Exceptions

TABLE 2-7 Support of Java Checked Exceptions

Exception	Supported	Not Supported
ClassNotFoundException		X
CloneNotSupportedException		X
IllegalAccessException		X
InstantiationException		X
InterruptedException		X
NoSuchFieldException		X
NoSuchMethodException		X

2.3.3.3 Runtime Exceptions

TABLE 2-8 Support of Java Runtime Exceptions

Runtime Exception	Supported	Not Supported
ArithmeticException	X	
ArrayStoreException	X	
ClassCastException	X	
IllegalArgumentException		X
IllegalThreadStateException		X
NumberFormatException		X
IllegalMonitorStateException		X
IllegalStateException		X
IndexOutOfBoundsException	X	
ArrayIndexOutOfBoundsException	X	
StringIndexOutOfBoundsException		X
NegativeArraySizeException	X	
NullPointerException	X	
SecurityException	X	

2.3.3.4 Errors

TABLE 2-9 Support of Java Errors

Error	Supported	Not Supported
AssertionError		X
LinkageError	X	
ClassCircularityError	X	
ClassFormatError	X	
ExceptionInInitializerError	X	
IncompatibleClassChangeError	X	
AbstractMethodError	X	
IllegalAccessError	X	
InstantiationError	X	

TABLE 2-9 Support of Java Errors

Error	Supported	Not Supported
NoSuchFieldError	X	
NoSuchMethodError	X	
NoClassDefFoundError	X	
UnsatisfiedLinkError	X	
VerifyError	X	
ThreadDeath		X
VirtualMachineError	X	
InternalError	X	
OutOfMemoryError	X	
StackOverflowError	X	
UnknownError	X	
UnsupportedClassVersionError	X	

CHAPTER 3

Structure of the Java Card Virtual Machine

The specification of the Java Card virtual machine is in many ways quite similar to that of the Java virtual machine. This similarity is of course intentional, as the design of the Java Card virtual machine was based on that of the Java virtual machine. Rather than reiterate all the details of this specification which are shared with that of the Java virtual machine, this chapter will mainly refer to its counterpart in *The Java Virtual Machine Specification, 2nd Edition*, providing new information only where the Java Card virtual machine differs.

3.1 Data Types and Values

The Java Card virtual machine supports the same two kinds of data types as the Java virtual machine: *primitive types* and *reference types*. Likewise, the same two kinds of values are used: *primitive values* and *reference values*.

The primitive data types supported by the Java Card virtual machine are the *numeric types*, the *boolean type*, and the `returnAddress` type. The numeric types consist only of these types:

- `byte`, whose values are 8-bit signed two's complement integers
- `short`, whose values are 16-bit signed two's complement integers

Some Java Card virtual machine implementations may also support an additional integral type:

- `int`, whose values are 32-bit signed two's complement integers

Support for the boolean type is identical to that in the Java virtual machine. The value 1 is used to represent true and the value of 0 is used to represent false.

Support for `reference` types is identical to that in the Java virtual machine.

3.2 Words

The Java Card virtual machine is defined in terms of an abstract storage unit called a *word*. This specification does not mandate the actual size in bits of a word on a specific platform. A word is large enough to hold a value of type `byte`, `short`, `reference` or `returnAddress`. Two words are large enough to hold a value of type `int`.

The actual storage used for values in an implementation is platform-specific. There is enough information present in the descriptor component of a `CAP` file to allow an implementation to optimize the storage used for values in variables and on the stack.

3.3 Runtime Data Areas

The Java Card virtual machine can support only a single thread of execution. Any runtime data area in the Java virtual machine which is duplicated on a per-thread basis will have only one global copy in the Java Card virtual machine.

The Java Card virtual machine's heap is not required to be garbage collected. Objects allocated from the heap will not necessarily be reclaimed.

This specification does not include support for native methods, so there are no native method stacks.

Otherwise, the runtime data areas are as documented for the Java virtual machine.

3.4 Contexts

Each applet running on a Java Card virtual machine is associated with an execution *context*. The Java Card virtual machine uses the context of the current frame to enforce security policies for inter-applet operations.

There is a one-to-one mapping between contexts and packages in which applets are defined. An easy way to think of a context is as the runtime equivalent of a package, since Java packages are compile-time constructs and have no direct representation at runtime. As a consequence, all applet instances from the same package will share the same context.

The Java Card Runtime Environment also has its own context. Framework objects execute in this *Java Card RE context*.

The context of the currently executing method is known as the *current context*. Every object in a Java Card virtual machine is owned by a particular context. The *owning context* is the context that was current when the object was created.

When a method in one context successfully invokes a method on an object in another context, the Java Card virtual machine performs a *context switch*. Afterwards the invoked method's context becomes the current context. When the invoked method returns, the current context is switched back to the previous context.

3.5 Frames

Java Card virtual machine *frames* are very similar to those defined for the Java virtual machine. Each frame has a set of local variables and an operand stack. Frames also contain a reference to a constant pool, but since all constant pools for all classes in a package are merged, the reference is to the constant pool for the current class' package.

Each frame also includes a reference to the context in which the current method is executing.

3.6 Representation of Objects

The Java Card virtual machine does not mandate a particular internal structure for objects or a particular layout of their contents. However, the core components in a `CAP` file are defined assuming a default structure for certain runtime structures (such as descriptions of classes), and a default layout for the contents of dynamically allocated objects. Information from the descriptor component of the `CAP` file can be used to format objects in whatever way an implementation requires.

3.7 Special Initialization Methods

The Java Card virtual machine supports *instance initialization methods* exactly as does the Java virtual machine.

The Java Card virtual machine includes only limited support for *class* or `interface initialization methods`. There is no general mechanism for executing `<clinit>` methods on a Java Card virtual machine. Instead, a `CAP` file includes information for initializing class data as defined in Section 2.2.4.6, "Class Initialization" on page 2-11.

3.8 Exceptions

Exception support in the Java Card virtual machine is identical to support for exceptions in the Java virtual machine.

3.9 Binary File Formats

This specification defines two binary file formats which enable platform-independent development, distribution and execution of Java Card programs.

The CAP file format describes files that contain executable code and can be downloaded and installed onto a Java Card technology-enabled device. A `CAP` file is produced by a Java Card platform Converter tool, and contains a converted form of an entire package of Java classes. This file format's relationship to the Java Card virtual machine is analogous to the relationship of the `class` file format to the Java virtual machine.

The `export` file format describes files that contain the public linking information of Java Card API packages. A package's `export` file is used when converting client packages of that package.

3.10 Instruction Set Summary

The Java Card virtual machine instruction set is quite similar to the Java virtual machine instruction set. Individual instructions consist of a one-byte *opcode* and zero or more *operands*. The pseudo-code for the Java Card virtual machine's instruction fetch-decode-execute loop is the same. Multi-byte operand data is also encoded in *big-endian order*.

There are a number of ways in which the Java Card virtual machine instruction set diverges from that of the Java virtual machine. Most of the differences are due to the Java Card virtual machine's more limited support for data types. Another source of divergence is that the Java Card virtual machine is intended to run on 8-bit and 16-bit architectures, whereas the Java virtual machine was designed for a 32-bit architecture. The rest of the differences are all oriented in one way or another toward optimizing the size or performance of either the Java Card virtual machine or Java Card programs. These changes include inlining constant pool data directly in instruction opcodes or operands, adding multiple versions of a particular instruction to deal with different datatypes, and creating composite instructions for operations on the current object.

3.10.1 Types and the Java Card Virtual Machine

The Java Card virtual machine supports only a subset of the types supported by the Java virtual machine. This subset is described in Chapter 2. Type support is reflected in the instruction set, as instructions encode the data types on which they operate.

Given that the Java Card virtual machine supports fewer types than the Java virtual machine, there is an opportunity for better support for smaller data types. Lack of support for large numeric data types frees up space in the instruction set. This extra instruction space has been used to directly support arithmetic operations on the short data type.

Some of the extra instruction space has also been used to optimize common operations. Type information is directly encoded in field access instructions, rather than being obtained from an entry in the constant pool.

TABLE 3-1 summarizes the type support in the instruction set of the Java Card virtual machine. Only instructions that exist for multiple types are listed. Wide and composite forms of instructions are not listed either. A specific instruction, with type information, is built by replacing the T in the instruction template in the opcode column by the letter representing the type in the type column. If the type column for some instruction is blank, then no instruction exists supporting that operation on that type. For instance, there is a load instruction for type `short`, *sload*, but there is no load instruction for type `byte`.

TABLE 3-1 Type Support in the Java Card Virtual Machine Instruction Set

opcode	byte	short	int	reference
Tspush	*bspush*	*sspush*		
Tipush	*bipush*	*sipush*	*iipush*	
Tconst		*sconst*	*iconst*	*aconst*
Tload		*sload*	*iload*	*aload*

TABLE 3-1 Type Support in the Java Card Virtual Machine Instruction Set

opcode	byte	short	int	reference
Tstore		*sstore*	*istore*	*astore*
Tinc		*sinc*	*iinc*	
Taload	*baload*	*saload*	*iaload*	*aaload*
Tastore	*bastore*	*sastore*	*iastore*	*aastore*
Tadd		*sadd*	*iadd*	
Tsub		*ssub*	*isub*	
Tmul		*smul*	*imul*	
Tdiv		*sdiv*	*idiv*	
Trem		*srem*	*irem*	
Tneg		*sneg*	*ineg*	
Tshl		*sshl*	*ishl*	
Tshr		*sshr*	*ishr*	
Tushr		*sushr*	*iushr*	
Tand		*sand*	*iand*	
Tor		*sor*	*ior*	
Txor		*sxor*	*ixor*	
s2T	*s2b*		*s2i*	
i2T	*i2b*	*i2s*		
Tcmp			*icmp*	
if_TcmpOP		*if_scmpOP*		*if_acmpOP*
Tlookupswitch		*slookupswitch*	*ilookupswitch*	
Ttableswitch		*stableswitch*	*itableswitch*	
Treturn		*sreturn*	*ireturn*	*areturn*
getstatic_T	*getstatic_b*	*getstatic_s*	*getstatic_i*	*getstatic_a*
putstatic_T	*putstatic_b*	*putstatic_s*	*putstatic_i*	*putstatic_a*
getfield_T	*getfield_b*	*getfield_s*	*getfield_i*	*getfield_a*
putfield_T	*putfield_b*	*putfield_s*	*putfield_i*	*putfield_a*

The mapping between Java storage types and Java Card virtual machine computational types is summarized in TABLE 3-2.

TABLE 3-2 Storage Types and Computational Types

Java (Storage) Type	Size in Bits	Computational Type
byte	8	short
short	16	short
int	32	int

Chapter 7 describes the Java Card virtual machine instruction set in detail.

CHAPTER 4

Binary Representation

This chapter presents information about the binary representation of Java Card programs. Java Card technology-based binaries ("Java Card binaries") are usually contained in files, therefore this chapter addresses binary representation in terms of this common case.

Several topics relating to binary representation are covered. The first section describes the basic organization of program representation in export and CAP files, as well as the use of the Java™ Archive (JAR) file containers. The second section covers how Java Card applets and packages are named using unique identifiers. The third section presents the scheme used for naming and linking items within Java Card API packages. The fourth and fifth sections describe the constraints for upward compatibility between different versions of a Java Card technology-based binary ("Java Card binary") program file, and versions assigned based upon that compatibility.

4.1 Java Card Platform File Formats

Java programs are represented in compiled, binary form as class files. Java class files are used not only to execute programs on a Java virtual machine, but also to provide type and name information to a Java compiler. In the latter role, a class file is essentially used to document the API of its class to client code. That client code is compiled into its own class file, including symbolic references used to dynamically link to the API class at runtime.

Java Card technology uses a different strategy for binary representation of programs. Executable binaries and interface binaries are represented in two separate files. These files are respectively called CAP files (for converted applet) and export files.

4.1.1 Export File Format

Export files are not used directly on a device that implements a Java Card virtual machine. However, the information in an export file is critical to the operation of the virtual machine on a device. An export file can be produced by a Java Card converter when a package is converted. This package's export file can be used later to convert another package that imports classes from the first package. Information in the export file is included in the CAP file of the second package, then is used on the device to link the contents of the second package to items imported from the first package.

A Java Card technology-based export file ("Java Card export file") contains the public interface information for an entire package of classes. This means that an export file only contains information about the public API of a package, and does not include information used to link classes within a package.

The name of an export file is the last portion of the package specification followed by the extension '.exp'. For example, the name of the export file of the javacard.framework package must be framework.exp. Operating systems that impose limitations on file name lengths may transform an export file's name according to their own conventions.

For a complete description of the Java Card export file format, see Chapter 5, "The Export File Format".

4.1.2 CAP File Format

A Java Card CAP file contains a binary representation of a package of classes that can be installed on a device and used to execute the package's classes on a Java Card virtual machine.

A CAP file is produced by a Java Card converter when a package of classes is converted. A CAP file consists of a set of components, each of which describes a different aspect of the contents. The set of components in a CAP file can vary, depending on whether the file contains a library or applet definition(s).

For a complete description of the Java Card CAP file format, see Chapter 6, "The CAP File Format".

4.1.3 JAR File Container

The JAR file format is used as the container format for CAP files. What this specification calls a "CAP file" is just a JAR file that contains the required set of CAP components (see Chapter 6, "The CAP File Format").

CAP file components are stored as files in a JAR file. Each CAP file component is located in a subdirectory called javacard that is in a directory representing the package. For example, the CAP file components of the package com.sun.framework are located in the directory com/sun/framework/javacard.

An export file may also be contained in a JAR file, whether that JAR file contains CAP file components or not. If an export file is included, it must be located in the same directory as the components for that package would be.

The name of a JAR file containing CAP file components is not defined as part of this specification. Other files, including CAP file components for another package, may also reside in a JAR file that contains CAP file components.

4.2 AID-based Naming

This section describes the mechanism used for naming applets and packages in Java Card CAP files and export files, and custom components in Java Card CAP files. Java class files use Unicode strings to name Java packages. As the Java Card platform does not include support for strings, an alternative mechanism for naming is provided.

ISO 7816 is a multipart standard that describes a broad range of technology for building smart card systems. ISO 7816-5 defines the AID (application identifier) data format to be used for unique identification of card applications (and certain kinds of files in card file systems). The Java Card platform uses the AID data format to identify applets and packages. AIDs are administered by the International Standards Organization (ISO), so they can be used as unique identifiers.

4.2.1 The AID Format

This section presents a minimal description of the AID data format used in Java Card technology. For complete details, refer to ISO 7816-5, AID Registration Category 'D' format.

The AID format used by the Java Card platform is an array of bytes that can be interpreted as two distinct pieces, as shown in TABLE 4-1. The first piece is a 5-byte value known as a RID (resource identifier). The second piece is a variable length value known as a PIX (proprietary identifier extension). A PIX can be from 0 to 11 bytes in length. Thus an AID can be from 5 to 16 bytes in total length.

TABLE 4-1 AID Format

RID (5 bytes)	PIX (0-11 bytes)

ISO controls the assignment of RIDs to companies, with each company obtaining its own unique RID from the ISO. Companies manage assignment of PIXs for AIDs using their own RIDs.

4.2.2 AID Usage

In the Java platform, packages are uniquely identified using Unicode strings and a naming scheme based on internet domain names. In the Java Card platform, packages and applets are identified using AIDs.

Any package that is represented in an export file must be assigned a unique AID. The AID for a package is constructed from the concatenation of the company's RID and a PIX for that package. This AID corresponds to the string name for the package, as shown in FIGURE 4-1.

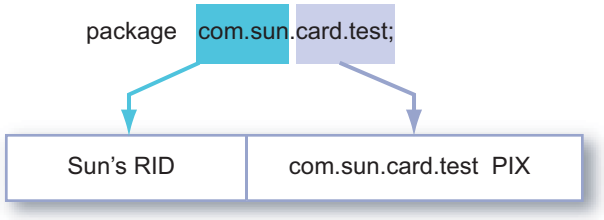


FIGURE 4-1 Mapping Package Identifiers To AIDs

Each applet installed on a Java Card technology enabled device must also have a unique AID. This AID is constructed similarly to a package AID. It is a concatenation of the applet provider's RID and PIX for that applet. An applet AID must not have the same value as the AID of any package or the AID of any other applet. The RID of each applet in a package must be the same as the RID of the package.

Custom components defined in a CAP file are also identified using AIDs. Like AIDs for applets and packages, component AIDs are formed by concatenating a RID and a PIX.

4.3 Token-based Linking

This section describes a scheme that allows downloaded software to be linked against APIs on a Java Card technology enabled device. The scheme represents referenced items as opaque tokens, instead of Unicode strings as are used in Java class files. The two basic requirements of this linking scheme are that it allows linking on the device, and that it does not require internal implementation details of APIs to be revealed to clients of those APIs. Secondary requirements are that the scheme be efficient in terms of resource use on the device, and have acceptable performance for linking. And of course, it must preserve the semantics of the Java language.

4.3.1 Externally Visible Items

Classes (including Interfaces) in Java packages may be declared with public or package visibility. A class's methods and fields may be declared with public, protected, package or private visibility. For purposes of this document, we define public classes, public or protected fields, and public or protected methods to be externally visible from the package.

Each externally visible item must have a token associated with it to enable references from other packages to the item to be resolved on a device. There are six kinds of items in a package that require external identification.

- Classes (including Interfaces)
- Static Fields
- Static Methods
- Instance Fields
- Virtual Methods
- Interface Methods

4.3.2 Private Tokens

Items that are not externally visible are internally visible. Internally visible items are not described in a package's export file, but some such items use private tokens to represent internal references. External references are represented by public tokens. There are three kinds of items that can be assigned private tokens.

- Instance Fields
- Virtual Methods
- Packages

4.3.3 The Export File and Conversion

An export file contains entries for externally visible items in the package. Each entry holds the item's name and its token. Some entries may include additional information as well. For detailed information on the export file format, see Chapter 5, "The Export File Format".

The export file is used to map names for imported items to tokens during package conversion. The Java Card converter uses these tokens to represent references to items in an imported package.

For example, during the conversion of the class files of applet A, the export file of javacard.framework is used to find tokens for items in the API that are used by the applet. Applet A creates a new instance of framework class OwnerPIN. The framework export file contains an entry for javacard.framework.OwnerPIN that holds the token for this class. The converter places this token in the CAP file's constant pool to represent an unresolved reference to the class. The token value is later used to resolve the reference on a device.

4.3.4 References – External and Internal

In the context of a CAP file, references to items are made indirectly through a package's constant pool. References to items in other packages are called external, and are represented in terms of tokens. References to items in the same CAP file are called internal, and are represented either in terms of tokens, or in a different internal format.

An external reference to a class is composed of a package token and a class token. Together those tokens specify a certain class in a certain package. An internal reference to a class is a 15-bit value that is a pointer to the class structure's location within the CAP file.

An external reference to a static class member, either a field or method, consists of a package token, a class token, and a token for the static field or static method. An internal reference to a static class member is a 16-bit value that is a pointer to the item's location in the CAP file.

References to instance fields, virtual methods and interface methods consist of a class reference and a token of the appropriate type. The class reference determines whether the reference is external or internal.

4.3.5 Installation and Linking

External references in a CAP file can be resolved on a device from token form into the internal representation used by the virtual machine.

A token can only be resolved in the context of the package that defines it. Just as the export file maps from a package's externally visible names to tokens, there is a set of link information for each package on the device that maps from tokens to resolved references.

4.3.6 Token Assignment

Tokens for an API are assigned by the API's owner and published in the package export file(s) for that API. Since the name-to-token mappings are published, an API owner may choose any order for tokens (subject to the constraints listed below).

A particular device platform can resolve tokens into whatever internal representation is most useful for that implementation of a Java Card virtual machine. Some tokens may be resolved to indices. For example, an instance field token may be resolved to an index into a class instance's fields. In such cases, the token value is distinct from and unrelated to the value of the resolved index.

4.3.7 Token Details

Each kind of item in a package has its own independent scope for tokens of that kind. The token range and assignment rules for each kind are listed in TABLE 4-2.

TABLE 4-2 Token Range, Type and Scope

Token Type	Range	Type	Scope
Package	0 - 127	Private	Package
Class	0 - 254	Public	Package
Static Field	0 - 255	Public	Class
Static Method	0 - 255	Public	Class
Instance Field	0 - 255	Public or Private	Class
Virtual Method	0 - 127	Public or Private	Class Hierarchy
Interface Method	0 - 127	Public	Class

4.3.7.1 Package

All package references from within a CAP file are assigned private package tokens. Package token values must be in the range from 0 to 127, inclusive. The tokens for all the packages referenced from classes in a CAP file are numbered consecutively starting at zero. The ordering of package tokens is not specified.

4.3.7.2 Classes and Interfaces

All externally visible classes and interfaces in a package are assigned public class tokens. Class token values must be in the range from 0 to 254, inclusive. The tokens for all the public classes and interfaces in a package are numbered consecutively starting at zero. The ordering of class tokens is not specified.

Package-visible classes and interfaces are not assigned tokens.

4.3.7.3 Static Fields

All externally visible static fields in a package are assigned public static field tokens. The tokens for all externally visible static fields in a class are numbered consecutively starting at zero. Static fields token values must be in the range from 0 to 255, inclusive. The ordering of static field tokens is not specified.

Package-visible and private static fields are not assigned tokens. In addition, no tokens are assigned for final static fields that are initialized to primitive, compile-time constants, as these fields are never represented as fields in CAP files.

4.3.7.4 Static Methods and Constructors

All externally visible static methods and constructors in a package are assigned public static method tokens. Constructors are included in this category because they are statically bound. Static method token values must be in the range from 0 to 255, inclusive. The tokens for all the externally visible static methods and constructors in a class are numbered consecutively starting at zero. The ordering of static method tokens is not specified.

Package-visible and private static methods as well as package-visible and private constructors are not assigned tokens.

4.3.7.5 Instance Fields

All instance fields defined in a package are assigned either public or private instance field tokens. The scope of a set of instance field tokens is limited to the class that declares the instance fields, not including the fields declared by superclasses of that class.

Instance field token values must be in the range from 0 to 255, inclusive. Public and private tokens for instance fields are assigned from the same namespace. The tokens for all the instance fields in a class are numbered consecutively starting at zero, except that the token after an int field is skipped and the token for the following field is numbered two greater than the token of the int field.

Within a class, tokens for externally visible fields must be numbered less than the tokens for package and private fields. For public tokens, the tokens for reference type fields must be numbered greater than the tokens for primitive type fields. For private tokens, the tokens for reference type fields must be numbered less than the tokens for primitive type fields. Beyond that, the ordering of instance field tokens in a class is not specified.

TABLE 4-3 Tokens For Instance Fields

Visibility	Category	Type	Token Value
public and protected fields (public tokens)	primitive	`boolean`	0
		`byte`	1
		`short`	2
	reference	`byte[]`	3
		`Applet`	4
package and private fields (private tokens)	reference	`short[]`	5
		`Object`	6
	primitive	`int`	7
		`short`	9

4.3.7.6 Virtual Methods

Virtual methods are instance methods that are resolved dynamically. The set includes all public, protected and package-visible instance methods. Private instance methods and all constructors are not virtual methods, but instead are resolved statically during compilation.

All virtual methods defined in a package are assigned either public or private virtual method tokens. Virtual method token values must be in the range from 0 to 127, inclusive. Public and private tokens for virtual methods are assigned from different namespaces. The high bit of the byte containing a virtual method token is set to one if the token is a private token.

Public tokens for the externally visible (public or protected) introduced virtual methods in a class are numbered consecutively starting at one greater than the highest numbered public virtual method token of the class's superclass. If a method overrides a method implemented in the class's superclass, that method is assigned the same token number as the method in the superclass. The high bit of the byte containing a public virtual method token is always set to zero, to indicate it is a public token. The ordering of public virtual method tokens in a class is not specified.

Private virtual method tokens are assigned to package-visible virtual methods. They are assigned differently from public virtual method tokens. If a class and its superclass are defined in the same package, the tokens for the package-visible introduced virtual methods in that class are numbered consecutively starting at one greater than the highest numbered private virtual method token of the class's superclass. If the class and its superclass are defined in different packages, the tokens for the package-visible introduced virtual methods in that class are numbered consecutively starting at zero. If a method overrides a method implemented in the class's superclass, that method uses the same token number as the method in the superclass. The definition of the Java programming language specifies that overriding a package-visible virtual method is only possible if both the class and its superclass are defined in the same package. The high bit of the byte containing a virtual method token is always set to one, to indicate it is a private token. The ordering of private virtual method tokens in a class is not specified.

4.3.7.7 Interface Methods

All interface methods defined in a package are assigned public interface method tokens, as interface methods are always public. Interface methods tokens values must be in the range from 0 to 127, inclusive. The tokens for all the interface methods defined in or inherited by an interface are numbered consecutively starting at zero. The token value for an interface method in a given interface is unrelated to the token values of that same method in any of the interface's superinterfaces. Each interface includes its own token values for all the methods inherited from super-interfaces as well as its defined methods. The high bit of the byte containing an interface method token is always set to zero, to indicate it is a public token. The ordering of interface method tokens is not specified.

4.4 Binary Compatibility

In the Java programming language the granularity of binary compatibility can be between classes since binaries are stored in individual class files. In Java Card systems Java packages are processed as a single unit, and therefore the granularity of binary compatibility is between packages. In Java Card systems the binary of a package is represented in a CAP file, and the API of a package is represented in an export file.

In a Java Card system, a change to a type in a Java package results in a new CAP file. A new CAP file is binary compatible with (equivalently, does not break compatibility with) a preexisting CAP file if another CAP file converted using the export file of the preexisting CAP file can link with the new CAP file without errors.

shows an example of binary compatible CAP files, p1 and p1′. The preconditions for the example are: the package p1 is converted to create the p1 CAP file and p1 export file, and package p1 is modified and converted to create the p1′ CAP file. Package p2 imports package p1, and therefore when the p2 CAP file is created the export file of p1 is used. In the example, p2 is converted using the original p1 export file. Because p1′ is binary compatible with p1, p2 may be linked with either the p1 CAP file or the p1′ CAP file.

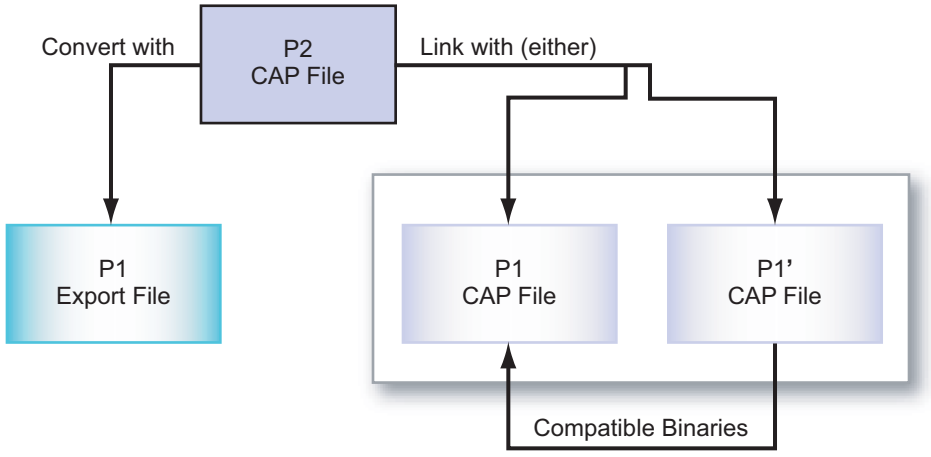


FIGURE 4-2 Binary Compatibility Example

Any modification that causes binary incompatibility in the Java programming language also causes binary incompatibility in Java Card systems. These modifications are described as causing a potential error in The Java Language

Specification. Any modification that does not cause binary incompatibility in the Java programming language does not cause binary incompatibility in a Java Card system, except under the following conditions:

- The value of a token assigned to an element in the API of a package is changed.
- The value of an externally visible final static field (compile-time constant) is changed.
- An externally visible virtual method that does not override a preexisting method is added to a non-final public class.
- An externally visible interface method that does not override a preexisting method is added to a public interface.

Tokens are used to resolve references to imported elements of a package. If a token value is modified, a linker on a device is unable to associate the new token value with the previous token value of the element, and therefore is unable to resolve the reference correctly.

Compile-time constants are not stored as fields in CAP files. Instead their values are recorded in export files and placed inline in the bytecodes in CAP files. These values are said to be pre-linked in a CAP file of a package that imports those constants. During execution, information is not available to determine whether the value of an inlined constant is the same as the value defined by the binary of the imported package.

As described above, tokens assigned to public and protected virtual methods are scoped to the hierarchy of a class. Tokens assigned to public and protected virtual methods introduced in a subclass have values starting at one greater than the maximum token value assigned in a superclass. If a new, non-override, public or protected virtual method is introduced in a superclass it is assigned a token value that would otherwise have been assigned in a subclass. Therefore, two unique virtual methods could be assigned the same token value within the same class hierarchy, making resolution of a reference to one of the methods ambiguous.

The addition of an externally visible, non-override method to a public interface is a binary incompatible change. It allows classes which are not themselves abstract to contain an abstract method. For example, consider the case of an interface I implemented by a class C that is not abstract, where I and C reside in different packages. If a new method is added to I, creating I′, then C cannot link with the new version of I′ because this would result in the class C containing an abstract method without the class C being abstract. The fact that C can not link with I′ means that I and I′ are not binary compatible.

4.5 Package Versions

Each implementation of a package in a Java Card system is assigned a pair of major and minor version numbers. These version numbers are used to indicate binary compatibility or incompatibility between successive implementations of a package.

4.5.1 Assigning

The major and minor versions of a package are assigned by the package provider. It is recommended that the initial implementation of a package be assigned a major version of 1 and a minor version of 0. However, any values may be chosen. It is also recommended that when either a major or a minor version is incremented, it is incremented exactly by 1.

A major version must be changed when a new implementation of a package is not binary compatible with the previous implementation. The value of the new major version must be greater than the major version of the previous implementation. When a major version is changed, the associated minor version must be assigned the value of 0.

When a new implementation of a package is binary compatible with the previous implementation, it must be assigned a major version equal to the major version of the previous implementation. The minor version assigned to the new implementation must be greater than the minor version of the previous implementation.

4.5.2 Linking

Both an export file and a CAP file contain the major and minor version numbers of the package described. When a CAP file is installed on a Java Card technology-enabled device a resident image of the package is created, and the major and minor version numbers are recorded as part of that image. When an export file is used during preparation of a CAP file, the version numbers indicated in the export file are recorded in the CAP file.

During installation, references from the package of the CAP file being installed to an imported package can be resolved only when the version numbers indicated in the export file used during preparation of the CAP file are compatible with the version numbers of the resident image. They are compatible when the major version numbers are equal and the minor version of the export file is less than or equal to the minor version of the resident image.

CHAPTER 5

The Export File Format

This chapter describes the export file format. Compliant Java Card Converters must be capable of producing and consuming all export files that conform to the specification provided in this chapter.

An export file consists of a stream of 8-bit bytes. All 16-bit and 32-bit quantities are constructed by reading in two and four consecutive 8-bit bytes, respectively. Multibyte data items are always stored in big-endian order, where the high-order bytes come first.

This chapter defines its own set of data types representing Java Card export file data: the types `u1`, `u2`, and `u4` represent an unsigned one-, two-, and four-byte quantities, respectively.

The Java Card export file format is presented using pseudo structures written in a C-like structure notation. To avoid confusion with the fields of Java Card virtual machine classes and class instances, the contents of the structures describing the Java Card export file format are referred to as items. Unlike the fields of a C structure, successive items are stored in the Java Card platform file sequentially, without padding or alignment.

Variable-sized *tables*, consisting of variable-sized items, are used in several export file structures. Although we will use C-like array syntax to refer to table items, the fact that tables are streams of varying-sized structures means that it is not possible to directly translate a table index into a byte offset into the table.

In a data structure that is referred to as an *array*, the elements are equal in size.

5.1 Export File Name

As described in Section 4.1.1, "Export File Format" on page 4-2, the name of a export file must be the last portion of the package specification followed by the extension '`.exp`'. For example, the name of the export file of the `javacard.framework` package must be `framework.exp`. Operating systems that impose limitations on file name lengths may transform an export file's name according to its conventions.

5.2 Containment in a JAR File

As described in Section 4.1.3, "JAR File Container" on page 4-2, Java Card CAP files are contained in a JAR file. If an export file is also stored in a JAR file, it must also be located in a directory called `javacard` that is a subdirectory of the package's directory. For example, the `framework.exp` file would be located in the subdirectory `javacard/framework/javacard`.

5.3 Ownership

An `export` file is owned by the entity that owns the package it represents. The owner of a package defines the API of that package, and may or may not provide all implementations of that package. All implementations, however, must conform to the definition provided in the `export` file provided by the owner.

A particular example of export file ownership is the Java Card API packages. Sun Microsystems, Inc. defines these packages. Sun Microsystems, Inc. also provides the `export` files for these packages. All implementations of the Java Card API packages must conform to the definitions provided by Sun Microsystems, Inc., and comply with the token assignments provided in these `export` files.

5.4 Hierarchies Represented

Classes and interfaces represented in an export file include public elements defined within their respective hierarchies. For example, instead of indicating the immediate superclass or superinterface, all public superclasses or superinterfaces are listed. This design concept is applied not only to superclasses or superinterfaces, but also to virtual methods and implemented interfaces.

5.5 Export File

An `export` file is defined by the following structure:

```
ExportFile {
     u4 magic
     u1 minor_version
     u1 major_version
     u2 constant_pool_count
     cp_info constant_pool[constant_pool_count]
     u2 this_package
     u1 export_class_count
     class_info classes[export_class_count]
}
```

The items in the `ExportFile` structure are as follows:

`magic`

The `magic` item contains the magic number identifying the `ExportFile` format; it has the value `0x00FACADE`.

`minor_version, major_version`

The `minor_version` and `major_version` items are the minor and major version numbers of this export file. Together, a major and a minor version number determine the version of the export file format. If an export file has the major version number of *M* and minor version number of *m*, the version of the export file's format is *M.m*.

A change in the major version number indicates a major incompatibility change, one that requires a fundamentally different Java Card virtual machine. A Java Card virtual machine is not required to support `export` files with different major version numbers. A Java Card virtual machine is required to support `export` files having a given major version number and all valid minor version numbers in the range 0 through some particular `minor_version` where a valid minor version number is a minor version number that has been defined in a version of the Java Card virtual machine specification.

In this specification, the major version of the export file format has the value 2 and the minor version has the value 2. Only Sun Microsystems, Inc. may define the meaning and values of new export file format versions.

`constant_pool_count`

The `constant_pool_count` item is a non-zero, positive value that indicates the number of constants in the constant pool.

`constant_pool[]`

The `constant_pool` is a table of variable-length structures representing various string constants, class names, field names and other constants referred to within the `ExportFile` structure.

Each of the `constant_pool` table entries, including entry zero, is a variable-length structure whose format is indicated by its first "tag" byte.

There are no ordering constrains on entries in the `constant_pool` table.

`this_package`

The value of `this_package` must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Package_info` (Section 5.6.1, "CONSTANT_Package" on page 5-5) structure representing the package defined by this `ExportFile`.

`export_class_count`

The value of the `export_class_count` item gives the number of elements in the classes table.

`classes[]`

Each value of the classes table is a variable-length `class_info` structure (Section 5.7, "Classes and Interfaces" on page 5-8) giving the description of a publicly accessible class or interface declared in this package. If the `ACC_LIBRARY` flag item in the `CONSTANT_Package_info` (Section 5.6.1, "CONSTANT_Package" on page 5-5) structure indicated by the this_package item is set, the classes table has an entry for each public class and interface declared in this package. If the `ACC_LIBRARY` flag item is not set, the classes table has an entry for each public shareable interface declared in this package.[1]

5.6 Constant Pool

All `constant_pool` table entries have the following general format:

1. This restriction of exporting only shareable interfaces in non-library packages is imposed by the firewall defined in the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

```
cp_info {
     u1 tag
     u1 info[]
}
```

Each item in the `constant_pool` must begin with a 1-byte tag indicating the kind of `cp_info` entry. The content of the info array varies with the value of tag. The valid tags and their values are listed in TABLE 5-1. Each tag byte must be followed by two or more bytes giving information about the specific constant. The format of the additional information varies with the tag value.

TABLE 5-1 Export File Constant Pool Tags

Constant Type	Value
CONSTANT_Package	13
CONSTANT_Classref	7
CONSTANT_Integer	3
CONSTANT_Utf8	1

5.6.1 CONSTANT_Package

The `CONSTANT_Package_info` structure is used to represent a package:

```
CONSTANT_Package_info {
     u1 tag
     u1 flags
     u2 name_index
     u1 minor_version
     u1 major_version
     u1 aid_length
     u1 aid[aid_length]
}
```

The items of the `CONSTANT_Package_info` structure are the following:

`tag`

The tag item has the value of `CONSTANT_Package` (13).

`flags`

The `flags` item is a mask of modifiers that apply to this package. The `flags` modifiers are shown in the following table.

TABLE 5-2 Export File Package Flags

Flags	Value
ACC_LIBRARY	0x01

The ACC_LIBRARY flag has the value of one if this package does not define and declare any applets. In this case it is called a library package. Otherwise ACC_LIBRARY has the value of zero.

If the package is not a library package this export file can only contain shareable interfaces.[1] A shareable interface is either the javacard.framework.Shareable interface or an interface that extends the javacard.framework.Shareable interface.

All other flag values are reserved. Their values must be zero.

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info (Section 5.6.4, "CONSTANT_Utf8" on page 5-7) structure representing a valid Java package name.

As in Java class files, ASCII periods ('.') that normally separate the identifiers in a package name are replaced by ASCII forward slashes ('/'). For example, the package name javacard.framework is represented in a CONSTANT_Utf8_info structure as javacard/framework.

minor_version, major_version

The minor_version and major_version items are the minor and major version numbers of this package. These values uniquely identify the particular implementation of this package and indicate the binary compatibility between packages. See Section 4.5, "Package Versions" on page 4-13 for a description of assigning and using package version numbers.

aid_length

The value of the aid_length item gives the number of bytes in the aid array. Valid values are between 5 and 16, inclusive.

aid[]

The aid array contains the ISO AID of this package (Section 4.2, "AID-based Naming" on page 4-3).

5.6.2 CONSTANT_Classref

The CONSTANT_Classref_info structure is used to represent a class or interface:

```
CONSTANT_Classref_info {
     u1 tag
     u2 name_index
}
```

1. This restriction is imposed by the firewall defined in the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

The items of the CONSTANT_Classref_info structure are the following:

tag

The tag item has the value of CONSTANT_Classref (7).

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info (Section 5.6.4, "CONSTANT_Utf8" on page 5-7) structure representing a valid fully qualified Java class or interface name. This name is fully qualified since it may represent a class or interface defined in a package other than the one described in the export file.

As in Java class files, ASCII periods ('.') that normally separate the identifiers in a class or interface name are replaced by ASCII forward slashes ('/'). For example, the interface name javacard.framework.Shareable is represented in a CONSTANT_Utf8_info structure as javacard/framework/Shareable.

5.6.3 CONSTANT_Integer

The CONSTANT_Integer_info structure is used to represent four-byte numeric (int) constants:

```
CONSTANT_Integer_info {
     u1 tag
     u4 bytes
}
```

The items of the CONSTANT_Integer_info structure are the following:

tag

The tag item has the value of CONSTANT_Integer (3).

bytes

The bytes item of the CONSTANT_Integer_info structure contains the value of the int constant. The bytes of the value are stored in big-endian (high byte first) order.

The value of a boolean type is 1 to represent true and 0 to represent false.

5.6.4 CONSTANT_Utf8

The CONSTANT_Utf8_info structure is used to represent constant string values. UTF-8 strings are encoded in the same way as described in *The Java Virtual Machine Specification* (§ 4.4.7).

The CONSTANT_Utf8_info structure is:

```
CONSTANT_Utf8_info {
     u1 tag
     u2 length
     u1 bytes[length]
}
```

The items of the CONSTANT_Utf8_info structure are the following:

tag

The tag item has the value of CONSTANT_Utf8 (1).

length

The value of the length item gives the number of bytes in the bytes array (not the length of the resulting string). The strings in the CONSTANT_Utf8_info structure are not null-terminated.

bytes[]

The bytes array contains the bytes of the string. No byte may have the value (byte)0 or (byte)0xF0-(byte)0xFF.

5.7 Classes and Interfaces

Each class and interface is described by a variable-length class_info structure. The format of this structure is:

```
class_info {
     u1 token
     u2 access_flags
     u2 name_index
     u2 export_supers_count
     u2 supers[export_supers_count]
     u1 export_interfaces_count
     u2 interfaces[export_interfaces_count]
     u2 export_fields_count
     field_info fields[export_fields_count]
     u2 export_methods_count
     method_info methods[export_methods_count]
}
```

The items of the class_info structure are as follows:

token

The value of the token item is the class token (Section 4.3.7.2, “Classes and Interfaces” on page 4-8) assigned to this class or interface.

access_flags

The value of the `access_flags` item is a mask of modifiers used with class and interface declarations. The `access_flags` modifiers are shown in the following table.

TABLE 5-3 Export File Class Access and Modifier Flags

Name	Value	Meaning	Used By
`ACC_PUBLIC`	`0x0001`	Is public; may be accessed from outside its package	Class, interface
`ACC_FINAL`	`0x0010`	Is final; no subclasses allowed.	Class
`ACC_INTERFACE`	`0x0200`	Is an interface	Interface
`ACC_ABSTRACT`	`0x0400`	Is abstract; may not be instantiated	Class, interface
`ACC_SHAREABLE`	`0x0800`	Is shareable; may be shared between Java Card applets.	Class, interface
`ACC_REMOTE`	`0x1000`	Is remote; may be accessed by Java Card RMI	Class, interface

The `ACC_SHAREABLE` flag indicates whether this class or interface is shareable.[1] A class is shareable if it implements (directly or indirectly) the `javacard.framework.Shareable` interface. An interface is shareable if it is or extends (directly or indirectly) the `javacard.framework.Shareable` interface.

The `ACC_REMOTE` flag indicates whether this class or interface is remote. The value of this flag must be one if and only if the class or interface satisfies the requirements defined in Section 2.2.6.1, "Remote Classes and Remote Interfaces" on page 2-12.

All other class access and modifier flags are defined in the same way and with the same restrictions as described in *The Java Virtual Machine Specification*.

Since all classes and interfaces represented in an export file are public, the `ACC_PUBLIC` flag must always be set.

All other flag values are reserved. Their values must be zero.

`name_index`

1. The `ACC_SHAREABLE` flag is defined to enable Java Card virtual machines to implement the firewall restrictions defined by the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Classref_info (Section 5.6.2, "CONSTANT_Classref" on page 5-6) structure representing a valid, fully qualified Java class or interface name.

export_supers_count

The value of the export_supers_count item indicates the number of entries in the *supers* array.

supers[]

The supers array contains an entry for each public superclass of this class or interface. It does not include package visible superclasses.

For a class, each value in the supers array must be a valid index into the constant_pool table. The constant_pool entry at each value must be a CONSTANT_Classref_info structure (Section 5.6.2, "CONSTANT_Classref" on page 5-6) representing a valid, fully-qualified Java class name. Entries in the supers array can occur in any order.

For an interface, the supers array contains a single value representing a valid index into the constant_pool table. The *constant_pool* entry must be a CONSTANT_Classref_info structure (Section 5.6.2, "CONSTANT_Classref" on page 5-6) representing the fully-qualified name of the java.lang.Object class.

export_interfaces_count

The value of the export_interfaces_count item indicates the number of entries in the interfaces array.

interfaces[]

If this class_info structure describes a class, the interfaces array contains an entry for each public interface implemented by this class. It does not include package-visible interfaces. It does include all public superinterfaces in the hierarchies of public interfaces implemented by this class.

If this class_info structure describes an interface, the interfaces array contains an entry for each public interface extended by this interface. It does not include package visible interfaces. It does include all public superinterfaces in the hierarchies of public interfaces extended by this interface.

Each value in the interfaces array must be a valid index into the constant_pool table. The constant_pool entry at each value must be a CONSTANT_Classref_info structure (Section 5.6.2, "CONSTANT_Classref" on page 5-6) representing a valid, fully-qualified Java interface name. Entries in the interfaces array can occur in any order.

export_fields_count

The value of the export_fields_count item gives the number of entries in the fields table.

`fields[]`

Each value in the `fields` table is a variable-length `field_info` (Section 5.8, "Fields" on page 5-11) structure. The `field_info` contains an entry for each publicly accessible field, both class variables and instance variables, declared by this class or interface. It does not include items representing fields that are inherited from superclasses or superinterfaces.

`export_methods_count`

The value of the `export_methods_count` item gives the number of entries in the `methods` table.

`methods[]`

Each value in the `methods` table is a method_info (Section 5.9, "Methods" on page 5-13) structure. The `method_info` structure contains an entry for each publicly accessible class (static or constructor) method defined by this class, and each publicly accessible instance method defined by this class or its superclasses, or defined by this interface or its super-interfaces.

5.8 Fields

Each field is described by a variable-length `field_info` structure. The format of this structure is:

```
field_info {
     u1 token
     u2 access_flags
     u2 name_index
     u2 descriptor_index
     u2 attributes_count
     attribute_info attributes[attributes_count]
}
```

The items of the `field_info structure` are as follows:

`token`

The token item is the token assigned to this field. There are three scopes for field tokens: `final static` fields of primitive types (compile-time constants), all other `static` fields, and `instance` fields.

If this field is a compile-time constant, the value of the token item is 0xFF. Compile-time constants are represented in export files, but are not assigned token values suitable for late binding. Instead Java Card Converters must replace bytecodes that reference `final static` fields with bytecodes that load the constant value of the field.[1]

If this field is `static`, but is not a compile-time constant, the token item represents a static field token (Section 4.3.7.3, “Static Fields” on page 4-8).

If this field is an instance field, the token item represents an instance field token (Section 4.3.7.5, “Instance Fields” on page 4-9).

`access_flags`

The value of the `access_flags` item is a mask of modifiers used with fields. The *access_flags* modifiers are shown in the following table.

TABLE 5-4 Export File Field Access and Modifier Flags

Name	Value	Meaning	Used By
`ACC_PUBLIC`	`0x0001`	Is public; may be accessed from outside its package.	Any field
`ACC_PROTECTED`	`0x0004`	Is protected; may be accessed within subclasses.	Class field Instance field
`ACC_STATIC`	`0x0008`	Is static.	Class field Interface field
`ACC_FINAL`	`0x0010`	Is final; no further overriding or assignment after initialization.	Any field

Field access and modifier flags are defined in the same way and with the same restrictions as described in *The Java Virtual Machine Specification*.

Since all fields represented in an export file are either public or protected, exactly one of the `ACC_PUBLIC` or `ACC_PROTECTED` flag must be set.

The Java Card virtual machine reserves all other flag values. Their values must be zero.

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` (Section 5.6.4, “CONSTANT_Utf8” on page 5-7) structure representing a valid Java field name stored as a simple (not fully qualified) name, that is, as a Java identifier.

`descriptor_index`

1. Although Java compilers ordinarily replace references to final static fields of primitive types with primitive constants, this functionality is not required.

The value of the `descriptor_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` (Section 5.6.4, "CONSTANT_Utf8" on page 5-7) structure representing a valid Java field descriptor.

Representation of a field descriptor in an `export` file is the same as in a Java class file. See the specification described in *The Java Virtual Machine Specification* (§4.3.2).

If this field is a reference-type, the class referenced must be a public class.

`attributes_count`

The value of the `attributes_count` item indicates the number of additional attributes of this field. The only `field_info` attribute currently defined is the `ConstantValue` attribute (Section 5.10.1, "ConstantValue Attribute" on page 5-15). For static final fields of primitive types, the value must be `1`; that is, when both the `ACC_STATIC` and `ACC_FINAL` bits in the `flags` item are set an attribute must be present. For all other fields the value of the `attributes_count` item must be `0`.

`attributes[]`

The only attribute defined for the attributes table of a `field_info` structure by this specification is the ConstantValue attribute (Section 5.10.1, "ConstantValue Attribute" on page 5-15). This must be defined for static final fields of primitive types (`boolean`, `byte`, `short`, and `int`).

5.9 Methods

Each method is described by a variable-length `method_info` structure. The format of this structure is:

```
method_info {
     u1 token
     u2 access_flags
     u2 name_index
     u2 descriptor_index
}
```

The items of the `method_info` structure are as follows:

`token`

The `token` item is the token assigned to this method. If this method is a `static` method or constructor, the `token` item represents a `static` method token (Section 4.3.7.4, "Static Methods and Constructors" on page 4-8). If this method is a virtual method, the token item represents a virtual method token

(Section 4.3.7.6, "Virtual Methods" on page 4-9). If this method is an `interface` method, the `token` item represents an `interface` method token (Section 4.3.7.7, "Interface Methods" on page 4-10).

`access_flags`

The value of the `access_flags` item is a mask of modifiers used with methods. The `access_flags` modifiers are shown in the following table.

TABLE 5-5 Export File Method Access and Modifier Flags

Name	Value	Meaning	Used By
`ACC_PUBLIC`	`0x0001`	Is public; may be accessed from outside its package.	Any method
`ACC_PROTECTED`	`0x0004`	Is protected; may be accessed within subclasses.	Class/instance method
`ACC_STATIC`	`0x0008`	Is static.	Class/instance method
`ACC_FINAL`	`0x0010`	Is final; no further overriding or assignment after initialization.	Class/instance method
`ACC_ABSTRACT`	`0x0400`	Is abstract; no implementation is provided	Any method

Method access and modifier flags are defined in the same way and with the same restrictions as described in *The Java Virtual Machine Specification*.

Since all methods represented in an export file are either public or protected, exactly one of the `ACC_PUBLIC` or `ACC_PROTECTED` flag must be set.

Unlike in Java class files, the `ACC_NATIVE` flag is not supported in `export` files. Whether a method is native is an implementation detail that is not relevant to importing packages. The Java Card virtual machine reserves all other flag values. Their values must be zero.

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` (Section 5.6.4, "CONSTANT_Utf8" on page 5-7) structure representing either the special internal method name for constructors, `<init>`, or a valid Java method name stored as a simple (not fully qualified) name.

`descriptor_index`

The value of the descriptor_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info (Section 5.6.4, "CONSTANT_Utf8" on page 5-7) structure representing a valid Java method descriptor.

Representation of a method descriptor in an export file is the same as in a Java class file. See the specification described in *The Java Virtual Machine Specification* (Section 4.3.3, "The Export File and Conversion" on page 4-6).

All classes referenced in a descriptor must be public classes.

5.10 Attributes

Attributes are used in the field_info (Section 5.8, "Fields" on page 5-11) structure of the export file format. All attributes have the following general format:

```
attribute_info {
     u2 attribute_name_index
     u4 attribute_length
     u1 info[attribute_length]
}
```

5.10.1 ConstantValue Attribute

The ConstantValue attribute is a fixed-length attribute used in the attributes table of the field_info structures. A ConstantValue attribute represents the value of a final static field (compile-time constant); that is, both the ACC_STATIC and ACC_FINAL bits in the flags item of the field_info structure must be set. There can be no more than one ConstantValue attribute in the attributes table of a given field_info structure.

The ConstantValue attribute has the format:

```
ConstantValue_attribute {
     u2 attribute_name_index
     u4 attribute_length
     u2 constantvalue_index
}
```

The items of the ConstantValue_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info (Section 5.6.4, "CONSTANT_Utf8" on page 5-7) structure representing the string "ConstantValue."

`attribute_length`

The value of the `attribute_length` item of a `ConstantValue_attribute` structure must be `2`.

`constantvalue_index`

The value of the `constantvalue_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must give the constant value represented by this attribute.

The `constant_pool` entry must be of a type `CONSTANT_Integer` (Section 5.6.3, "CONSTANT_Integer" on page 5-7).

CHAPTER 6

The CAP File Format

This chapter describes the Java Card converted applet (CAP) file format. Each CAP file contains all of the classes and interfaces defined in one Java package. Java Card Converters must be capable of producing CAP files that conform to the specification provided in this chapter.

A CAP file consists of a stream of 8-bit bytes. All 16-bit and 32-bit quantities are constructed by reading in two and four consecutive 8-bit bytes, respectively. Multibyte data items are always stored in big-endian order, where the high-order bytes come first. The first bit read of an 8-bit quantity is considered the *high bit*.

This chapter defines its own set of data types representing Java Card CAP file data: the types `u1`, and `u2` represent an unsigned one-, and two-byte quantities, respectively. Some `u1` types are represented as *bitfield* structures, consisting of arrays of bits. The zeroeth bit in each bit array represents the most significant bit, or high bit.

The Java Card CAP file format is presented using pseudo structures written in a C-like structure notation. To avoid confusion with the fields of Java Card virtual machine classes and class instances, the contents of the structures describing the Java Card CAP file format are referred to as *items*. Unlike the fields of a C structure, successive items are stored in the Java Card platform file sequentially, without padding or alignment.

Variable-sized tables, consisting of variable-sized items, are used in several CAP file data structures. Although we will use C-like array syntax to refer to table items, the fact that tables are streams of variable-sized structures means that it is not possible to directly translate a table index into a byte offset into the table.

A data structure referred to as an array consists of items equal in size.

Some items in the structures of the CAP file format are describe using a C-like union notation. The bytes contained in a union structure have one of the two formats. Selection of the two formats is based on the value of the high bit of the structure.

6.1 Component Model

A Java Card CAP file consists of a set of components. Each component describes a set of elements in the Java package defined, or an aspect of the CAP file. A complete CAP file must contain all of the required components specified in this chapter. Three `components` are optional: the Applet Component (Section 6.5, "Applet Component" on page 6-12), Export Component (Section 6.12, "Export Component" on page 6-47), and Debug Component (Section 6.14, "Debug Component" on page 6-57). The Applet Component is included only if one or more Applets are defined in the package. The Export Component is included only if classes in other packages may import elements in the package defined. The Debug Component contains all of the data necessary for debugging a package.

The content of each component defined in a CAP file must conform to the corresponding format specified in this chapter. All components have the following general format:

```
component {
     u1 tag
     u2 size
     u1 info[]
}
```

Each component begins with a 1-byte tag indicating the kind of component. Valid tags and their values are listed in TABLE 6-1. The size item indicates the number of bytes in the info array of the component, not including the tag and size items.

The content and format of the info array varies with the type of component.

TABLE 6-1 CAP File Component Tags

Component Type	Value
COMPONENT_Header	1
COMPONENT_Directory	2
COMPONENT_Applet	3
COMPONENT_Import	4
COMPONENT_ConstantPool	5
COMPONENT_Class	6
COMPONENT_Method	7
COMPONENT_StaticField	8
COMPONENT_ReferenceLocation	9

TABLE 6-1 CAP File Component Tags

Component Type	Value
COMPONENT_Export	10
COMPONENT_Descriptor	11
COMPONENT_Debug	12

Sun Microsystems, Inc. may define additional components in future versions of this Java Card virtual machine specification. It is guaranteed that additional components will have `tag` values between 13 and 127, inclusive.

6.1.1 Containment in a JAR File

Each CAP file component is represented as a single file. The component file names are enumerated in TABLE 6-2. These names are not case sensitive.

TABLE 6-2 CAP File Component File Names

Component Type	File Name
COMPONENT_Header	Header.cap
COMPONENT_Directory	Directory.cap
COMPONENT_Applet	Applet.cap
COMPONENT_Import	Import.cap
COMPONENT_ConstantPool	ConstantPool.cap
COMPONENT_Class	Class.cap
COMPONENT_Method	Method.cap
COMPONENT_StaticField	StaticField.cap
COMPONENT_ReferenceLocation	RefLocation.cap
COMPONENT_Export	Export.cap
COMPONENT_Descriptor	Descriptor.cap
COMPONENT_Debug	Debug.cap

All CAP file components are stored in a JAR file. As described in Section 4.1.3, "JAR File Container" on page 4-2, the path to the CAP file component files in a JAR file consists of a directory called `javacard` that is in a subdirectory representing the package's directory. For example, the CAP file component files of the package

javacard.framework are located in the subdirectory javacard/framework/javacard. Other files, including other CAP files, may also reside in a JAR file that contains CAP file component files.

The JAR file format provides a vehicle suitable for the distribution of CAP file components. It is not intended or required that the JAR file format be used as the load file format for loading CAP file components onto a Java Card technology-enabled device. See Section 6.2, "Installation" on page 6-5 for more information.

The name of a JAR file containing CAP file components is not defined as part of this specification. The naming convention used by the Sun Microsystems, Inc. Java Card Converter Tool is to append .cap to the simple (meaning not fully qualified) package name. For example, the CAP file produced for the package com.sun.javacard.JavaLoyalty would be named JavaLoyalty.cap.

6.1.2 Defining New Components

Java Card CAP files are permitted to contain new, or custom, components. All new components not defined as part of this specification must not affect the semantics of the specified components, and Java Card virtual machines must be able to accept CAP files that do not contain new components. Java Card virtual machine implementations are required to silently ignore components they do not recognize.

New components are identified in two ways: they are assigned both an ISO 7816-5 AID (Section 4.2, "AID-based Naming" on page 4-3) and a tag value. Valid tag values are between 128 and 255, inclusive. Both of these identifiers are recorded in the custom_component item of the Directory Component (Section 6.4, "Directory Component" on page 6-9).

The new component must conform to the general component format defined in this chapter, with a tag value, a size value indicating the number of bytes in the component (excluding the tag and size items), and an info item containing the content of the new component.

A new component file is stored in a JAR file, following the same restrictions as those specified in Section 4.1.3, "JAR File Container" on page 4-2. That is, the file containing the new component must be located in the <package_directory>/javacard subdirectory of the JAR file and must have the extension .cap.

6.2 Installation

Installing a CAP file components onto a Java Card technology-enabled device entails communication between a Java Card technology-enabled terminal and that device. While it is beyond the scope of this specification to define a load file format or installation protocol between a terminal and a device, the CAP file component order shown in TABLE 6-3 is a reference load order suitable for an implementation with a simple memory management model on a limited memory device.[1]

TABLE 6-3 Reference Component Install Order

Component Type
COMPONENT_Header
COMPONENT_Directory
COMPONENT_Import
COMPONENT_Applet
COMPONENT_Class
COMPONENT_Method
COMPONENT_StaticField
COMPONENT_Export
COMPONENT_ConstantPool
COMPONENT_ReferenceLocation
COMPONENT_Descriptor (optional)

The component type COMPONENT_Debug is not intended for download to the device. It is intended to be used off-card in conjunction with a suitably instrumented Java Card virtual machine.

1. Both the Java Card Forum and Global Platform specification have adopted this component load order as a standard to enhance interoperability. In both cases, loading the Descriptor Component is optional. Furthermore, the Global Platform specification defines the format of packets (APDUs) used during installation.

6.3 Header Component

The Header Component contains general information about this CAP file and the package it defines. It is described by the following variable-length structure:

```
header_component {
     u1 tag
     u2 size
     u4 magic
     u1 minor_version
     u1 major_version
     u1 flags
     package_info package
     package_name_info package_name
}
```

The items in the `header_component` structure are as follows:

`tag`

The `tag` item has the value `COMPONENT_Header` (1).

`size`

The `size` item indicates the number of bytes in the `header_component` structure, excluding the `tag` and `size` items. The value of the `size` item must be greater than zero.

`magic`

The `magic` item supplies the `magic` number identifying the Java Card CAP file format; it has the value `0xDECAFFED`.

`minor_version, major_version`

The `minor_version` and `major_version` items are the minor and major version numbers of this CAP file. Together, a major and a minor version number determine the version of the CAP file format. If a CAP file has the major version number of *M* and minor version number of *m*, the version of the CAP file's format is *M.m*.

A change in the major version number indicates a major incompatibility change, one that requires a fundamentally different Java Card virtual machine. A Java Card virtual machine is not required to support CAP files with different major version numbers. A Java Card virtual machine is required to support CAP files having a given major version number and all valid minor version numbers in the range 0 through some particular `minor_version` where a valid minor version number is a minor version number that has been defined in a version of the Java Card virtual machine specification.

In this specification, the major version of the CAP file format has the value 2 and the minor version has the value 2. Only Sun Microsystems, Inc. may define the meaning and values of new CAP file format versions.

`flags`

The `flags` item is a mask of modifiers that apply to this package. The `flags` modifiers are shown in the following table.

TABLE 6-4 CAP File Package Flags

Flags	Value
ACC_INT	0x01
ACC_EXPORT	0x02
ACC_APPLET	0x04

The ACC_INT flag has the value of one if the Java int type is used in this package. The int type is used if one or more of the following is present:

- a parameter to a method of type int,
- a parameter to a method of type int array,
- a local variable of type int,
- a local variable of type int array,
- a field of type int,
- a field of type int array,
- an instruction of type int, or
- an instruction of type int array.

Otherwise the `ACC_INT` flag has the value of `0`.

The `ACC_EXPORT` flag has the value of `one` if an Export Component (Section 6.12, "Export Component" on page 6-47) is included in this CAP file. Otherwise it has the value of 0.

The `ACC_APPLET` flag has the value of `one` if an Applet Component (Section 6.5, "Applet Component" on page 6-12) is included in this CAP file. Otherwise it has the value of `0`.

All other bits in the flags item not defined in TABLE 6-4 are reserved for future use. Their values must be `zero`.

`package`

The `package` item describes the package defined in this CAP file. It is represented as a `package_info` structure:

```
package_info {
      u1 minor_version
      u1 major_version
```

```
        u1 AID_length
        u1 AID[AID_length]
    }
```

The items in the `package_info` structure are as follows:

`minor_version, major_version`

The `minor_version` and major_version items are the minor and major version numbers of this package. These values uniquely identify the particular implementation of this package and indicate the binary compatibility between packages. See Section 4.5, "Package Versions" on page 4-13 for a description of assigning and using package version numbers.

`AID_length`

The `AID_length` item represents the number of bytes in the AID item. Valid values are between 5 and 16, inclusive.

AID[]

The `AID` item represents the Java Card platform name of the package. See ISO 7816-5 for the definition of an AID (Section 4.2, "AID-based Naming" on page 4-3).

`package_name`

The `package_name` item describes the name of the package defined in this CAP file. It is represented as a `package_name_info[]` structure:

```
package_name_info {
        u1 name_length
        u1 name[name_length]
}
```

The items in the `package_name_info[]` structure are as follows:

`name_length`

The `name_length` item is the number of bytes used in the name item to represent the name of this package in UTF-8 format. The value of this item may be zero if and only if the package does not define any remote interfaces or remote classes.

`name[]`

The `name[]` item is a variable length representation of the fully qualified name of this package in UTF-8 format. The fully qualified name is represented in internal form as described in *Java Virtual Machine Specification* (Section 4.2).

6.4 Directory Component

The Directory Component lists the size of each of the components defined in this CAP file. When an optional component is not included, such as the Applet Component (Section 6.5, "Applet Component" on page 6-12), Export Component (Section 6.12, "Export Component" on page 6-47), or Debug Component (Section 6.14, "Debug Component" on page 6-57), it is represented in the Directory Component with size equal to zero. The Directory Component also includes entries for new (or custom) components.

The Directory Component is described by the following variable-length structure:

```
directory_component {
       u1 tag
       u2 size
       u2 component_sizes[12]
       static_field_size_info static_field_size
       u1 import_count
       u1 applet_count
       u1 custom_count
       custom_component_info custom_components[custom_count]
 }
```

The items in the `directory_component` structure are as follows:

`tag`

The tag item has the value `COMPONENT_Directory` (2).

`size`

The `size` item indicates the number of bytes in the `directory_component` structure, excluding the `tag` and `size` items. The value of the `size` item must be greater than zero.

`component_sizes[]`

The `component_sizes` item is an array representing the number of bytes in each of the components in this CAP file. All of the 12 components defined in this chapter are represented in the `component_sizes` array. The value of an index into the array is equal to the value of the tag of the component represented at that entry, minus 1.

The value in each entry in the `component_sizes` array is the same as the size item in the corresponding component. It represents the number of bytes in the component, excluding the `tag` and `size` items.

The value of an entry in the `component_sizes` array is zero for components not included in this CAP file. Components that may not be included are the Applet Component (Section 6.5, "Applet Component" on page 6-12), the Export

Component (Section 6.12, “Export Component” on page 6-47), and the Debug Component (Section 6.14, “Debug Component” on page 6-57). For all other components the value is greater than zero.

`static_field_size`

The `static_field_size` item is a `static_field_size_info` structure. The structure is defined as:

```
static_field_size_info {
   u2 image_size
   u2 array_init_count
   u2 array_init_size
}
```

The items in the `static_field_size_info` structure are the following:

`image_size`

The `image_size` item has the same value as the `image_size` item in the Static Field Component (Section 6.10, “Static Field Component” on page 6-41). It represents the total number of bytes in the `static` fields defined in this package, excluding final `static` fields of primitive types.

`array_init_count`

The `array_init_count` item has the same value as the `array_init_count` item in the Static Field Component (Section 6.10, “Static Field Component” on page 6-41). It represents the number of arrays initialized in all of the `<clinit>` methods in this package.

`array_init_size`

The `array_init_size` item represents the sum of the count items in the `array_init` table item of the Static Field Component (Section 6.10, “Static Field Component” on page 6-41). It is the total number of bytes in all of the arrays initialized in all of the `<clinit>` methods in this package.

`import_count`

The `import_count` item indicates the number of packages imported by classes and interfaces in this package. This item has the same value as the count item in the Import Component (Section 6.6, “Import Component” on page 6-13).

`applet_count`

The `applet_count` item indicates the number of applets defined in this package. If an Applet Component Section 6.5, “Applet Component” on page 6-12) is not included in this CAP file, the value of the `applet_count` item is zero. Otherwise the value of the `applet_count` item is the same as the value of the count item in the Applet Component (Section 6.5, “Applet Component” on page 6-12).

`custom_count`

The custom_count item indicates the number of entries in the custom_components table. Valid values are between 0 and 127, inclusive.

custom_components[]

The custom_components item is a table of variable-length custom_component_info structures. Each new component defined in this CAP file must be represented in the table. These components are not defined in this standard.

The custom_component_info structure is defined as:

```
custom_component_info {
    u1 component_tag
    u2 size
    u1 AID_length
    u1 AID[AID_length]
    }
```

The items in entries of the custom_component_info structure are:

component_tag

The component_tag item represents the tag of the component. Valid values are between 128 and 255, inclusive.

size

The size item represents the number of bytes in the component, excluding the tag and size items.

AID_length

The AID_length item represents the number of bytes in the AID item. Valid values are between 5 and 16, inclusive.

AID[]

The AID item represents the Java Card platform name of the component. See ISO 7816-5 for the definition of an AID (Section 4.2, "AID-based Naming" on page 4-3).

Each component is assigned an AID conforming to the ISO 7816-5 standard. Beyond that, there are no constraints on the value of an AID of a custom component.

6.5 Applet Component

The Applet Component contains an entry for each of the applets defined in this package. Applets are defined by implementing a non-abstract subclass, direct or indirect, of the `javacard.framework.Applet` class.[1] If no applets are defined, this component must not be present in this CAP file.

The Applet Component is described by the following variable-length structure:

```
applet_component {
     u1 tag
     u2 size
     u1 count
       { u1 AID_length
         u1 AID[AID_length]
         u2 install_method_offset
       } applets[count]
}
```

The items in the `applet_component` structure are as follows:

`tag`

The tag item has the value `COMPONENT_Applet` (3).

`size`

The `size` item indicates the number of bytes in the `applet_component` structure, excluding the `tag` and `size` items. The value of the `size` item must be greater than zero.

`count`

The `count` item indicates the number of applets defined in this package. The value of the `count` item must be greater than zero.

`applets[]`

The `applets` item represents a table of variable-length structures each describing an applet defined in this package.

The items in each entry of the `applets` table are defined as follows:

`AID_length`

The `AID_length` item represents the number of bytes in the AID item. Valid values are between 5 and 16, inclusive.

`AID[]`

The `AID` item represents the Java Card platform name of the applet.

1. Restrictions placed on an applet definition are imposed by the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

Each applet is assigned an AID conforming to the ISO 7816-5 standard (Section 4.2, "AID-based Naming" on page 4-3). The RID (first 5 bytes) of all of the applet AIDs must have the same value. In addition, the RID of each applet AIDs must have the same value as the RID of the package defined in this CAP file.

`install_method_offset`

The value of the `install_method_offset` item must be a 16-bit offset into the `info` item of the Method Component (Section 6.9, "Method Component" on page 6-35). The item at that offset must be a `method_info` structure that represents the `static install(byte[],short,byte)` method of the applet.[1] The `install(byte[],short,byte)` method must be defined in a class that extends the `javacard.framework.applet` class, directly or indirectly. The `install(byte[],short,byte)` method is called to initialize the applet.

6.6 Import Component

The Import Component lists the set of packages imported by the classes in this package. It does not include an entry for the package defined in this CAP file. The Import Component is represented by the following structure:

```
import_component {
     u1 tag
     u2 size
     u1 count
     package_info packages[count]
}
```

The items in the `import_component` structure are as follows:

`tag`

The `tag` item has the value `COMPONENT_Import` (4).

`size`

The `size` item indicates the number of bytes in the `import_component` structure, excluding the `tag` and `size` items. The value of the `size` item must be greater than zero.

`count`

The `count` item indicates the number of items in the `packages` table. The value of the `count` item must be between 0 and 128, inclusive.

1. Restrictions placed on the install(byte[],short,byte) method of an applet are imposed by the *Runtime Environment Specification, Java Card Platform, Version* 2.2.2.

packages[]

The packages item represents a table of variable-length package_info structures as defined for package under Section 6.3, "Header Component" on page 6-6. The table contains an entry for each of the packages referenced in the CAP file, not including the package defined.

The major and minor version numbers specified in the package_info structure are equal to the major and minor versions specified in the imported package's export file. See Section 4.5, "Package Versions" on page 4-13 for a description of assigning and using package version numbers.

Components of this CAP file refer to an imported package by using a index in this packages table. The index is called a package token (Section 4.3.7.1, "Package" on page 4-8).

6.7 Constant Pool Component

The Constant Pool Component contains an entry for each of the classes, methods, and fields referenced by elements in the Method Component (Section 6.9, "Method Component" on page 6-35) of this CAP file. The referencing elements in the Method Component may be instructions in the methods or exception handler catch types in the exception handler table.

Entries in the Constant Pool Component reference elements in the Class Component (Section 6.8, "Class Component" on page 6-21), Method Component (Section 6.9, "Method Component" on page 6-35), and Static Field Component (Section 6.10, "Static Field Component" on page 6-41). The Import Component (Section 6.6, "Import Component" on page 6-13) is also accessed using a package token (Section 4.3.7.1, "Package" on page 4-8) to describe references to classes, methods and fields defined in imported packages. Entries in the Constant Pool Component do not reference other entries internal to itself.

The Constant Pool Component is described by the following structure:

```
constant_pool_component {
      u1 tag
      u2 size
      u2 count
      cp_info constant_pool[count]
}
```

The items in the constant_pool_component structure are as follows:

tag

The tag item has the value COMPONENT_ConstantPool (5).

size

The `size` item indicates the number of bytes in the `constant_pool_component` structure, excluding the `tag` and `size` items. The value of the `size` item must be greater than zero.

`count`

The `count` item represents the number entries in the `constant_pool[]` array. Valid values are between 0 and 65535, inclusive.

`constant_pool[]`

The `constant_pool[]` item represents an array of `cp_info` structures:

```
cp_info {
    u1 tag
    u1 info[3]
}
```

Each item in the `constant_pool[]` array is a 4-byte structure. Each structure must begin with a 1-byte tag indicating the kind of `cp_info` entry. The content and format of the 3-byte info array varies with the value of the tag. The valid tags and their values are listed in the following table.

TABLE 6-5 CAP File Constant Pool Tags

Constant Type	Tag
CONSTANT_Classref	1
CONSTANT_InstanceFieldref	2
CONSTANT_VirtualMethodref	3
CONSTANT_SuperMethodref	4
CONSTANT_StaticFieldref	5
CONSTANT_StaticMethodref	6

Java Card platform constant types (“Java Card constant types”) are more specific than those in Java class files. The categories indicate not only the type of the item referenced, but also the manner in which it is referenced.

For example, in the Java constant pool there is one constant type for method references, while in the Java Card platform constant pool (“Java Card constant pool”) there are three constant types for method references: one for virtual method invocations using the `invokevirtual` bytecode, one for super method invocations using the `invokespecial` bytecode, and one for static method

invocations using either the `invokestatic` or `invokespecial` bytecode.[1] The additional information provided by a constant type in Java Card technologies simplifies resolution of references.

There are no ordering constraints on constant pool entries. It is recommended, however, that `CONSTANT_InstanceFieldref` (Section 6.7.2, "CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref" on page 6-18) constants occur early in the array to permit using `getfield_T` and `putfield_T` bytecodes instead of *getfield_T_w* and `putfield_T_w` bytecodes. The former have 1-byte constant pool index parameters while the latter have 2-byte constant pool index parameters.

The first entry in the constant pool can not be an exception handler class that is referenced by a `catch_type_index` of an `exception_handler_info` structure. In such a case the value of the `catch_type_index` would be equal to 0, but the value of 0 in a `catch_type_index` is reserved to indicate an `exception_handler_info` structure that describes a *finally* block.

6.7.1 CONSTANT_Classref

The `CONSTANT_Classref_info` structure is used to represent a reference to a class or an interface. The class or interface may be defined in this package or in an imported package.

```
CONSTANT_Classref_info {
     u1 tag
     union {
          u2 internal_class_ref
          { u1 package_token
            u1 class_token
          } external_class_ref
     } class_ref
     u1 padding
}
```

The items in the `CONSTANT_Classref_info` structure are the following:

`tag`

The `tag` item has the value `CONSTANT_Classref` (1).

`class_ref`

1. The constant pool index parameter of an *invokespecial* bytecode is to a `CONSTANT_StaticMethodref` when the method referenced is a constructor or a private instance method. In these cases the method invoked is fully known when the `CAP` file is created. In the cases of virtual method and super method references, the method invoked is dependent upon an instance of a class and its hierarchy, both of which may be partially unknown when the `CAP` file is created.

The class_ref item represents a reference to a class or interface. If the class or interface is defined in this package the structure represents an internal_class_ref and the high bit of the structure is zero. If the class or interface is defined in another package the structure represents an external_class_ref and the high bit of the structure is one.

internal_class_ref

The internal_class_ref structure represents a 16-bit offset into the info item of the Class Component (Section 6.8, "Class Component" on page 6-21) to an interface_info or class_info structure. The interface_info or class_info structure must represent the referenced class or interface.

The value of the internal_class_ref item must be between 0 and 32767, inclusive, making the high bit equal to zero.

external_class_ref

The external_class_ref structure represents a reference to a class or interface defined in an imported package. The high bit of this structure is one.

package_token

The package_token item represents a package token (Section 4.3.7.1, "Package" on page 4-8) defined in the Import Component (Section 6.6, "Import Component" on page 6-13) of this CAP file. The value of this token must be a valid index into the packages table item of the import_component structure. The package represented at that index must be the imported package.

The value of the package token must be between 0 and 127, inclusive.

The high bit of the package_token item is equal to one.

class_token

The class_token item represents the token of the class or interface (Section 4.3.7.2, "Classes and Interfaces" on page 4-8) of the referenced class or interface. It has the value of the class token of the class as defined in the Export file of the imported package.

padding

The padding item has the value zero. It is present to make the size of a CONSTANT_Classref_info structure the same as all other constants in the constant_pool[] array.

6.7.2 CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref

References to instance fields, and virtual methods are represented by similar structures:

```
CONSTANT_InstanceFieldref_info {
     u1 tag
     class_ref class
     u1 token
}

CONSTANT_VirtualMethodref_info {
     u1 tag
     class_ref class
     u1 token
}

CONSTANT_SuperMethodref_info {
     u1 tag
     class_ref class
     u1 token
}
```

The items in these structures are as follows:

`tag`

The tag item of a `CONSTANT_InstanceFieldref_info` structure has the value `CONSTANT_InstanceFieldref` (2).

The tag item of a `CONSTANT_VirtualMethodref_info` structure has the value `CONSTANT_VirtualMethodref` (3).

The tag item of a `CONSTANT_SuperMethodref_info` structure has the value `CONSTANT_SuperMethodref` (4).

`class`

The `class` item represents the class associated with the referenced instance field, virtual method, or super method invocation. It is a `class_ref` structure (Section 6.7.1, "CONSTANT_Classref" on page 6-16). If the referenced class is defined in this package the high bit is equal to zero. If the reference class is defined in an imported package the high bit of this structure is equal to one.

The class referenced in the `CONSTANT_InstanceField_info` structure must be the class that contains the declaration of the instance field.

The class referenced in the `CONSTANT_VirtualMethodref_info` structure must be a class that contains a declaration or definition of the virtual method.

The class referenced in the `CONSTANT_SuperMethodref_info` structure must be the class that defines the method that contains the Java language-level super invocation.

token

The token item in the CONSTANT_InstanceFieldref_info structure represents an instance field token (Section 4.3.7.5, “Instance Fields” on page 4-9) of the referenced field. The value of the instance field token is defined within the scope of the class indicated by the class item.

The token item of the CONSTANT_VirtualMethodref_info structure represents the virtual method token (Section 4.3.7.6, “Virtual Methods” on page 4-9) of the referenced method. The virtual method token is defined within the scope of the hierarchy of the class indicated by the class item. If the referenced method is public or protected the high bit of the token item is zero. If the referenced method is package-visible the high bit of the token item is one. In this case the class item must represent a reference to a class defined in this package.

The token item of the CONSTANT_SuperMethodref_info structure represents the virtual method token (Section 4.3.7.6, “Virtual Methods” on page 4-9) of the referenced method. Unlike in the CONSTANT_VirtualMethodref_info structure, the virtual method token is defined within the scope of the hierarchy of the superclass of the class indicated by the class item. If the referenced method is public or protected the high bit of the token item is zero. If the referenced method is package-visible the high bit of the token item is one. In the latter case the class item must represent a reference to a class defined in this package and at least one superclass of the class that contains a definition of the virtual method must also be defined in this package.

6.7.3 CONSTANT_StaticFieldref and CONSTANT_StaticMethodref

References to static fields and methods are represented by similar structures:

```
CONSTANT_StaticFieldref_info {
     u1 tag
     union {
          { u1 padding
            u2 offset
          } internal_ref
          { u1 package_token
            u1 class_token
            u1 token
          } external_ref
     } static_field_ref
}

CONSTANT_StaticMethodref_info {
     u1 tag
     union {

          { u1 padding
           u2 offset
          } internal_ref
          { u1 package_token
```

```
            u1 class_token
            u1 token
          } external_ref
      } static_method_ref
}
```

The items in these structures are as follows:

`tag`

The `tag` item of a `CONSTANT_StaticFieldref_info` structure has the value `CONSTANT_StaticFieldref` (5).

The `tag` item of a `CONSTANT_StaticMethodref_info` structure has the value `CONSTANT_StaticMethodref` (6).

`static_field_ref and static_method_ref`

The `static_field_ref` and `static_method_ref` item represents a reference to a `static` field or `static` method, respectively. Static method references include references to static methods, constructors, and private virtual methods.

If the referenced item is defined in this package the structure represents an `internal_ref` and the high bit of the structure is zero. If the referenced item is defined in another package the structure represents an `external_ref` and the high bit of the structure is one.

`internal_ref`

The internal_ref item represents a reference to a `static` field or method defined in this package. The items in the structure are:

`padding`

The `padding` item is equal to 0.

`offset`

The `offset` item of a `CONSTANT_StaticFieldref_info` structure represents a 16-bit offset into the Static Field Image defined by the Static Field component (Section 6.10, “Static Field Component” on page 6-41) to this static field.

The `offset` item of a `CONSTANT_StaticMethodref_info` structure represents a 16-bit offset into the info item of the Method Component (Section 6.9, “Method Component” on page 6-35) to a `method_info` structure. The `method_info` structure must represent the referenced method.

`external_ref`

The `external_ref` item represents a reference to a static field or method defined in an imported package. The items in the structure are:

`package_token`

The package_token item represents a package token (Section 4.3.7.1, “Package” on page 4-8) defined in the Import Component (Section 6.6, “Import Component” on page 6-13) of this CAP file. The value of this token must be a valid index into the packages table item of the import_component structure. The package represented at that index must be the imported package.

The value of the package token must be between 0 and 127, inclusive.

The high bit of the package_token item is equal to one.

class_token

The class_token item represents the token (Section 4.3.7.2, “Classes and Interfaces” on page 4-8) of the class of the referenced class. It has the value of the class token of the class as defined in the Export file of the imported package.

The class indicated by the class_token item must define the referenced field or method.

token

The token item of a CONSTANT_StaticFieldref_info structure represents a static field token (Section 4.3.7.3, “Static Fields” on page 4-8) as defined in the Export file of the imported package. It has the value of the token of the referenced field.

The token item of a CONSTANT_StaticMethodref_info structure represents a static method token (Section 4.3.7.4, “Static Methods and Constructors” on page 4-8) as defined in the Export file of the imported package. It has the value of the token of the referenced method.

6.8 Class Component

The Class Component describes each of the classes and interfaces defined in this package. It does not contain complete access information and content details for each class and interface. Instead, the information included is limited to that required to execute operations associated with a particular class or interface, without performing verification. Complete details regarding the classes and interfaces defined in this package are included in the Descriptor Component (Section 6.13, “Descriptor Component” on page 6-49).

The information included in the Class Component for each interface is sufficient to uniquely identify the interface and to test whether or not a cast to that interface is valid.

The information included in the Class Component for each class is sufficient to resolve operations associated with instances of a class. The operations include creating an instance, testing whether or not a cast of the instance is valid, dispatching virtual method invocations, and dispatching interface method invocations. Also included is sufficient information to locate instance fields of type `reference`, including arrays.

The classes represented in the Class Component reference other entries in the Class Component in the form of superclass, superinterface and implemented interface references. When a superclass, superinterface or implemented interface is defined in an imported package the Import Component is used in the representation of the reference.

The classes represented in the Class Component also contain references to virtual methods defined in the Method Component (Section 6.9, "Method Component" on page 6-35) of this CAP file. References to virtual methods defined in imported packages are not explicitly described. Instead such methods are located through a superclass within the hierarchy of the class, where the superclass is defined in the same imported package as the virtual method.

The Constant Pool Component (Section 6.7, "Constant Pool Component" on page 6-14), Export Component (Section 6.12, "Export Component" on page 6-47), Descriptor Component (Section 6.13, "Descriptor Component" on page 6-49) and Debug Component (Section 6.14, "Debug Component" on page 6-57) reference classes and interfaces defined in the Class Component. No other CAP file components reference the Class Component.

The Class Component is represented by the following structure:

```
class_component {
     u1 tag
     u2 size
     u2 signature_pool_length
     type_descriptor signature_pool[]
     interface_info interfaces[]
}
```

The items in the `class_component` structure are as follows:

`tag`

The tag item has the value `COMPONENT_Class` (6).

`size`

The `size` item indicates the number of bytes in the `class_component` structure, excluding the `tag` and `size` items. The value of the `size` item must be greater than zero.

`signature_pool_length`

The signature_pool_length item indicates the number of bytes in the signature_pool[] item. The value of the signature_pool_length item must be zero if the package does not define any remote interfaces or remote classes.

signature_pool[]

The signature_pool[] item represents a list of variable-length type_descriptor structures. These descriptors represent the signatures of the remote methods.

interfaces[]

The interfaces item represents an array of interface_info structures. Each interface defined in this package is represented in the array. The entries are ordered based on hierarchy such that a superinterface has a lower index than any of its subinterfaces.

classes[]

The classes item represents a table of variable-length class_info structures. Each class defined in this package is represented in the array. The entries are ordered based on hierarchy such that a superclass has a lower index than any of its subclasses.

6.8.1 type_descriptor

The type_descriptor structure represents the type of a field or the signature of a method.

```
type_descriptor {
    u1 nibble_count;
    u1 type[(nibble_count+1) / 2];
}
```

The type_descriptor structure contains the following elements:

nibble_count

The nibble_count value represents the number of nibbles required to describe the type encoded in the type array.

type[]

The `type` array contains an encoded description of the type, composed of individual nibbles. If the `nibble_count` item is an odd number, the last nibble in the `type` array must be `0x0`. The values of the type descriptor nibbles are defined in the following table.

TABLE 6-6 Type Descriptor Values

Type	Value
`void`	`0x1`
`boolean`	`0x2`
`byte`	`0x3`
`short`	`0x4`
`int`	`0x5`
`reference`	`0x6`
`array of boolean`	`0xA`
`array of byte`	`0xB`
`array of short`	`0xC`
`array of int`	`0xD`
`array of reference`	`0xE`

Class `reference` types are described using the reference nibble `0x6`, followed by a 2-byte (4-nibble) `class_ref` structure. The `class_ref` structure is defined as part of the `CONSTANT_Classref_info` structure (Section 6.7.1, "CONSTANT_Classref" on page 6-16). For example, a field of type reference to `p1.c1` in a CAP file defining package `p0` is described as:

TABLE 6-7 Encoded Reference Type p1.c1

Nibble	Value	Description
0	`0x6`	reference
1	<p1>	package token (high bit on)
2		
3	<c1>	class token
4		
5	`0x0`	padding

The following are examples of the array types:

TABLE 6-8 Encoded Byte Array Type

Nibble	Value	Description
0	0xB	array of byte
1	0x0	padding

TABLE 6-9 Encoded Reference Array Type p1.c1

Nibble	Value	Description
0	0xE	array of reference
1	<p1>	package token (high bit on)
2		
3	<c1>	class token
4		
5	0x0	padding

Method signatures are encoded in the same way, with the return type of the method encoded at the end of the sequence of nibbles. The return type is encoded in as many nibbles as required to represent it. For example:

TABLE 6-10 Encoded Method Signature ()V

Nibble	Value	Description
0	0x1	void
1	0x0	padding

TABLE 6-11 Encoded Method Signature (Lp1.ci;)S

Nibble	Value	Description
0	0x6	reference
1	<p1>	package token (high bit on)
2		

TABLE 6-11 Encoded Method Signature (Lp1.ci;)S

Nibble	Value	Description
3	<c1>	class token
4		
5	0x4	short

6.8.2 interface_info and class_info

The `interface_info` and `class_info` structures represent interfaces and classes, respectively. The two are differentiated by the value of the high bit in the structures. They are defined as follows:

```
interface_info {
        u1 bitfield {
         bit[4] flags
         bit[4] interface_count
         }
         class_ref superinterfaces[interface_count]
     interface_name_info interface_name[1]
}

class_info {
        u1 bitfield {
         bit[4] flags
         bit[4] interface_count
}
class_ref super_class_ref
u1 declared_instance_size
u1 first_reference_token
u1 reference_count
u1 public_method_table_base
u1 public_method_table_count
u1 package_method_table_base
u1 package_method_table_count
u2 public_virtual_method_table[public_method_table_count]
u2 package_virtual_method_table[package_method_table_count]
 implemented_interface_info interfaces[interface_count]
remote_interface_info remote_interfaces [2]
}
```

1 The `interface_name[]` item is required if the value of `ACC_REMOTE` is one. This item must be omitted otherwise. See the description of this field for more information.

2 The `remote_interfaces` item is required if the value of `ACC_REMOTE` is one. This item must be omitted otherwise. See the description of this field for more information.

6.8.2.1 interface_info and class_info shared Items

`flags`

The `flags` item is a mask of modifiers used to describe this interface or class. Valid values are shown in the following table:

TABLE 6-12 CAP File Interface and Class Flags

Name	Value
ACC_INTERFACE	0x8
ACC_SHAREABLE	0x4
ACC_REMOTE	0x2

The `ACC_INTERFACE` flag indicates whether this `interface_info` or `class_info` structure represents an interface or a class. The value must be one if it represents an `interface_info` structure and zero if a *class_info* structure.

The `ACC_SHAREABLE` flag in an interface_info structure indicates whether this interface is shareable. The value of this flag must be one if and only if the interface is `javacard.framework.Shareable` interface or extends that interface directly or indirectly.

The `ACC_SHAREABLE` flag in a `class_info` structure indicates whether this class is shareable.[1] The value of this flag must be one if and only if this class or any of its superclasses implements an interface that is shareable.

The `ACC_REMOTE` flag indicates whether this class or interface is remote. The value of this flag must be one if and only if the class or interface satisfies the requirements defined in Section 2.2.6.1, "Remote Classes and Remote Interfaces" on page 2-12.

All other flag values are reserved. Their values must be zero.

`interface_count`

The `interface_count` item of the `interface_info` structure indicates the number of entries in the `superinterfaces[]` table item. The value represents the number of direct and indirect superinterfaces of this interface. Indirect superinterfaces are the set of superinterfaces of the direct superinterfaces. Valid values are between 0 and 14, inclusive.

The `interface_count` item of the `class_info` structure indicates the number of entries in the interfaces table item. The value represents the number of interfaces implemented by this class, including superinterfaces of those interfaces and potentially interfaces implemented by superclasses of this class. Valid values are between 0 and 15, inclusive.

1. A Java Card virtual machine uses the `ACC_SHAREABLE` flag to implement the firewall restrictions defined by the *Runtime Environment Specification, Java Card Platform, Version* 2.2.2.

6.8.2.2 interface_info Items

`superinterfaces[]`

The `superinterfaces[]` item of the `interface_info` structure is an array of `class_ref` structures representing the superinterfaces of this interface. The `class_ref` structure is defined as part of the `CONSTANT_Classref_info` structure (Section 6.7.1, "CONSTANT_Classref" on page 6-16). This array is empty if this interface has no superinterfaces. Both direct and indirect superinterfaces are represented in the array. Class Object is not included.

`interface_name[]`

The `interface_name[]` item represents interface name information required if the interface is remote. The `interface_name[]` item is defined by a `interface_name_info` structure. If the value of the `ACC_REMOTE` flag is zero, the structure is defined as:

```
interface_name_info {
    }
```

If the value of the `ACC_REMOTE` flag is one, the structure is defined as:

```
interface_name_info {
   u1 interface_name_length
   u1 interface_name[interface_name_length]
  }
```

The values in the `interface_name_info` structure are defined as follows:

`interface_name_length`

The `interface_name_length` item is the number of bytes in `interface_name[]` item.

`interface_name`

The item is a variable length representation of the name of this interface in UTF-8 format.

6.8.2.3 class_info Items

`super_class_ref`

The `super_class_ref` item of the `class_info` structure is a `class_ref` structure representing the superclass of this class. The `class_ref` structure is defined as part of the `CONSTANT_Classref_info` structure (Section 6.7.1, "CONSTANT_Classref" on page 6-16).

The `super_class_ref` item has the value of `0xFFFF` only if this class does not have a superclass. Otherwise the value of the `super_class_ref` item is limited only by the constraints of the `class_ref` structure.

`declared_instance_size`

The `declared_instance_size` item of the `class_info` structure represents the number of 16-bit cells required to represent the instance fields declared by this class. It does not include instance fields declared by superclasses of this class.

Instance fields of type int are represented in two 16-bit cells, while all other field types are represented in one 16-bit cell.

`first_reference_token`

The `first_reference_token` item of the `class_info` structure represents the instance field token (Section 4.3.7.5, "Instance Fields" on page 4-9) value of the first reference type instance field defined by this class. It does not include instance fields defined by superclasses of this class.

If this class does not define any reference type instance fields, the value of the `first_reference_token` is `0xFF`. Otherwise the value of the `first_reference_token` item must be within the range of the set of instance field tokens of this class.

`reference_count`

The `reference_count` item of the `class_info` structure represents the number of reference type instance field defined by this class. It does not include reference type instance fields defined by superclasses of this class.

Valid values of the `reference_count` item are between 0 and the maximum number of instance fields defined by this class.

`public_method_table_base`

The `public_method_table_base` item of the class_info structure is equal to the virtual method token value (Section 4.3.7.6, "Virtual Methods" on page 4-9) of the first method in the public_virtual_method_table[] array. If the `public_virtual_method_table[]` array is empty, the value of the `public_method_table_base` item is equal to the public_method_table_base item of the `class_info` structure of this class' superclass plus the `public_method_table_count` item of the `class_info` structure of this class' superclass. If this class has no superclass and the `public_virtual_method_table[]` array is empty, the value of the `public_method_table_base` item is zero.

`public_method_table_count`

The `public_method_table_count` item of the class_info structure indicates the number of entries in the `public_virtual_method_table[]` array.

If this class does not define any public or protected override methods, the minimum valid value of `public_method_table_count` item is the number of public and protected virtual methods declared by this class. If this class defines one or more public or protected override methods, the minimum valid value of

public_method_table_count item is the value of the largest public or protected virtual method token, minus the value of the smallest public or protected virtual override method token, plus one.

The maximum valid value of the public_method_table_count item is the value of the largest public or protected virtual method token, plus one.

Any value for the public_method_table_count item between the minimum and maximum specified here is valid. However, the value must correspond to the number of entries in the public_virtual_method_table[] array.

package_method_table_base

The package_method_table_base item of the class_info structure is equal to the virtual method token value (Section 4.3.7.6, "Virtual Methods" on page 4-9) of the first entry in the package_virtual_method_table[] array. If the package_virtual_method_table[] array is empty, the value of the package_method_table_base item is equal to the package_method_table_base item of the class_info structure of this class' superclass, plus the package_method_table_count item of the class_info structure of this class' superclass. If this class has no superclass or inherits from a class defined in another package and the package_virtual_method_table[] array is empty, the value of the package_method_table_base item is zero.

package_method_table_count

The package_method_table_count item of the class_info structure indicates the number of entries in the package_virtual_method_table[] array.

If this class does not define any override methods, the minimum valid value of package_method_table_count item is the number of package visible virtual methods declared by this class. If this class defines one or more package visible override methods, the minimum valid value of package_method_table_count item is the value of the largest package visible virtual method token, minus the value of the smallest package visible virtual override method token, plus one.

The maximum valid value of the package_method_table_count item is the value of the largest package visible method token, plus one.

Any value for the package_method_table_count item between the minimum and maximum specified here are valid. However, the value must correspond to the number of entries in the package_virtual_method_table[].

public_virtual_method_table[]

The public_virtual_method_table[] item of the class_info structure represents an array of public and protected virtual methods. These methods can be invoked on an instance of this class. The public_virtual_method_table[] array includes methods declared or defined by this class. It may also include methods declared or defined by any or all of its superclasses. The value of an index into this table must be equal to the value of the virtual method token of the indicated method, minus the value of the public_method_table_base item.

Entries in the public_virtual_method_table[] array that represent methods defined or declared in this package contain offsets into the info item of the Method Component (Section 6.9, "Method Component" on page 6-35) to the method_info structure representing the method. Entries that represent methods defined or declared in an imported package contain the value 0xFFFF.

Entries for methods that are declared abstract are represented in the public_virtual_method_table[] array in the same way as non-abstract methods.

package_virtual_method_table[]

The package_virtual_method_table[] item of the class_info structure represents an array of package-visible virtual methods. These methods can be invoked on an instance of this class. The package_virtual_method_table[] array includes methods declared or defined by this class. It may also include methods declared or defined by any or all of its superclasses that are defined in this package. The value of an index into this table must be equal to the value of the virtual method token of the indicated method & 0x7F, minus the value of the package_method_table_base item.

All entries in the package_virtual_method_table[] array represent methods defined or declared in this package. They contain offsets into the info item of the Method Component (Section 6.9, "Method Component" on page 6-35) to the method_info structure representing the method.

Entries for methods that are declared abstract, not including those defined by interfaces, are represented in the package_virtual_method_table[] array in the same way as non-abstract methods.

interfaces[]

The interfaces item of the class_info structure represents a table of variable-length implemented_interface_info structures. The table must contain an entry for each of the directly implemented interfaces indicated in the declaration of this class and each of the interfaces in the hierarchies of those interfaces. Interfaces that occur more than once are represented by a single entry.

Given the declarations below, the number of entries for class c0 is 1 and the entry in the interfaces array is i0. The number of entries for class c1 is 3 and the entries in the interfaces array are i1, i2, and i3. The entries for class c1 must not include interface i0, which is implemented only by the superclass of c1.

```
interface i0 {}
interface i1 {}
interface i2 extends i1 {}
interface i3 {}
class c0 implements i0 {}
class c1 extends c0 implements i2, i3 {}
```

remote_interfaces

The `remote_interfaces` item represents information required if this class or any of its super classes implements a rets information required if this class or any of its super classes implements a rets information required if this class or plements a rets information required if this class or n required if this class or

all member mnemonics and opcodes. For example, the forms line for the sconst_<s> family of instructions, giving mnemonic and opcode information for the two instructions in that family (sconst_0 and sconst_1), is

Forms sconst_0 = 3 (0x3),
sconst_1 = 4 (0x4)

In the description of the Java Card virtual machine instructions, the effect of an instruction's execution on the operand stack (Section 3.5, "Frames" on page 3-3) of the current frame (Section 3.5, "Frames" on page 3-3) is represented textually, with the stack growing from left to right and each word represented separately. Thus,

Stack…, value1, value2 ->
…, result

shows an operation that begins by having a one-word value2 on top of the operand stack with a one-word value1 just beneath it. As a result of the execution of the instruction, value1 and value2 are popped from the operand stack and replaced by a one-word result, which has been calculated by the instruction. The remainder of the operand stack, represented by an ellipsis (…), is unaffected by the instruction's execution.

The type int takes two words on the operand stack. In the operand stack representation, each word is represented separately using a dot notation:

Stack…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

The Virtual Machine Specification for the Java Card Platform, v2.2.2 does not mandate how the two words are used to represent the 32-bit int value; it only requires that a particular implementation be internally consistent.

7.5.1 aaload

Load reference from array

Format

aaload

Forms

aaload = 36 (0x24)

Stack

…, arrayref, index ->
…, value

Description

The arrayref must be of type reference and must refer to an array whose components are of type reference. The index must be of type short. Both arrayref and index are popped from the operand stack. The reference value in the component of the array at index is retrieved and pushed onto the top of the operand stack.

Runtime Exceptions

If arrayref is null, aaload throws a NullPointerException.

Otherwise, if index is not within the bounds of the array referenced by arrayref, the aaload instruction throws an ArrayIndexOutOfBoundsException.

Notes

In some circumstances, the aaload instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

7.5.2 aastore

Store into reference array

Format

aastore

Forms

aastore = 55 (0x37)

Stack

…, arrayref, index, value ->
…

Description

The arrayref must be of type reference and must refer to an array whose components are of type reference. The index must be of type short and the value must be of type reference. The arrayref, index and value are popped from the operand stack. The reference value is stored as the component of the array at index.

At runtime the type of value must be confirmed to be assignment compatible with the type of the components of the array referenced by arrayref. Assignment of a value of reference type S (source) to a variable of reference type T (target) is allowed only when the type S supports all of the operations defined on type T. The detailed rules follow:

- If S is a class type, then:
  - If T is a class type, then S must be the same class as T, or S must be a subclass of T;
  - If T is an interface type, then S must implement interface T.
- If S is an interface type[1], then:
  - If T is a class type, then T must be Object (Section 2.2.2.4, "Classes" on page 2-7);
  - If T is an interface type, T must be the same interface as S or a superinterface of S.
- If S is an array type, namely the type SC[], that is, an array of components of type SC, then:
  - If T is a class type, then T must be Object.
  - If T is an array type, namely the type TC[], an array of components of type TC, then one of the following must be true:

    TC and SC are the same primitive type (Section 3.1, "Data Types and Values" on page 3-1").

    TC and SC are reference types[2] (Section 3.1, "Data Types and Values" on page 3-1) with type SC assignable to TC, by these rules.
- If T is an interface type, T must be one of the interfaces implemented by arrays.

Runtime Exceptions

If arrayref is null, aastore throws a NullPointerException.

Otherwise, if index is not within the bounds of the array referenced by arrayref, the aastore instruction throws an ArrayIndexOutOfBoundsException.

Otherwise, if arrayref is not null and the actual type of value is not assignment compatible with the actual type of the component of the array, aastore throws an ArrayStoreException.

Notes

1. When both *S* and *T* are arrays of reference types, this algorithm is applied recursively using the types of the arrays, namely *SC* and *TC*. In the recursive call, *S*, which was *SC* in the original call, may be an interface type. This rule can only be reached in this manner. Similarly, in the recursive call,*T*, which was *TC* in the original call, may be an interface type.
2. This version of the Java Card virtual machine does not support multi-dimensional arrays. Therefore, neither *SC* or *TC* can be an array type.

In some circumstances, the aastore instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

7.5.3 aconst_null

Push null

Format

aconst_null

Forms

aconst_null = 1 (0x1)

Stack

… ->
…, null

Description

Push the null object reference onto the operand stack.

7.5.4 aload

Load reference from local variable

Format

aload
index

Forms

aload = 21 (0x15)

Stack

… ->
…, objectref

Description

The index is an unsigned byte that must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variable at index must contain a reference. The objectref in the local variable at index is pushed onto the operand stack.

Notes

The `aload` instruction cannot be used to load a value of type returnAddress from a local variable onto the operand stack. This asymmetry with the `astore` instruction is intentional.

7.5.5 aload_<n>

Load reference from local variable

Format

aload_<n>

Forms

aload_0 = 24 (0x18)
aload_1 = 25 (0x19)
aload_2 = 26 (0x1a)
aload_3 = 27 (0x1b)

Stack

… ->
…, objectref

Description

The <n> must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variable at <n> must contain a reference. The objectref in the local variable at <n> is pushed onto the operand stack.

Notes

An aload_<n> instruction cannot be used to load a value of type returnAddress from a local variable onto the operand stack. This asymmetry with the corresponding astore_<n> instruction is intentional.

Each of the aload_<n> instructions is the same as aload with an index of <n>, except that the operand <n> is implicit.

7.5.6 anewarray

Create new array of reference

Format

anewarray
indexbyte1
indexbyte2

Forms

anewarray = 145 (0x91)

Stack

…, count ->
…, arrayref

Description

The count must be of type short. It is popped off the operand stack. The count represents the number of components of the array to be created. The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The item at that index in the constant pool must be of type CONSTANT_Classref (Section 6.7.1, “CONSTANT_Classref” on page 6-16), a reference to a class or interface type. The reference is resolved. A new array with components of that type, of length count, is allocated from the heap, and a reference arrayref to this new array object is pushed onto the operand stack. All components of the new array are initialized to null, the default value for reference types.

Runtime Exception

If count is less than zero, the anewarray instruction throws a NegativeArraySizeException.

7.5.7 areturn

Return reference from method

Format

areturn

Forms

areturn = 119 (0x77)

Stack

…, objectref ->
[empty]

Description

The objectref must be of type reference. The objectref is popped from the operand stack of the current frame (Section 3.5, “Frames” on page 3-3) and pushed onto the operand stack of the frame of the invoker. Any other values on the operand stack of the current method are discarded.

The virtual machine then reinstates the frame of the invoker and returns control to the invoker.

7.5.8 arraylength

Get length of array

Format

arraylength

Forms

arraylength = 146 (0x92)

Stack

…, arrayref ->
…, length

Description

The arrayref must be of type reference and must refer to an array. It is popped from the operand stack. The length of the array it references is determined. That length is pushed onto the top of the operand stack as a short.

Runtime Exception

If arrayref is null, the arraylength instruction throws a NullPointerException.

Notes

In some circumstances, the arraylength instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

7.5.9 astore

Store reference into local variable

Format

astore
index

Forms

astore = 40 (0x28)

Stack

…, objectref ->
…

Description

The index is an unsigned byte that must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The objectref on the top of the operand stack must be of type returnAddress or of type reference. The objectref is popped from the operand stack, and the value of the local variable at index is set to objectref.

Notes

The astore instruction is used with an objectref of type returnAddress when implementing Java’s finally keyword. The aload instruction cannot be used to load a value of type returnAddress from a local variable onto the operand stack. This asymmetry with the astore instruction is intentional.

7.5.10 astore_<n>

Store reference into local variable

Format

astore_<n>

Forms

astore_0 = 43 (0x2b)
astore_1 = 44 (0x2c)
astore_2 = 45 (0x2d)
astore_3 = 46 (0x2e)

Stack

…, objectref ->
…

Description

The <n> must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The objectref on the top of the operand stack must be of type returnAddress or of type reference. It is popped from the operand stack, and the value of the local variable at <n> is set to objectref.

Notes

An astore_<n> instruction is used with an objectref of type returnAddress when implementing Java’s finally keyword. An aload_<n> instruction cannot be used to load a value of type returnAddress from a local variable onto the operand stack. This asymmetry with the corresponding astore_<n> instruction is intentional.

Each of the astore_<n> instructions is the same as astore with an index of <n>, except that the operand <n> is implicit.

7.5.11 athrow

Throw exception or error

Format

athrow

Forms

athrow = 147 (0x93)

Stack

…, objectref ->
objectref

Description

The objectref must be of type reference and must refer to an object that is an instance of class Throwable or of a subclass of Throwable. It is popped from the operand stack. The objectref is then thrown by searching the current frame (Section 3.5, "Frames" on page 3-3) for the most recent catch clause that catches the class of objectref or one of its superclasses.

If a catch clause is found, it contains the location of the code intended to handle this exception. The pc register is reset to that location, the operand stack of the current frame is cleared, objectref is pushed back onto the operand stack, and execution continues. If no appropriate clause is found in the current frame, that frame is popped, the frame of its invoker is reinstated, and the objectref is rethrown.

If no catch clause is found that handles this exception, the virtual machine exits.

Runtime Exception

If objectref is null, athrow throws a NullPointerException instead of objectref.

Notes

In some circumstances, the athrow instruction may throw a SecurityException if the current context (Section 3.4, "Contexts" on page 3-2) is not the owning context (Section 3.4, "Contexts" on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

7.5.12 baload

Load byte or boolean from array

Format

baload

Forms

baload = 37 (0x25)

Stack

…, arrayref, index ->
…, value

Description

The arrayref must be of type reference and must refer to an array whose components are of type byte or of type boolean. The index must be of type short. Both arrayref and index are popped from the operand stack. The byte value in the component of the array at index is retrieved, sign-extended to a short value, and pushed onto the top of the operand stack.

Runtime Exceptions

If arrayref is null, baload throws a NullPointerException.

Otherwise, if index is not within the bounds of the array referenced by arrayref, the baload instruction throws an ArrayIndexOutOfBoundsException.

Notes

In some circumstances, the baload instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

7.5.13 bastore

Store into byte or boolean array

Format

bastore

Forms

bastore = 56 (0x38)

Stack

…, arrayref, index, value ->
…

Description

The arrayref must be of type reference and must refer to an array whose components are of type byte or of type boolean. The index and value must both be of type short. The arrayref, index and value are popped from the operand stack. The short value is truncated to a byte and stored as the component of the array indexed by index.

Runtime Exceptions

If arrayref is null, bastore throws a NullPointerException.

Otherwise, if index is not within the bounds of the array referenced by arrayref, the bastore instruction throws an ArrayIndexOutOfBoundsException.

Notes

In some circumstances, the bastore instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

7.5.14 bipush

Push byte

Format

bipush
byte

Forms

bipush = 18 (0x12)

Stack

… ->
…, value.word1, value.word2

Description

The immediate byte is sign-extended to an int, and the resulting value is pushed onto the operand stack.

Note – If a virtual machine does not support the int data type, the bipush instruction will not be available.

7.5.15 bspush

Push byte

Format

bspush
byte

Forms

bspush = 16 (0x10)

Stack

… ->
…, value

Description

The immediate byte is sign-extended to a short, and the resulting value is pushed onto the operand stack.

7.5.16 checkcast

Check whether object is of given type

Format

checkcast
atype
indexbyte1
indexbyte2

Forms

checkcast = 148 (0x94)

Stack

…, objectref ->
…, objectref

Description

The unsigned byte atype is a code that indicates if the type against which the object is being checked is an array type or a class type. It must take one of the following values or zero:

TABLE 7-2 Array Values

Array Type	atype
T_BOOLEAN	10
T_BYTE	11
T_SHORT	12
T_INT	13
T_REFERENCE	14

If the value of atype is 10, 11, 12, or 13, the values of the indexbyte1 and indexbyte2 must be zero, and the value of atype indicates the array type against which to check the object. Otherwise the unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, "Frames" on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The item at that index in the constant pool must be of type CONSTANT_Classref (Section 6.7.1, "CONSTANT_Classref" on page 6-16), a reference to a class or interface type. The reference is resolved. If the value of atype is 14, the object is checked against an array type that is an array of object references of the type of the resolved class. If the value of atype is zero, the object is checked against a class or interface type that is the resolved class.

The objectref must be of type reference. If objectref is null or can be cast to the specified array type or the resolved class or interface type, the operand stack is unchanged; otherwise the checkcast instruction throws a ClassCastException.

The following rules are used to determine whether an objectref that is not null can be cast to the resolved type: if S is the class of the object referred to by objectref and T is the resolved class, array or interface type, checkcast determines whether objectref can be cast to type T as follows:

- If S is a class type, then:
  - If T is a class type, then S must be the same class as T, or S must be a subclass of T;
  - If T is an interface type, then S must implement interface T.
- If S is an interface type[1], then:

1. When both *S* and *T* are arrays of reference types, this algorithm is applied recursively using the types of the arrays, namely *SC* and *TC*. In the recursive call, *S*, which was *SC* in the original call, may be an interface type. This rule can only be reached in this manner. Similarly, in the recursive call,*T*, which was *TC* in the original call, may be an interface type.

- If T is a class type, then T must be Object (Section 2.2.2.4, “Classes” on page 2-7);
- If T is an interface type, T must be the same interface as S or a superinterface of S.

- If S is an array type, namely the type SC[], that is, an array of components of type SC, then:
  - If T is a class type, then T must be Object.
  - If T is an array type, namely the type TC[], an array of components of type TC, then one of the following must be true:

    TC and SC are the same primitive type (Section 3.1, “Data Types and Values” on page 3-1).

    TC and SC are reference types[1] (Section 3.1, “Data Types and Values” on page 3-1) with type SC assignable to TC, by these rules.
  - If T is an interface type, T must be one of the interfaces implemented by arrays.

Runtime Exception

If objectref cannot be cast to the resolved class, array, or interface type, the checkcast instruction throws a ClassCastException.

Notes

The checkcast instruction is fundamentally very similar to the instanceof instruction. It differs in its treatment of null, its behavior when its test fails (checkcast throws an exception, instanceof pushes a result code), and its effect on the operand stack.

In some circumstances, the checkcast instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the value of atype may not be 13 (array type = T_INT).

7.5.17 dup

Duplicate top operand stack word

1. This version of the Java Card virtual machine specification does not support multi-dimensional arrays. Therefore, neither *SC* or *TC* can be an array type.

Format

dup

Forms

dup = 61 (0x3d)

Stack

…, word ->
…, word, word

Description

The top word on the operand stack is duplicated and pushed onto the operand stack.

The dup instruction must not be used unless word contains a 16-bit data type.

Notes

Except for restrictions preserving the integrity of 32-bit data types, the dup instruction operates on an untyped word, ignoring the type of data it contains.

7.5.18 dup_x

Duplicate top operand stack words and insert below

Format

dup_x
mn

Forms

dup_x = 63 (0x3f)

Stack

…, wordN, …, wordM, …, word1 ->
…, wordM, …, word1, wordN, …, wordM, …, word1

Description

The unsigned byte mn is used to construct two parameter values. The high nibble, (mn & 0xf0) >> 4, is used as the value m. The low nibble, (mn & 0xf), is used as the value n. Permissible values for m are 1 through 4. Permissible values for n are 0 and m through m+4.

For positive values of n, the top m words on the operand stack are duplicated and the copied words are inserted n words down in the operand stack. When n equals 0, the top m words are copied and placed on top of the stack.

The dup_x instruction must not be used unless the ranges of words 1 through m and words m+1 through n each contain either a 16-bit data type, two 16-bit data types, a 32-bit data type, a 16-bit data type and a 32-bit data type (in either order), or two 32-bit data types.

Notes

Except for restrictions preserving the integrity of 32-bit data types, the dup_x instruction operates on untyped words, ignoring the types of data they contain.

If a virtual machine does not support the int data type, the permissible values for m are 1 or 2, and permissible values for n are 0 and m through m+2.

7.5.19 dup2

Duplicate top two operand stack words

Format

dup2

Forms

dup2 = 62 (0x3e)

Stack

…, word2, word1 ->
…, word2, word1, word2, word1

Description

The top two words on the operand stack are duplicated and pushed onto the operand stack, in the original order.

The dup2 instruction must not be used unless each of word1 and word2 is a word that contains a 16-bit data type or both together are the two words of a single 32-bit datum.

Notes

Except for restrictions preserving the integrity of 32-bit data types, the dup2 instruction operates on untyped words, ignoring the types of data they contain.

7.5.20 getfield_<t>

Fetch field from object

Format

getfield_<t>
index

Forms

getfield_a = 131 (0x83)
getfield_b = 132 (0x84)
getfield_s = 133 (0x85)
getfield_i = 134 (0x86)

Stack

…, objectref ->
…, value

OR

…, objectref ->
…, value.word1, value.word2

Description

The objectref, which must be of type reference, is popped from the operand stack. The unsigned index is used as an index into the constant pool of the current package (Section 3.5, "Frames" on page 3-3). The constant pool item at the index must be of type CONSTANT_InstanceFieldref (Section 6.7.2, "CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref" on page 6-18), a reference to a class and a field token.

The class of objectref must not be an array. If the field is protected, and it is a member of a superclass of the current class, and the field is not declared in the same package as the current class, then the class of objectref must be either the current class or a subclass of the current class.

The item must resolve to a field with a type that matches t, as follows:

- a field must be of type reference

- b field must be of type byte or type boolean
- s field must be of type short
- i field must be of type int

The width of a field in a class instance is determined by the field type specified in the instruction. The item is resolved, determining the field offset[1]. The value at that offset into the class instance referenced by objectref is fetched. If the value is of type byte or type boolean, it is sign-extended to a short. The value is pushed onto the operand stack.

Runtime Exception

If objectref is null, the getfield_<t> instruction throws a NullPointerException.

Notes

In some circumstances, the getfield_<t> instruction may throw a SecurityException if the current context (Section 3.4, "Contexts" on page 3-2) is not the owning context (Section 3.4, "Contexts" on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the getfield_i instruction will not be available.

7.5.21 getfield_<t>_this

Fetch field from current object

Format

getfield_<t>_this
index

Forms

getfield_a_this = 173 (0xad)
getfield_b_this = 174 (0xae)
getfield_s_this = 175 (0xaf)
getfield_i_this = 176 (0xb0)

Stack

1. The offset may be computed by adding the field token value to the size of an instance of the immediate superclass. However, this method is not required by this specification. A Java Card virtual machine may define any mapping from token value to offset into an instance.

… ->
…, value

OR

… ->
…, value.word1, value.word2

Description

The currently executing method must be an instance method. The local variable at index 0 must contain a reference objectref to the currently executing method's this parameter. The unsigned index is used as an index into the constant pool of the current package (Section 3.5, "Frames" on page 3-3). The constant pool item at the index must be of type CONSTANT_InstanceFieldref (Section 6.7.2, "CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref" on page 6-18), a reference to a class and a field token.

The class of objectref must not be an array. If the field is protected, and it is a member of a superclass of the current class, and the field is not declared in the same package as the current class, then the class of objectref must be either the current class or a subclass of the current class.

The item must resolve to a field with a type that matches t, as follows:

- a field must be of type reference
- b field must be of type byte or type boolean
- s field must be of type short
- i field must be of type int

The width of a field in a class instance is determined by the field type specified in the instruction. The item is resolved, determining the field offset[1]. The value at that offset into the class instance referenced by objectref is fetched. If the value is of type byte or type boolean, it is sign-extended to a short. The value is pushed onto the operand stack.

Runtime Exception

If objectref is null, the getfield_<t>_this instruction throws a NullPointerException.

Notes

1. The offset may be computed by adding the field token value to the size of an instance of the immediate superclass. However, this method is not required by this specification. A Java Card virtual machine may define any mapping from token value to offset into an instance.

In some circumstances, the getfield_<t>_this instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the getfield_i_this instruction will not be available.

7.5.22 getfield_<t>_w

Fetch field from object (wide index)

Format

getfield_<t>_w
indexbyte1
indexbyte2

Forms

getfield_a_w = 169 (0xa9)
getfield_b_w = 170 (0xaa)
getfield_s_w = 171 (0xab)
getfield_i_w = 172 (0xac)

Stack

…, objectref ->
…, value

OR

…, objectref ->
…, value.word1, value.word2

Description

The objectref, which must be of type reference, is popped from the operand stack. The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The constant pool item at the index must be of type CONSTANT_InstanceFieldref (Section 6.7.2, “CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref” on page 6-18), a reference to a class and a field token. The item must resolve to a field of type reference.

The class of objectref must not be an array. If the field is protected, and it is a member of a superclass of the current class, and the field is not declared in the same package as the current class, then the class of objectref must be either the current class or a subclass of the current class.

The item must resolve to a field with a type that matches t, as follows:

- a field must be of type reference
- b field must be of type byte or type boolean
- s field must be of type short
- i field must be of type int

The width of a field in a class instance is determined by the field type specified in the instruction. The item is resolved, determining the field offset[1]. The value at that offset into the class instance referenced by objectref is fetched. If the value is of type byte or type boolean, it is sign-extended to a short. The value is pushed onto the operand stack.

Runtime Exception

If objectref is null, the getfield_<t>_w instruction throws a NullPointerException.

Notes

In some circumstances, the getfield_<t>_w instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context Section 3.4, “Contexts” on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the getfield_i_w instruction will not be available.

7.5.23 getstatic_<t>

Get static field from class

Format

getstatic_<t>
indexbyte1
indexbyte2

1. The offset may be computed by adding the field token value to the size of an instance of the immediate superclass. However, this method is not required by this specification. A Java Card virtual machine may define any mapping from token value to offset into an instance.

Forms

getstatic_a = 123 (0x7b)
getstatic_b = 124 (0x7c)
getstatic_s = 125 (0x7d)
getstatic_i = 126 (0x7e)

Stack

… ->
…, value

OR

… ->
…, value.word1, value.word2

Description

The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The constant pool item at the index must be of type CONSTANT_StaticFieldref (Section 6.7.3, “CONSTANT_StaticFieldref and CONSTANT_StaticMethodref” on page 6-19), a reference to a static field.

The item must resolve to a field with a type that matches t, as follows:

- a field must be of type reference
- b field must be of type byte or type boolean
- s field must be of type short
- i field must be of type int

The width of a class field is determined by the field type specified in the instruction. The item is resolved, determining the field offset. The item is resolved, determining the class field. The value of the class field is fetched. If the value is of type byte or boolean, it is sign-extended to a short. The value is pushed onto the operand stack.

Notes

If a virtual machine does not support the int data type, the getstatic_i instruction will not be available.

7.5.24 goto

Branch always

Format

goto
branch

Forms

goto = 112 (0x70)

Stack

No change

Description

The value branch is used as a signed 8-bit offset. Execution proceeds at that offset from the address of the opcode of this goto instruction. The target address must be that of an opcode of an instruction within the method that contains this goto instruction.

7.5.25 goto_w

Branch always (wide index)

Format

goto_w
branchbyte1
branchbyte2

Forms

goto_w = 168 (0xa8)

Stack

No change

Description

The unsigned bytes branchbyte1 and branchbyte2 are used to construct a signed 16-bit branchoffset, where branchoffset is (branchbyte1 << 8) | branchbyte2. Execution proceeds at that offset from the address of the opcode of this goto instruction. The target address must be that of an opcode of an instruction within the method that contains this goto instruction.

7.5.26 i2b

Convert int to byte

Format

i2b

Forms

i2b = 93 (0x5d)

Stack

…, value.word1, value.word2 ->
…, result

Description

The value on top of the operand stack must be of type int. It is popped from the operand stack and converted to a byte result by taking the low-order 16 bits of the int value, and discarding the high-order 16 bits. The low-order word is truncated to a byte, then sign-extended to a short result. The result is pushed onto the operand stack.

Notes

The i2b instruction performs a narrowing primitive conversion. It may lose information about the overall magnitude of value. The result may also not have the same sign as value.

If a virtual machine does not support the int data type, the i2b instruction will not be available.

7.5.27 i2s

Convert int to short

Format

i2s

Forms

i2s = 94 (0x5e)

Stack

…, value.word1, value.word2 ->
…, result

Description

The value on top of the operand stack must be of type int. It is popped from the operand stack and converted to a short result by taking the low-order 16 bits of the int value and discarding the high-order 16 bits. The result is pushed onto the operand stack.

Notes

The i2s instruction performs a narrowing primitive conversion. It may lose information about the overall magnitude of value. The result may also not have the same sign as value.

If a virtual machine does not support the int data type, the i2s instruction will not be available.

7.5.28 iadd

Add int

Format

iadd

Forms

iadd = 66 (0x42)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. The int result is value1 + value2. The result is pushed onto the operand stack.

If an iadd instruction overflows, then the result is the low-order bits of the true mathematical result in a sufficiently wide two's-complement format. If overflow occurs, then the sign of the result may not be the same as the sign of the mathematical sum of the two values.

Notes

If a virtual machine does not support the int data type, the iadd instruction will not be available.

7.5.29 iaload

Load int from array

Format

iaload

Forms

iaload = 39 (0x27)

Stack

…, arrayref, index ->
…, value.word1, value.word2

Description

The arrayref must be of type reference and must refer to an array whose components are of type int. The index must be of type short. Both arrayref and index are popped from the operand stack. The int value in the component of the array at index is retrieved and pushed onto the top of the operand stack.

Runtime Exceptions

If arrayref is null, iaload throws a NullPointerException.

Otherwise, if index is not within the bounds of the array referenced by arrayref, the iaload instruction throws an ArrayIndexOutOfBoundsException.

Notes

In some circumstances, the iaload instruction may throw a SecurityException if the current context Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the iaload instruction will not be available.

7.5.30 iand

Boolean AND int

Format

iand

Forms

iand = 84 (0x54)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. They are popped from the operand stack. An int result is calculated by taking the bitwise AND (conjunction) of value1 and value2. The result is pushed onto the operand stack.

Notes

If a virtual machine does not support the int data type, the iand instruction will not be available.

7.5.31 iastore

Store into int array

Format

iastore

Forms

iastore = 58 (0x3a)

Stack

…, arrayref, index, value.word1, value.word2 ->
…

Description

The arrayref must be of type reference and must refer to an array whose components are of type int. The index must be of type short and value must be of type int. The arrayref, index and value are popped from the operand stack. The int value is stored as the component of the array indexed by index.

Runtime Exception

If arrayref is null, iastore throws a NullPointerException.

Otherwise, if index is not within the bounds of the array referenced by arrayref, the iastore instruction throws an ArrayIndexOutOfBoundsException.

Notes

In some circumstances, the iastore instruction may throw a SecurityException if the current context (Section 3.4, "Contexts" on page 3-2) is not the owning context (Section 3.4, "Contexts" on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the iastore instruction will not be available.

7.5.32 icmp

Compare int

Format

icmp

Forms

icmp = 95 (0x5f)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result

Description

Both value1 and value2 must be of type int. They are both popped from the operand stack, and a signed integer comparison is performed. If value1 is greater than value2, the short value 1 is pushed onto the operand stack. If value1 is equal to value2, the short value 0 is pushed onto the operand stack. If value1 is less than value2, the short value –1 is pushed onto the operand stack.

Notes

If a virtual machine does not support the int data type, the icmp instruction will not be available.

7.5.33 iconst_<i>

Push int constant

Format

iconst_<i>

Forms

iconst_m1 = 10 (0x09)
iconst_0 = 11 (0xa)
iconst_1 = 12 (0xb)
iconst_2 = 13 (0xc)
iconst_3 = 14 (0xd)
iconst_4 = 15 (0xe)
iconst_5 = 16 (0xf)

Stack

… ->
…, <i>.word1, <i>.word2

Description

Push the int constant <i> (-1, 0, 1, 2, 3, 4, or 5) onto the operand stack.

Notes

If a virtual machine does not support the int data type, the iconst_<i> instruction will not be available.

7.5.34 idiv

Divide int

Format

idiv

Forms

idiv = 72 (0x48)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. The int result is the value of the Java expression value1 / value2. The result is pushed onto the operand stack.

An int division rounds towards 0; that is, the quotient produced for int values in n/d is an int value q whose magnitude is as large as possible while satisfying | d · q | <= | n |. Moreover, q is a positive when | n | >= | d | and n and d have the same sign, but q is negative when | n | >= | d | and n and d have opposite signs.

There is one special case that does not satisfy this rule: if the dividend is the negative integer of the largest possible magnitude for the int type, and the divisor is –1, then overflow occurs, and the result is equal to the dividend. Despite the overflow, no exception is thrown in this case.

Runtime Exception

If the value of the divisor in an int division is 0, idiv throws an ArithmeticException.

Notes

If a virtual machine does not support the int data type, the idiv instruction will not be available.

7.5.35 if_acmp<cond>

Branch if reference comparison succeeds.

Format

if_acmp<cond>
branch

Forms

if_acmpeq = 104 (0x68)
if_acmpne = 105 (0x69)

Stack

…, value1, value2 ->
…

Description

Both value1 and value2 must be of type reference. They are both popped from the operand stack and compared. The results of the comparisons are as follows:

- eq succeeds if and only if value1 = value2
- ne succeeds if and only if value1 ¼ value2

If the comparison succeeds, branch is used as signed 8-bit offset, and execution proceeds at that offset from the address of the opcode of this if_acmp<cond> instruction. The target address must be that of an opcode of an instruction within the method that contains this if_acmp<cond> instruction.

Otherwise, execution proceeds at the address of the instruction following this if_acmp<cond> instruction.

7.5.36 if_acmp<cond>_w

Branch if reference comparison succeeds (wide index)

Format

if_acmp<cond>_w
branchbyte1
branchbyte2

Forms

if_acmpeq_w = 160 (0xa0)
if_acmpne_w = 161 (0xa1)

Stack

…, value1, value2 ->
…

Description

Both value1 and value2 must be of type reference. They are both popped from the operand stack and compared. The results of the comparisons are as follows:

- eq succeeds if and only if value1 = value2
- ne succeeds if and only if value1 ¼ value2

If the comparison succeeds, the unsigned bytes branchbyte1 and branchbyte2 are used to construct a signed 16-bit branchoffset, where branchoffset is (branchbyte1 << 8) | branchbyte2. Execution proceeds at that offset from the address of the opcode of this if_acmp<cond>_w instruction. The target address must be that of an opcode of an instruction within the method that contains this if_acmp<cond>_w instruction.

Otherwise, execution proceeds at the address of the instruction following this if_acmp<cond>_w instruction.

7.5.37 if_scmp<cond>

Branch if short comparison succeeds

Format

if_scmp<cond>
branch

Forms

if_scmpeq = 106 (0x6a)
if_scmpne = 107 (0x6b)
if_scmplt = 108 (0x6c)
if_scmpge = 109 (0x6d)
if_scmpgt = 110 (0x6e)
if_scmple = 111 (0x6f)

Stack

…, value1, value2 ->
…

Description

Both value1 and value2 must be of type short. They are both popped from the operand stack and compared. All comparisons are signed. The results of the comparisons are as follows:

- eq succeeds if and only if value1 = value2
- ne succeeds if and only if value1 ¼ value2
- lt succeeds if and only if value1 < value2
- le succeeds if and only if value1 £ value2
- gt succeeds if and only if value1 > value2
- ge succeeds if and only if value1 Š value2

If the comparison succeeds, branch is used as signed 8-bit offset, and execution proceeds at that offset from the address of the opcode of this if_scmp<cond> instruction. The target address must be that of an opcode of an instruction within the method that contains this if_scmp<cond> instruction.

Otherwise, execution proceeds at the address of the instruction following this if_scmp<cond> instruction.

7.5.38 if_scmp<cond>_w

Branch if short comparison succeeds (wide index)

Format

if_scmp<cond>_w
branchbyte1
branchbyte2

Forms

if_scmpeq_w = 162 (0xa2)
if_scmpne_w = 163 (0xa3)
if_scmplt_w = 164 (0xa4)
if_scmpge_w = 165 (0xa5)
if_scmpgt_w = 166 (0xa6)
if_scmple_w = 167 (0xa7)

Stack

…, value1, value2 ->
…

Description

Both value1 and value2 must be of type short. They are both popped from the operand stack and compared. All comparisons are signed. The results of the comparisons are as follows:

- eq succeeds if and only if value1 = value2
- ne succeeds if and only if value1 ¼ value2
- lt succeeds if and only if value1 < value2
- le succeeds if and only if value1 £ value2
- gt succeeds if and only if value1 > value2
- ge succeeds if and only if value1 Š value2

If the comparison succeeds, the unsigned bytes branchbyte1 and branchbyte2 are used to construct a signed 16-bit branchoffset, where branchoffset is (branchbyte1 << 8) | branchbyte2. Execution proceeds at that offset from the address of the opcode of this if_scmp<cond>_w instruction. The target address must be that of an opcode of an instruction within the method that contains this if_scmp<cond>_w instruction.

Otherwise, execution proceeds at the address of the instruction following this if_scmp<cond>_w instruction.

7.5.39 if<cond>

Branch if short comparison with zero succeeds

Format

if<cond>
branch

Forms

ifeq = 96 (0x60)
ifne = 97 (0x61)
iflt = 98 (0x62)
ifge = 99 (0x63)
ifgt = 100 (0x64)
ifle = 101 (0x65)

Stack

…, value ->
…

Description

The value must be of type short. It is popped from the operand stack and compared against zero. All comparisons are signed. The results of the comparisons are as follows:

- eq succeeds if and only if value = 0
- ne succeeds if and only if value ¼ 0
- lt succeeds if and only if value < 0
- le succeeds if and only if value £ 0
- gt succeeds if and only if value > 0
- ge succeeds if and only if value Š 0

If the comparison succeeds, branch is used as signed 8-bit offset, and execution proceeds at that offset from the address of the opcode of this if<cond> instruction. The target address must be that of an opcode of an instruction within the method that contains this if<cond> instruction.

Otherwise, execution proceeds at the address of the instruction following this if<cond> instruction.

7.5.40 if<cond>_w

Branch if short comparison with zero succeeds (wide index)

Format

if<cond>_w
branchbyte1
branchbyte2

Forms

ifeq_w = 152 (0x98)
ifne_w = 153 (0x99)
iflt_w = 154 (0x9a)
ifge_w = 155 (0x9b)
ifgt_w = 156 (0x9c)
ifle_w = 157 (0x9d)

Stack

…, value ->
…

Description

The value must be of type short. It is popped from the operand stack and compared against zero. All comparisons are signed. The results of the comparisons are as follows:

- eq succeeds if and only if value = 0
- ne succeeds if and only if value ¼ 0
- lt succeeds if and only if value < 0
- le succeeds if and only if value £ 0
- gt succeeds if and only if value > 0
- ge succeeds if and only if value Š 0

If the comparison succeeds, the unsigned bytes branchbyte1 and branchbyte2 are used to construct a signed 16-bit branchoffset, where branchoffset is (branchbyte1 << 8) | branchbyte2. Execution proceeds at that offset from the address of the opcode of this if<cond>_w instruction. The target address must be that of an opcode of an instruction within the method that contains this if<cond>_w instruction.

Otherwise, execution proceeds at the address of the instruction following this if<cond>_w instruction.

7.5.41 ifnonnull

Branch if reference not null

Format

ifnonnull
branch

Forms

ifnonnull = 103 (0x67)

Stack

…, value ->
…

Description

The value must be of type reference. It is popped from the operand stack. If the value is not null, branch is used as signed 8-bit offset, and execution proceeds at that offset from the address of the opcode of this ifnonnull instruction. The target address must be that of an opcode of an instruction within the method that contains this ifnonnull instruction.

Otherwise, execution proceeds at the address of the instruction following this ifnonnull instruction.

7.5.42 ifnonnull_w

Branch if reference not null (wide index)

Format

ifnonnull_w
branchbyte1
branchbyte2

Forms

ifnonnull_w = 159 (0x9f)

Stack

…, value ->
…

Description

The value must be of type reference. It is popped from the operand stack. If the value is not null, the unsigned bytes branchbyte1 and branchbyte2 are used to construct a signed 16-bit branchoffset, where branchoffset is (branchbyte1 << 8) | branchbyte2. Execution proceeds at that offset from the address of the opcode of this ifnonnull_w instruction. The target address must be that of an opcode of an instruction within the method that contains this ifnonnull_w instruction.

Otherwise, execution proceeds at the address of the instruction following this ifnonnull_w instruction.

7.5.43 ifnull

Branch if reference is null

Format

ifnull
branch

Forms

ifnull = 102 (0x66)

Stack

…, value ->
…

Description

The value must be of type reference. It is popped from the operand stack. If the value is null, branch is used as signed 8-bit offset, and execution proceeds at that offset from the address of the opcode of this ifnull instruction. The target address must be that of an opcode of an instruction within the method that contains this ifnull instruction.

Otherwise, execution proceeds at the address of the instruction following this ifnull instruction.

7.5.44 ifnull_w

Branch if reference is null (wide index)

Format

ifnull_w
branchbyte1
branchbyte2

Forms

ifnull_w = 158 (0x9e)

Stack

…, value ->
…

Description

The value must be of type reference. It is popped from the operand stack. If the value is null, the unsigned bytes branchbyte1 and branchbyte2 are used to construct a signed 16-bit branchoffset, where branchoffset is (branchbyte1 << 8) | branchbyte2. Execution proceeds at that offset from the address of the opcode of this ifnull_w instruction. The target address must be that of an opcode of an instruction within the method that contains this ifnull_w instruction.

Otherwise, execution proceeds at the address of the instruction following this ifnull_w instruction.

7.5.45 iinc

Increment local int variable by constant

Format

iinc
index
const

Forms

iinc = 90 (0x5a)

Stack

No change

Description

The index is an unsigned byte. Both index and index + 1 must be valid indices into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variables at index and index + 1 together must contain an int. The const is an immediate signed byte. The value const is first sign-extended to an int, then the int contained in the local variables at index and index + 1 is incremented by that amount.

Notes

If a virtual machine does not support the int data type, the iinc instruction will not be available.

7.5.46 iinc_w

Increment local int variable by constant

Format

iinc_w
index
byte1
byte2

Forms

iinc_w = 151 (0x97)

Stack

No change

Description

The index is an unsigned byte. Both index and index + 1 must be valid indices into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variables at index and index + 1 together must contain an int. The immediate unsigned byte1 and byte2 values are assembled into an intermediate short where the value of the short is (byte1 << 8) | byte2. The intermediate value is then sign-extended to an int const. The int contained in the local variables at index and index + 1 is incremented by const.

Notes

If a virtual machine does not support the int data type, the iinc_w instruction will not be available.

7.5.47 iipush

Push int

Format

iipush
byte1
byte2
byte3
byte4

Forms

iipush = 20 (0x14)

Stack

… ->
…, value1.word1, value1.word2

Description

The immediate unsigned byte1, byte2, byte3, and byte4 values are assembled into a signed int where the value of the int is (byte1 << 24) | (byte2 << 16) | (byte3 << 8) | byte4. The resulting value is pushed onto the operand stack.

Notes

If a virtual machine does not support the int data type, the iipush instruction will not be available.

7.5.48 iload

Load int from local variable

Format

iload
index

Forms

iload = 23 (0x17)

Stack

… ->
…, value1.word1, value1.word2

Description

The index is an unsigned byte. Both index and index + 1 must be valid indices into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variables at index and index + 1 together must contain an int. The value of the local variables at index and index + 1 is pushed onto the operand stack.

Notes

If a virtual machine does not support the int data type, the iload instruction will not be available.

7.5.49 iload_<n>

Load int from local variable

Format

iload_<n>

Forms

iload_0 = 32 (0x20)
iload_1 = 33 (0x21)
iload_2 = 34 (0x22)
iload_3 = 35 (0x23)

Stack

… ->
…, value1.word1, value1.word2

Description

Both <n> and <n> + 1 must be a valid indices into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variables at <n> and <n> + 1 together must contain an int. The value of the local variables at <n> and <n> + 1 is pushed onto the operand stack.

Notes

Each of the iload_<n> instructions is the same as iload with an index of <n>, except that the operand <n> is implicit.

If a virtual machine does not support the int data type, the iload_<n> instruction will not be available.

7.5.50 ilookupswitch

Access jump table by key match and jump

Format

ilookupswitch
defaultbyte1
defaultbyte2
npairs1
npairs2
match-offset pairs…

Pair Format

matchbyte1
matchbyte2
matchbyte3

matchbyte4
offsetbyte1
offsetbyte2

Forms

ilookupswitch = 118 (0x76)

Stack

…, key.word1, key.word2 ->
…

Description

An ilookupswitch instruction is a variable-length instruction. Immediately after the ilookupswitch opcode follow a signed 16-bit value default, an unsigned 16-bit value npairs, and then npairs pairs. Each pair consists of an int match and a signed 16-bit offset. Each match is constructed from four unsigned bytes as (matchbyte1 << 24) | (matchbyte2 << 16) | (matchbyte3 << 8) | matchbyte4. Each offset is constructed from two unsigned bytes as (offsetbyte1 << 8) | offsetbyte2.

The table match-offset pairs of the ilookupswitch instruction must be sorted in increasing numerical order by match.

The key must be of type int and is popped from the operand stack and compared against the match values. If it is equal to one of them, then a target address is calculated by adding the corresponding offset to the address of the opcode of this ilookupswitch instruction. If the key does not match any of the match values, the target address is calculated by adding default to the address of the opcode of this ilookupswitch instruction. Execution then continues at the target address.

The target address that can be calculated from the offset of each match-offset pair, as well as the one calculated from default, must be the address of an opcode of an instruction within the method that contains this ilookupswitch instruction.

Notes

The match-offset pairs are sorted to support lookup routines that are quicker than linear search.

If a virtual machine does not support the int data type, the ilookupswitch instruction will not be available.

7.5.51 imul

Multiply int

Format

imul

Forms

imul = 70 (0x46)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. The int result is value1 * value2. The result is pushed onto the operand stack.

If an imul instruction overflows, then the result is the low-order bits of the mathematical product as an int. If overflow occurs, then the sign of the result may not be the same as the sign of the mathematical product of the two values.

Notes

If a virtual machine does not support the int data type, the imul instruction will not be available.

7.5.52 ineg

Negate int

Format

ineg

Forms

ineg = 76 (0x4c)

Stack

…, value.word1, value.word2 ->
…, result.word1, result.word2

Description

The value must be of type int. It is popped from the operand stack. The int result is the arithmetic negation of value, -value. The result is pushed onto the operand stack.

For int values, negation is the same as subtraction from zero. Because the Java Card virtual machine uses two's-complement representation for integers and the range of two's-complement values is not symmetric, the negation of the maximum negative int results in that same maximum negative number. Despite the fact that overflow has occurred, no exception is thrown.

For all int values x, -x equals (~x) + 1.

Notes

If a virtual machine does not support the int data type, the ineg instruction will not be available.

7.5.53 instanceof

Determine if object is of given type

Format

instanceof
atype
indexbyte1
indexbyte2

Forms

instanceof = 149 (0x95)

Stack

…, objectref ->
…, result

Description

The unsigned byte atype is a code that indicates if the type against which the object is being checked is an array type or a class type. It must take one of the following values or zero:

TABLE 7-3 Array Values

Array Type	atype
T_BOOLEAN	10
T_BYTE	11
T_SHORT	12
T_INT	13
T_REFERENCE	14

If the value of atype is 10, 11, 12, or 13, the values of the indexbyte1 and indexbyte2 must be zero, and the value of atype indicates the array type against which to check the object. Otherwise the unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, "Frames" on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The item at that index in the constant pool must be of type CONSTANT_Classref (Section 6.7.1, "CONSTANT_Classref" on page 6-16), a reference to a class or interface type. The reference is resolved. If the value of atype is 14, the object is checked against an array type that is an array of object references of the type of the resolved class. If the value of atype is zero, the object is checked against a class or interface type that is the resolved class.

The objectref must be of type reference. It is popped from the operand stack. If objectref is not null and is an instance of the resolved class, array or interface, the instanceof instruction pushes a short result of 1 on the operand stack. Otherwise it pushes a short result of 0.

The following rules are used to determine whether an objectref that is not null is an instance of the resolved type: if S is the class of the object referred to by objectref and T is the resolved class, array or interface type, instanceof determines whether objectref is an instance of T as follows:

- If S is a class type, then:
  - If T is a class type, then S must be the same class as T, or S must be a subclass of T;
  - If T is an interface type, then S must implement interface T.
- If S is an interface type[1], then:

1. When both *S* and *T* are arrays of reference types, this algorithm is applied recursively using the types of the arrays, namely *SC* and *TC*. In the recursive call, *S*, which was *SC* in the original call, may be an interface type. This rule can only be reached in this manner. Similarly, in the recursive call,*T*, which was *TC* in the original call, may be an interface type.

- If T is a class type, then T must be Object (Section 2.2.2.4, “Classes” on page 2-7);
- If T is an interface type, T must be the same interface as S or a superinterface of S.

- If S is an array type, namely the type SC[], that is, an array of components of type SC, then:
  - If T is a class type, then T must be Object.
  - If T is an array type, namely the type TC[], an array of components of type TC, then one of the following must be true:

    TC and SC are the same primitive type (Section 3.1, “Data Types and Values” on page 3-1).

    TC and SC are reference types[1] (Section 3.1, “Data Types and Values” on page 3-1) with type SC assignable to TC, by these rules.
  - If T is an interface type, T must be one of the interfaces implemented by arrays.

Notes

The instanceof instruction is fundamentally very similar to the checkcast instruction. It differs in its treatment of null, its behavior when its test fails (checkcast throws an exception, instanceof pushes a result code), and its effect on the operand stack.

In some circumstances, the instanceof instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the value of atype may not be 13 (array type = T_INT).

7.5.54 invokeinterface

Invoke interface method

Format

invokeinterface
nargs

1. This version of the Java Card virtual machine specification does not support multi-dimensional arrays. Therefore, neither *SC* or *TC* can be an array type.

indexbyte1
indexbyte2
method

Forms

invokeinterface = 142 (0x8e)

Stack

…, objectref, [arg1, [arg2 …]] ->
…

Description

The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The constant pool item at that index must be of type CONSTANT_Classref (Section 6.7.1, “CONSTANT_Classref” on page 6-16), a reference to an interface class. The specified interface is resolved.

The nargs operand is an unsigned byte that must not be zero.

The method operand is an unsigned byte that is the interface method token for the method to be invoked. The interface method must not be <init> or an instance initialization method.

The objectref must be of type reference and must be followed on the operand stack by nargs – 1 words of arguments. The number of words of arguments and the type and order of the values they represent must be consistent with those of the selected interface method.

The interface table of the class of the type of objectref is determined. If objectref is an array type, then the interface table of class Object (Section 2.2.2.4, “Classes” on page 2-7) is used. The interface table is searched for the resolved interface. The result of the search is a table that is used to map the method token to a index.

The index is an unsigned byte that is used as an index into the method table of the class of the type of objectref. If the objectref is an array type, then the method table of class Object is used. The table entry at that index includes a direct reference to the method’s code and modifier information.

The nargs – 1 words of arguments and objectref are popped from the operand stack. A new stack frame is created for the method being invoked, and objectref and the arguments are made the values of its first nargs words of local variables, with objectref in local variable 0, arg1 in local variable 1, and so on. The new stack frame

is then made current, and the Java Card virtual machine pc is set to the opcode of the first instruction of the method to be invoked. Execution continues with the first instruction of the method.

Runtime Exception

If objectref is null, the invokeinterface instruction throws a NullPointerException.

Notes

In some circumstances, the invokeinterface instruction may throw a SecurityException if the current context (Section 3.4, "Contexts" on page 3-2) is not the context (Section 3.4, "Contexts" on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*. If the current context is not the object's context and the Java Card RE permits invocation of the method, the invokeinterface instruction will cause a context switch (Section 3.4, "Contexts" on page 3-2) to the object's context before invoking the method, and will cause a return context switch to the previous context when the invoked method returns.

7.5.55 invokespecial

Invoke instance method; special handling for superclass, private, and instance initialization method invocations

Format

invokespecial
indexbyte1
indexbyte2

Forms

invokespecial = 140 (0x8c)

Stack

…, objectref, [arg1, [arg2 …]] ->
…

Description

The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, "Frames" on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. If the invoked method is a private instance method or an instance initialization method, the constant pool item

at index must be of type CONSTANT_StaticMethodref (Section 6.7.3, "CONSTANT_StaticFieldref and CONSTANT_StaticMethodref" on page 6-19), a reference to a statically linked instance method. If the invoked method is a superclass method, the constant pool item at index must be of type CONSTANT_SuperMethodref (Section 6.7.2, "CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref" on page 6-18), a reference to an instance method of a specified class. The reference is resolved. The resolved method must not be <clinit>, a class or interface initialization method. If the method is <init>, an instance initialization method, then the method must only be invoked once on an uninitialized object, and before the first backward branch following the execution of the new instruction that allocated the object. Finally, if the resolved method is protected, and it is a member of a superclass of the current class, and the method is not declared in the same package as the current class, then the class of objectref must be either the current class or a subclass of the current class.

The resolved method includes the code for the method, an unsigned byte nargs that must not be zero, and the method's modifier information.

The objectref must be of type reference, and must be followed on the operand stack by nargs – 1 words of arguments, where the number of words of arguments and the type and order of the values they represent must be consistent with those of the selected instance method.

The nargs – 1 words of arguments and objectref are popped from the operand stack. A new stack frame is created for the method being invoked, and objectref and the arguments are made the values of its first nargs words of local variables, with objectref in local variable 0, arg1 in local variable 1, and so on. The new stack frame is then made current, and the Java Card virtual machine pc is set to the opcode of the first instruction of the method to be invoked. Execution continues with the first instruction of the method.

Runtime Exception

If objectref is null, the invokespecial instruction throws a NullPointerException.

7.5.56 invokestatic

Invoke a class (static) method

Format

invokestatic
indexbyte1
indexbyte2

Forms

invokestatic = 141 (0x8d)

Stack

…, [arg1, [arg2 …]] ->
…

Description

The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The constant pool item at that index must be of type CONSTANT_StaticMethodref (Section 6.7.3, “CONSTANT_StaticFieldref and CONSTANT_StaticMethodref” on page 6-19), a reference to a static method. The method must not be <init>, an instance initialization method, or <clinit>, a class or interface initialization method. It must be static, and therefore cannot be abstract.

The resolved method includes the code for the method, an unsigned byte nargs that may be zero, and the method’s modifier information.

The operand stack must contain nargs words of arguments, where the number of words of arguments and the type and order of the values they represent must be consistent with those of the resolved method.

The nargs words of arguments are popped from the operand stack. A new stack frame is created for the method being invoked, and the words of arguments are made the values of its first nargs words of local variables, with arg1 in local variable 0, arg2 in local variable 1, and so on. The new stack frame is then made current, and the Java Card virtual machine pc is set to the opcode of the first instruction of the method to be invoked. Execution continues with the first instruction of the method.

7.5.57 invokevirtual

Invoke instance method; dispatch based on class

Format

invokevirtual
indexbyte1
indexbyte2

Forms

invokevirtual = 139 (0x8b)

Stack

…, objectref, [arg1, [arg2 …]] ->
…

Description

The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The constant pool item at that index must be of type CONSTANT_VirtualMethodref (Section 6.7.2, “CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref” on page 6-18), a reference to a class and a virtual method token. The specified method is resolved. The method must not be <init>, an instance initialization method, or <clinit>, a class or interface initialization method. Finally, if the resolved method is protected, and it is a member of a superclass of the current class, and the method is not declared in the same package as the current class, then the class of objectref must be either the current class or a subclass of the current class.

The resolved method reference includes an unsigned index into the method table of the resolved class and an unsigned byte nargs that must not be zero.

The objectref must be of type reference. The index is an unsigned byte that is used as an index into the method table of the class of the type of objectref. If the objectref is an array type, then the method table of class Object (Section 2.2.2.4, “Classes” on page 2-7) is used. The table entry at that index includes a direct reference to the method’s code and modifier information.

The objectref must be followed on the operand stack by nargs – 1 words of arguments, where the number of words of arguments and the type and order of the values they represent must be consistent with those of the selected instance method.

The nargs – 1 words of arguments and objectref are popped from the operand stack. A new stack frame is created for the method being invoked, and objectref and the arguments are made the values of its first nargs words of local variables, with objectref in local variable 0, arg1 in local variable 1, and so on. The new stack frame is then made current, and the Java Card virtual machine pc is set to the opcode of the first instruction of the method to be invoked. Execution continues with the first instruction of the method.

Runtime Exception

If objectref is null, the invokevirtual instruction throws a NullPointerException.

In some circumstances, the invokevirtual instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the context (Section 3.4, “Contexts” on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*. If the current

context is not the object's context and the Java Card RE permits invocation of the method, the invokevirtual instruction will cause a context switch (Section 3.4, "Contexts" on page 3-2) to the object's context before invoking the method, and will cause a return context switch to the previous context when the invoked method returns.

7.5.58 ior

Boolean OR int

Format

ior

Forms

ior = 86 (0x56)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. An int result is calculated by taking the bitwise inclusive OR of value1 and value2. The result is pushed onto the operand stack.

Notes

If a virtual machine does not support the int data type, the ior instruction will not be available.

7.5.59 irem

Remainder int

Format

irem

Forms

irem = 74 (0x4a)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. The int result is the value of the Java expression value1 – (value1 / value2) * value2. The result is pushed onto the operand stack.

The result of the irem instruction is such that (a/b)*b + (a%b) is equal to a. This identity holds even in the special case that the dividend is the negative int of largest possible magnitude for its type and the divisor is –1 (the remainder is 0). It follows from this rule that the result of the remainder operation can be negative only if the dividend is negative and can be positive only if the dividend is positive. Moreover, the magnitude of the result is always less than the magnitude of the divisor.

Runtime Exception

If the value of the divisor for a short remainder operator is 0, irem throws an ArithmeticException.

Notes

If a virtual machine does not support the int data type, the irem instruction will not be available.

7.5.60 ireturn

Return int from method

Format

ireturn

Forms

ireturn = 121 (0x79)

Stack

…, value.word1, value.word2 ->
[empty]

Description

The value must be of type int. It is popped from the operand stack of the current frame (Section 3.5, “Frames” on page 3-3) and pushed onto the operand stack of the frame of the invoker. Any other values on the operand stack of the current method are discarded.

The virtual machine then reinstates the frame of the invoker and returns control to the invoker.

Notes

If a virtual machine does not support the int data type, the ireturn instruction will not be available.

7.5.61 ishl

Shift left int

Format

ishl

Forms

ishl = 78 (0x4e)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. An int result is calculated by shifting value1 left by s bit positions, where s is the value of the low five bits of value2. The result is pushed onto the operand stack.

Notes

This is equivalent (even if overflow occurs) to multiplication by 2 to the power s. The shift distance actually used is always in the range 0 to 31, inclusive, as if value2 were subjected to a bitwise logical AND with the mask value 0x1f.

If a virtual machine does not support the int data type, the ishl instruction will not be available.

7.5.62 ishr

Arithmetic shift right int

Format

ishr

Forms

ishr = 80 (0x50)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. An int result is calculated by shifting value1 right by s bit positions, with sign extension, where s is the value of the low five bits of value2. The result is pushed onto the operand stack.

Notes

The resulting value is Î(value1) / 2s°, where s is value2 & 0x1f. For nonnegative value1, this is equivalent (even if overflow occurs) to truncating int division by 2 to the power s. The shift distance actually used is always in the range 0 to 31, inclusive, as if value2 were subjected to a bitwise logical AND with the mask value 0x1f.

Notes

If a virtual machine does not support the int data type, the ishr instruction will not be available.

7.5.63 istore

Store int into local variable

Format

istore
index

Forms

istore = 42 (0x2a)

Stack

…, value.word1, value.word2 ->
…

Description

The index is an unsigned byte. Both index and index + 1 must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The value on top of the operand stack must be of type int. It is popped from the operand stack, and the local variables at index and index + 1 are set to value.

Notes

If a virtual machine does not support the int data type, the istore instruction will not be available.

7.5.64 istore_<n>

Store int into local variable

Format

istore_<n>

Forms

istore_0 = 51 (0x33)
istore_1 = 52 (0x34)
istore_2 = 53 (0x35)
istore_3 = 54 (0x36)

Stack

…, value.word1, value.word2 ->
…

Description

Both <n> and <n> + 1 must be a valid indices into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The value on top of the operand stack must be of type int. It is popped from the operand stack, and the local variables at index and index + 1 are set to value.

Notes

If a virtual machine does not support the int data type, the istore_<n> instruction will not be available.

7.5.65 isub

Subtract int

Format

isub

Forms

isub = 68 (0x44)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. The int result is value1 - value2. The result is pushed onto the operand stack.

For int subtraction, a – b produces the same result as a + (–b). For int values, subtraction from zeros is the same as negation.

Despite the fact that overflow or underflow may occur, in which case the result may have a different sign than the true mathematical result, execution of an isub instruction never throws a runtime exception.

Notes

If a virtual machine does not support the int data type, the isub instruction will not be available.

7.5.66 itableswitch

Access jump table by int index and jump

Format

itableswitch
defaultbyte1
defaultbyte2
lowbyte1
lowbyte2
lowbyte3
lowbyte4
highbyte1
highbyte2
highbyte3
highbyte4
jump offsets…

Offset Format

offsetbyte1
offsetbyte2

Forms

itableswitch = 116 (0x74)

Stack

…, index ->
…

Description

An itableswitch instruction is a variable-length instruction. Immediately after the itableswitch opcode follow a signed 16-bit value default, a signed 32-bit value low, a signed 32-bit value high, and then high – low + 1 further signed 16-bit offsets. The value low must be less than or equal to high. The high – low + 1 signed 16-bit offsets are treated as a 0-based jump table. Each of the signed 16-bit values is constructed from two unsigned bytes as (byte1 << 8) | byte2. Each of the signed 32-bit values is constructed from four unsigned bytes as (byte1 << 24) | (byte2 << 16) | (byte3 << 8) | byte4.

The index must be of type int and is popped from the stack. If index is less than low or index is greater than high, then a target address is calculated by adding default to the address of the opcode of this itableswitch instruction. Otherwise, the offset at

position index – low of the jump table is extracted. The target address is calculated by adding that offset to the address of the opcode of this itableswitch instruction. Execution then continues at the target address.

The target addresses that can be calculated from each jump table offset, as well as the one calculated from default, must be the address of an opcode of an instruction within the method that contains this itableswitch instruction.

Notes

If a virtual machine does not support the int data type, the itableswitch instruction will not be available.

7.5.67 iushr

Logical shift right int

Format

iushr

Forms

iushr = 82 (0x52)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. An int result is calculated by shifting the result right by s bit positions, with zero extension, where s is the value of the low five bits of value2. The result is pushed onto the operand stack.

Notes

If value1 is positive and s is value2 & 0x1f, the result is the same as that of value1 >> s; if value1 is negative, the result is equal to the value of the expression (value1 >> s) + (2 << ~s). The addition of the (2 << ~s) term cancels out the propagated sign bit. The shift distance actually used is always in the range 0 to 31, inclusive, as if value2 were subjected to a bitwise logical AND with the mask value 0x1f.

If a virtual machine does not support the int data type, the iushr instruction will not be available.

7.5.68 ixor

Boolean XOR int

Format

ixor

Forms

ixor = 88 (0x58)

Stack

…, value1.word1, value1.word2, value2.word1, value2.word2 ->
…, result.word1, result.word2

Description

Both value1 and value2 must be of type int. The values are popped from the operand stack. An int result is calculated by taking the bitwise exclusive OR of value1 and value2. The result is pushed onto the operand stack.

Notes

If a virtual machine does not support the int data type, the ixor instruction will not be available.

7.5.69 jsr

Jump subroutine

Format

jsr
branchbyte1
branchbyte2

Forms

jsr = 113 (0x71)

Stack

… ->
…, address

Description

The address of the opcode of the instruction immediately following this jsr instruction is pushed onto the operand stack as a value of type returnAddress. The unsigned branchbyte1 and branchbyte2 are used to construct a signed 16-bit offset, where the offset is (branchbyte1 << 8) | branchbyte2. Execution proceeds at that offset from the address of this jsr instruction. The target address must be that of an opcode of an instruction within the method that contains this jsr instruction.

Notes

The jsr instruction is used with the ret instruction in the implementation of the finally clause of the Java language. Note that jsr pushes the address onto the stack and ret gets it out of a local variable. This asymmetry is intentional.

7.5.70 new

Create new object

Format

new
indexbyte1
indexbyte2

Forms

new = 143 (0x8f)

Stack

… ->
…, objectref

Description

The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The item at that index in the constant pool must be of type CONSTANT_Classref (Section 6.7.1, “CONSTANT_Classref” on page 6-16), a reference to a class or interface type. The reference is resolved and must result in a class type (it must not result in an interface type). Memory for a new instance of that class is allocated from the heap, and the instance variables of the new object are initialized to their default initial values. The objectref, a reference to the instance, is pushed onto the operand stack.

Notes

The new instruction does not completely create a new instance; instance creation is not completed until an instance initialization method has been invoked on the uninitialized instance.

7.5.71 newarray

Create new array

Format

newarray
atype

Forms

newarray = 144 (0x90)

Stack

…, count ->
…, arrayref

Description

The count must be of type short. It is popped off the operand stack. The count represents the number of elements in the array to be created.

The unsigned byte atype is a code that indicates the type of array to create. It must take one of the following values:

TABLE 7-4 Array Values

Array Type	*atype*
T_BOOLEAN	10
T_BYTE	11
T_SHORT	12
T_INT	13

A new array whose components are of type atype, of length count, is allocated from the heap. A reference arrayref to this new array object is pushed onto the operand stack. All of the elements of the new array are initialized to the default initial value for its type.

Runtime Exception

If count is less than zero, the newarray instruction throws a NegativeArraySizeException.

Notes

If a virtual machine does not support the int data type, the value of atype may not be 13 (array type = T_INT).

7.5.72 nop

Do nothing

Format

nop

Forms

nop = 0 (0x0)

Stack

No change

Description

Do nothing.

7.5.73 pop

Pop top operand stack word

Format

pop

Forms

pop = 59 (0x3b)

Stack

…, word ->
…

Description

The top word is popped from the operand stack. The pop instruction must not be used unless the word contains a 16-bit data type.

Notes

The pop instruction operates on an untyped word, ignoring the type of data it contains.

7.5.74 pop2

Pop top two operand stack words

Format

pop2

Forms

pop2 = 60 (0x3c)

Stack

…, word2, word1 ->
…

Description

The top two words are popped from the operand stack.

The pop2 instruction must not be used unless each of word1 and word2 is a word that contains a 16-bit data type or both together are the two words of a single 32-bit datum.

Notes

Except for restrictions preserving the integrity of 32-bit data types, the pop2 instruction operates on an untyped word, ignoring the type of data it contains.

7.5.75 putfield_<t>

Set field in object

Format

putfield_<t>
index

Forms

putfield_a = 135 (0x87)
putfield_b = 136 (0x88)
putfield_s = 137 (0x89)
putfield_i = 138 (0x8a)

Stack

…, objectref, value ->
…

OR

…, objectref, value.word1, value.word2 ->
…

Description

The unsigned index is used as an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3). The constant pool item at the index must be of type CONSTANT_InstanceFieldref (Section 6.7.2, “CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref” on page 6-18), a reference to a class and a field token.

The class of objectref must not be an array. If the field is protected, and it is a member of a superclass of the current class, and the field is not declared in the same package as the current class, then the class of objectref must be either the current class or a subclass of the current class. If the field is final, it must be declared in the current class.

The item must resolve to a field with a type that matches t, as follows:

- a field must be of type reference
- b field must be of type byte or type boolean
- s field must be of type short
- i field must be of type int

value must be of a type that is assignment compatible with the field descriptor (t) type.

The width of a field in a class instance is determined by the field type specified in the instruction. The item is resolved, determining the field offset[1]. The objectref, which must be of type reference, and the value are popped from the operand stack. If the field is of type byte or type boolean, the value is truncated to a byte. The field at the offset from the start of the object referenced by objectref is set to the value.

Runtime Exception

If objectref is null, the putfield_<t> instruction throws a NullPointerException.

Notes

In some circumstances, the putfield_<t> instruction may throw a SecurityException if the current context Section 3.4, "Contexts" on page 3-2) is not the owning context (Section 3.4, "Contexts" on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the putfield_i instruction will not be available.

7.5.76 putfield_<t>_this

Set field in current object

Format

putfield_<t>_this
index

Forms

putfield_a_this = 181 (0xb5)
putfield_b_this = 182 (0xb6)
putfield_s_this = 183 (0xb7)
putfield_i_this = 184 (0xb8)

Stack

…, value ->
…

OR

1. The offset may be computed by adding the field token value to the size of an instance of the immediate superclass. However, this method is not required by this specification. A Java Card virtual machine may define any mapping from token value to offset into an instance.

…, value.word1, value.word2 ->
…

Description

The currently executing method must be an instance method that was invoked using the invokevirtual, invokeinterface or invokespecial instruction. The local variable at index 0 must contain a reference objectref to the currently executing method's this parameter. The unsigned index is used as an index into the constant pool of the current package (Section 3.5, "Frames" on page 3-3). The constant pool item at the index must be of type CONSTANT_InstanceFieldref (Section 6.7.2, "CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref" on page 6-18), a reference to a class and a field token.

The class of objectref must not be an array. If the field is protected, and it is a member of a superclass of the current class, and the field is not declared in the same package as the current class, then the class of objectref must be either the current class or a subclass of the current class. If the field is final, it must be declared in the current class.

The item must resolve to a field with a type that matches t, as follows:

- a field must be of type reference
- b field must be of type byte or type boolean
- s field must be of type short
- i field must be of type int

value must be of a type that is assignment compatible with the field descriptor (t) type.

The width of a field in a class instance is determined by the field type specified in the instruction. The item is resolved, determining the field offset[1]. The value is popped from the operand stack. If the field is of type byte or type boolean, the value is truncated to a byte. The field at the offset from the start of the object referenced by objectref is set to the value.

Runtime Exception

If objectref is null, the putfield_<t>_this instruction throws a NullPointerException.

Notes

1. The offset may be computed by adding the field token value to the size of an instance of the immediate superclass. However, this method is not required by this specification. A Java Card virtual machine may define any mapping from token value to offset into an instance.

In some circumstances, the putfield_<t>_this instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the putfield_i_this instruction will not be available.

7.5.77 putfield_<t>_w

Set field in object (wide index)

Format

putfield<t>_w
indexbyte1
indexbyte2

Forms

putfield_a_w = 177 (0xb1)
putfield_b_w = 178 (0xb2)
putfield_s_w = 179 (0xb3)
putfield_i_w = 180 (0xb4)

Stack

…, objectref, value ->
…

OR

…, objectref, value.word1, value.word2 ->
…

Description

The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, “Frames” on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The constant pool item at the index must be of type CONSTANT_InstanceFieldref (Section 6.7.2, “CONSTANT_InstanceFieldref, CONSTANT_VirtualMethodref, and CONSTANT_SuperMethodref” on page 6-18), a reference to a class and a field token.

The class of objectref must not be an array. If the field is protected, and it is a member of a superclass of the current class, and the field is not declared in the same package as the current class, then the class of objectref must be either the current class or a subclass of the current class. If the field is final, it must be declared in the current class.

The item must resolve to a field with a type that matches t, as follows:

- a field must be of type reference
- b field must be of type byte or type boolean
- s field must be of type short
- i field must be of type int

value must be of a type that is assignment compatible with the field descriptor (t) type.

The width of a field in a class instance is determined by the field type specified in the instruction. The item is resolved, determining the field offset[1]. The objectref, which must be of type reference, and the value are popped from the operand stack. If the field is of type byte or type boolean, the value is truncated to a byte. The field at the offset from the start of the object referenced by objectref is set to the value.

Runtime Exception

If objectref is null, the putfield_<t>_w instruction throws a NullPointerException.

Notes

In some circumstances, the putfield_<t>_w instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the object referenced by objectref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the putfield_i_w instruction will not be available.

7.5.78 putstatic_<t>

Set static field in class

1. The offset may be computed by adding the field token value to the size of an instance of the immediate superclass. However, this method is not required by this specification. A Java Card virtual machine may define any mapping from token value to offset into an instance.

Format

putstatic_<t>
indexbyte1
indexbyte2

Forms

putstatic_a = 127 (0x7f)
putstatic_b = 128 (0x80)
putstatic_s = 129 (0x81)
putstatic_i = 130 (0x82)

Stack

…, value ->
…

OR

…, value.word1, value.word2 ->
…

Description

The unsigned indexbyte1 and indexbyte2 are used to construct an index into the constant pool of the current package (Section 3.5, "Frames" on page 3-3), where the value of the index is (indexbyte1 << 8) | indexbyte2. The constant pool item at the index must be of type CONSTANT_StaticFieldref (Section 6.7.3, "CONSTANT_StaticFieldref and CONSTANT_StaticMethodref" on page 6-19), a reference to a static field. If the field is final, it must be declared in the current class.

The item must resolve to a field with a type that matches t, as follows:

- a field must be of type reference
- b field must be of type byte or type boolean
- s field must be of type short
- i field must be of type int

value must be of a type that is assignment compatible with the field descriptor (t) type.

The width of a class field is determined by the field type specified in the instruction. The item is resolved, determining the class field. The value is popped from the operand stack. If the field is of type byte or type boolean, the value is truncated to a byte. The field is set to the value.

Notes

In some circumstances, the putstatic_a instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the object being stored in the field. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

If a virtual machine does not support the int data type, the putstatic_i instruction will not be available.

7.5.79 ret

Return from subroutine

Format

ret
index

Forms

ret = 114 (0x72)

Stack

No change

Description

The index is an unsigned byte that must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variable at index must contain a value of type returnAddress. The contents of the local variable are written into the Java Card virtual machine’s pc register, and execution continues there.

Notes

The ret instruction is used with the jsr instruction in the implementation of the finally keyword of the Java language. Note that jsr pushes the address onto the stack and ret gets it out of a local variable. This asymmetry is intentional.

The ret instruction should not be confused with the return instruction. A return instruction returns control from a Java method to its invoker, without passing any value back to the invoker.

7.5.80 return

Return void from method

Format

return

Forms

return = 122 (0x7a)

Stack

… ->
[empty]

Description

Any values on the operand stack of the current method are discarded. The virtual machine then reinstates the frame of the invoker and returns control to the invoker.

7.5.81 s2b

Convert short to byte

Format

s2b

Forms

s2b = 91 (0x5b)

Stack

…, value ->
…, result

Description

The value on top of the operand stack must be of type short. It is popped from the top of the operand stack, truncated to a byte result, then sign-extended to a short result. The result is pushed onto the operand stack.

Notes

The s2b instruction performs a narrowing primitive conversion. It may lose information about the overall magnitude of value. The result may also not have the same sign as value.

7.5.82 s2i

Convert short to int

Format

s2i

Forms

s2i = 92 (0x5c)

Stack

…, value ->
…, result.word1, result.word2

Description

The value on top of the operand stack must be of type short. It is popped from the operand stack and sign-extended to an int result. The result is pushed onto the operand stack.

Notes

The s2i instruction performs a widening primitive conversion. Because all values of type short are exactly representable by type int, the conversion is exact.

If a virtual machine does not support the int data type, the s2i instruction will not be available.

7.5.83 sadd

Add short

Format

sadd

Forms

sadd = 65 (0x41)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. The short result is value1 + value2. The result is pushed onto the operand stack.

If a sadd instruction overflows, then the result is the low-order bits of the true mathematical result in a sufficiently wide two's-complement format. If overflow occurs, then the sign of the result may not be the same as the sign of the mathematical sum of the two values.

7.5.84 saload

Load short from array

Format

saload

Forms

saload = 38 (0x46)

Stack

…, arrayref, index ->
…, value

Description

The arrayref must be of type reference and must refer to an array whose components are of type short. The index must be of type short. Both arrayref and index are popped from the operand stack. The short value in the component of the array at index is retrieved and pushed onto the top of the operand stack.

Runtime Exceptions

If arrayref is null, saload throws a NullPointerException.

Otherwise, if index is not within the bounds of the array referenced by arrayref, the saload instruction throws an ArrayIndexOutOfBoundsException.

Notes

In some circumstances, the saload instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

7.5.85 sand

Boolean AND short

Format

sand

Forms

sand = 83 (0x53)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 are popped from the operand stack. A short result is calculated by taking the bitwise AND (conjunction) of value1 and value2. The result is pushed onto the operand stack.

7.5.86 sastore

Store into short array

Format

sastore

Forms

sastore = 57 (0x39)

Stack

…, arrayref, index, value ->
…

Description

The arrayref must be of type reference and must refer to an array whose components are of type short. The index and value must both be of type short. The arrayref, index and value are popped from the operand stack. The short value is stored as the component of the array indexed by index.

Runtime Exception

If arrayref is null, sastore throws a NullPointerException.

Otherwise, if index is not within the bounds of the array referenced by arrayref, the sastore instruction throws an ArrayIndexOutOfBoundsException.

Notes

In some circumstances, the sastore instruction may throw a SecurityException if the current context (Section 3.4, “Contexts” on page 3-2) is not the owning context (Section 3.4, “Contexts” on page 3-2) of the array referenced by arrayref. The exact circumstances when the exception will be thrown are specified in Chapter 6 of the *Runtime Environment Specification, Java Card Platform, Version 2.2.2*.

7.5.87 sconst_<s>

Push short constant

Format

sconst_<s>

Forms

sconst_m1 = 2 (0x2)
sconst_0 = 3 (0x3)
sconst_1 = 4 (0x4)
sconst_2 = 5 (0x5)
sconst_3 = 6 (0x6)
sconst_4= 7 (0x7)
sconst_5 = 8 (0x8)

Stack

… ->
…, <s>

Description

Push the short constant <s> (-1, 0, 1, 2, 3, 4, or 5) onto the operand stack.

7.5.88 sdiv

Divide short

Format

sdiv

Forms

sdiv = 71 (0x47)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. The short result is the value of the Java expression value1 / value2. The result is pushed onto the operand stack.

A short division rounds towards 0; that is, the quotient produced for short values in n/d is a short value q whose magnitude is as large as possible while satisfying | d · q | <= | n |. Moreover, q is a positive when | n | >= | d | and n and d have the same sign, but q is negative when | n | >= | d | and n and d have opposite signs.

There is one special case that does not satisfy this rule: if the dividend is the negative integer of the largest possible magnitude for the short type, and the divisor is –1, then overflow occurs, and the result is equal to the dividend. Despite the overflow, no exception is thrown in this case.

Runtime Exception

If the value of the divisor in a short division is 0, sdiv throws an ArithmeticException.

7.5.89 sinc

Increment local short variable by constant

Format

sinc
index
const

Forms

sinc = 89 (0x59)

Stack

No change

Description

The index is an unsigned byte that must be a valid index into the local variable of the current frame (Section 3.5, “Frames” on page 3-3). The const is an immediate signed byte. The local variable at index must contain a short. The value const is first sign-extended to a short, then the local variable at index is incremented by that amount.

7.5.90 sinc_w

Increment local short variable by constant

Format

sinc_w
index
byte1
byte2

Forms

sinc_w = 150 (0x96)

Stack

No change

Description

The index is an unsigned byte that must be a valid index into the local variable of the current frame (Section 3.5, “Frames” on page 3-3). The immediate unsigned byte1 and byte2 values are assembled into a short const where the value of const is (byte1 << 8) | byte2. The local variable at index, which must contain a short, is incremented by const.

7.5.91 sipush

Push short

Format

sipush
byte1
byte2

Forms

sipush = 19 (0x13)

Stack

… ->
…, value1.word1, value1.word2

Description

The immediate unsigned byte1 and byte2 values are assembled into a signed short where the value of the short is (byte1 << 8) | byte2. The intermediate value is then sign-extended to an int, and the resulting value is pushed onto the operand stack.

Notes

If a virtual machine does not support the int data type, the sipush instruction will not be available.

7.5.92 sload

Load short from local variable

Format

sload
index

Forms

sload = 22 (0x16)

Stack

… ->
…, value

Description

The index is an unsigned byte that must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variable at index must contain a short. The value in the local variable at index is pushed onto the operand stack.

7.5.93 sload_<n>

Load short from local variable

Format

sload_<n>

Forms

sload_0 = 28 (0x1c)
sload_1 = 29 (0x1d)
sload_2 = 30 (0x1e)
sload_3 = 31 (0x1f)

Stack

… ->
…, value

Description

The <n> must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The local variable at <n> must contain a short. The value in the local variable at <n> is pushed onto the operand stack.

Notes

Each of the sload_<n> instructions is the same as sload with an index of <n>, except that the operand <n> is implicit.

7.5.94 slookupswitch

Access jump table by key match and jump

Format

slookupswitch
defaultbyte1
defaultbyte2
npairs1
npairs2
match-offset pairs…

Pair Format

matchbyte1
matchbyte2
offsetbyte1
offsetbyte2

Forms

slookupswitch = 117 (0x75)

Stack

…, key ->
…

Description

A slookupswitch instruction is a variable-length instruction. Immediately after the slookupswitch opcode follow a signed 16-bit value default, an unsigned 16-bit value npairs, and then npairs pairs. Each pair consists of a short match and a signed 16-bit offset. Each of the signed 16-bit values is constructed from two unsigned bytes as (byte1 << 8) | byte2.

The table match-offset pairs of the slookupswitch instruction must be sorted in increasing numerical order by match.

The key must be of type short and is popped from the operand stack and compared against the match values. If it is equal to one of them, then a target address is calculated by adding the corresponding offset to the address of the opcode of this

slookupswitch instruction. If the key does not match any of the match values, the target address is calculated by adding default to the address of the opcode of this slookupswitch instruction. Execution then continues at the target address.

The target address that can be calculated from the offset of each match-offset pair, as well as the one calculated from default, must be the address of an opcode of an instruction within the method that contains this slookupswitch instruction.

Notes

The match-offset pairs are sorted to support lookup routines that are quicker than linear search.

7.5.95 smul

Multiply short

Format

smul

Forms

smul = 69 (0x45)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. The short result is value1 * value2. The result is pushed onto the operand stack.

If a smul instruction overflows, then the result is the low-order bits of the mathematical product as a short. If overflow occurs, then the sign of the result may not be the same as the sign of the mathematical product of the two values.

7.5.96 sneg

Negate short

Format

sneg

Forms

sneg = 72 (0x4b)

Stack

…, value ->
…, result

Description

The value must be of type short. It is popped from the operand stack. The short result is the arithmetic negation of value, -value. The result is pushed onto the operand stack.

For short values, negation is the same as subtraction from zero. Because the Java Card virtual machine uses two's-complement representation for integers and the range of two's-complement values is not symmetric, the negation of the maximum negative short results in that same maximum negative number. Despite the fact that overflow has occurred, no exception is thrown.

For all short values x, -x equals (~x) + 1.

7.5.97 sor

Boolean OR short

Format

sor

Forms

sor = 85 (0x55)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. A short result is calculated by taking the bitwise inclusive OR of value1 and value2. The result is pushed onto the operand stack.

7.5.98 srem

Remainder short

Format

srem

Forms

srem = 73 (0x49)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. The short result is the value of the Java expression value1 – (value1 / value2) * value2. The result is pushed onto the operand stack.

The result of the irem instruction is such that (a/b)*b + (a%b) is equal to a. This identity holds even in the special case that the dividend is the negative short of largest possible magnitude for its type and the divisor is –1 (the remainder is 0). It follows from this rule that the result of the remainder operation can be negative only if the dividend is negative and can be positive only if the dividend is positive. Moreover, the magnitude of the result is always less than the magnitude of the divisor.

Runtime Exception

If the value of the divisor for a short remainder operator is 0, srem throws an ArithmeticException.

7.5.99 sreturn

Return short from method

Format

sreturn

Forms

sreturn = 120 (0x78)

Stack

…, value ->
[empty]

Description

The value must be of type short. It is popped from the operand stack of the current frame (Section 3.5, “Frames” on page 3-3) and pushed onto the operand stack of the frame of the invoker. Any other values on the operand stack of the current method are discarded.

The virtual machine then reinstates the frame of the invoker and returns control to the invoker.

7.5.100 sshl

Shift left short

Format

sshl

Forms

sshl = 77 (0x4d)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. A short result is calculated by shifting value1 left by s bit positions, where s is the value of the low five bits of value2. The result is pushed onto the operand stack.

Notes

This is equivalent (even if overflow occurs) to multiplication by 2 to the power s. The shift distance actually used is always in the range 0 to 31, inclusive, as if value2 were subjected to a bitwise logical AND with the mask value 0x1f.

The mask value of 0x1f allows shifting beyond the range of a 16-bit short value. It is used by this instruction, however, to ensure results equal to those generated by the Java instruction ishl.

7.5.101 sshr

Arithmetic shift right short

Format

sshr

Forms

sshr = 79 (0x4f)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. A short result is calculated by shifting value1 right by s bit positions, with sign extension, where s is the value of the low five bits of value2. The result is pushed onto the operand stack.

Notes

The resulting value is Î(value1) / 2s°, where s is value2 & 0x1f. For nonnegative value1, this is equivalent (even if overflow occurs) to truncating short division by 2 to the power s. The shift distance actually used is always in the range 0 to 31, inclusive, as if value2 were subjected to a bitwise logical AND with the mask value 0x1f.

The mask value of 0x1f allows shifting beyond the range of a 16-bit short value. It is used by this instruction, however, to ensure results equal to those generated by the Java instruction ishr.

7.5.102 sspush

Push short

Format

sspush
byte1
byte2

Forms

sspush = 17 (0x11)

Stack

… ->
…, value

Description

The immediate unsigned byte1 and byte2 values are assembled into a signed short where the value of the short is (byte1 << 8) | byte2. The resulting value is pushed onto the operand stack.

7.5.103 sstore

Store short into local variable

Format

sstore
index

Forms

sstore = 41 (0x29)

Stack

…, value ->
…

Description

The index is an unsigned byte that must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The value on top of the operand stack must be of type short. It is popped from the operand stack, and the value of the local variable at index is set to value.

7.5.104 sstore_<n>

Store short into local variable

Format

sstore_<n>

Forms

sstore_0 = 47 (0x2f)
sstore_1 = 48 (0x30)
sstore_2 = 49 (0x31)
sstore_3 = 50 (0x32)

Stack

…, value ->
…

Description

The <n> must be a valid index into the local variables of the current frame (Section 3.5, “Frames” on page 3-3). The value on top of the operand stack must be of type short. It is popped from the operand stack, and the value of the local variable at <n> is set to value.

7.5.105 ssub

Subtract short

Format

ssub

Forms

ssub = 67 (0x43)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. The short result is value1 - value2. The result is pushed onto the operand stack.

For short subtraction, a – b produces the same result as a + (–b). For short values, subtraction from zeros is the same as negation.

Despite the fact that overflow or underflow may occur, in which case the result may have a different sign than the true mathematical result, execution of a ssub instruction never throws a runtime exception.

7.5.106 stableswitch

Access jump table by short index and jump

Format

stableswitch
defaultbyte1
defaultbyte2
lowbyte1
lowbyte2
highbyte1
highbyte2
jump offsets…

Offset Format

offsetbyte1
offsetbyte2

Forms

stableswitch = 115 (0x73)

Stack

…, index ->
…

Description

A stableswitch instruction is a variable-length instruction. Immediately after the stableswitch opcode follow a signed 16-bit value default, a signed 16-bit value low, a signed 16-bit value high, and then high – low + 1 further signed 16-bit offsets. The value low must be less than or equal to high. The high – low + 1 signed 16-bit offsets are treated as a 0-based jump table. Each of the signed 16-bit values is constructed from two unsigned bytes as (byte1 << 8) | byte2.

The index must be of type short and is popped from the stack. If index is less than low or index is greater than high, than a target address is calculated by adding default to the address of the opcode of this stableswitch instruction. Otherwise, the offset at position index – low of the jump table is extracted. The target address is calculated by adding that offset to the address of the opcode of this stableswitch instruction. Execution then continues at the target address.

The target addresses that can be calculated from each jump table offset, as well as the one calculated from default, must be the address of an opcode of an instruction within the method that contains this stableswitch instruction.

7.5.107 sushr

Logical shift right short

Format

sushr

Forms

sushr = 81 (0x51)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. A short result is calculated by sign-extending value1 to 32 bits[1] and shifting the result right by s bit positions, with zero extension, where s is the value of the low five bits of value2. The resulting value is then truncated to a 16-bit result. The result is pushed onto the operand stack.

Notes

If value1 is positive and s is value2 & 0x1f, the result is the same as that of value1 >> s; if value1 is negative, the result is equal to the value of the expression (value1 >> s) + (2 << ~s). The addition of the (2 << ~s) term cancels out the propagated sign bit. The shift distance actually used is always in the range 0 to 31, inclusive, as if value2 were subjected to a bitwise logical AND with the mask value 0x1f.

The mask value of 0x1f allows shifting beyond the range of a 16-bit short value. It is used by this instruction, however, to ensure results equal to those generated by the Java instruction iushr.

7.5.108 swap_x

Swap top two operand stack words

Format

swap_x
mn

Forms

swap_x = 64 (0x40)

Stack

…, wordM+N, …, wordM+1, wordM, …, word1 ->
…, wordM, …, word1, wordM+N, …, wordM+1

Description

The unsigned byte mn is used to construct two parameter values. The high nibble, (mn & 0xf0) >> 4, is used as the value m. The low nibble, (mn & 0xf), is used as the value n. Permissible values for both m and n are 1 and 2.

The top m words on the operand stack are swapped with the n words immediately below.

The swap_x instruction must not be used unless the ranges of words 1 through m and words m+1 through n each contain either a 16-bit data type, two 16-bit data types, a 32-bit data type, a 16-bit data type and a 32-bit data type (in either order), or two 32-bit data types.

1. Sign extension to 32 bits ensures that the result computed by this instruction will be exactly equal to that computed by the Java iushr instruction, regardless of the input values. In a Java Card virtual machine the expression "0xffff >>> 0x01" yields 0xffff, where ">>>" is performed by the `sushr` instruction. The same result is rendered by a Java virtual machine.

Notes

Except for restrictions preserving the integrity of 32-bit data types, the swap_x instruction operates on untyped words, ignoring the types of data they contain.

If a virtual machine does not support the int data type, the only permissible value for both m and n is 1.

7.5.109 sxor

Boolean XOR short

Format

sxor

Forms

sxor = 87 (0x57)

Stack

…, value1, value2 ->
…, result

Description

Both value1 and value2 must be of type short. The values are popped from the operand stack. A short result is calculated by taking the bitwise exclusive OR of value1 and value2. The result is pushed onto the operand stack.

CHAPTER 8

Tables of Instructions

The following pages contain lists of the APDU instructions recognized by the Java Card platform, organized by opcode value (TABLE 8-1) and by opcode mnemonic (TABLE 8-2).

TABLE 8-1 Instructions by Opcode Value

dec	hex	mnemonic	dec	hex	mnemonic
0	00	nop	47	2F	sstore_0
1	01	aconst_null	48	30	sstore_1
2	02	sconst_m1	49	31	sstore_2
3	03	sconst_0	50	32	sstore_3
4	04	sconst_1	51	33	istore_0
5	05	sconst_2	52	34	istore_1
6	06	sconst_3	53	35	istore_2
7	07	sconst_4	54	36	istore_3
8	08	sconst_5	55	37	aastore
9	09	iconst_m1	56	38	bastore
10	0A	iconst_0	57	39	sastore
11	0B	iconst_1	58	3A	iastore
12	0C	iconst_2	59	3B	pop
13	0D	iconst_3	60	3C	pop2
14	0E	iconst_4	61	3D	dup
15	0F	iconst_5	62	3E	dup2
16	10	bspush	63	3F	dup_x
17	11	sspush	64	40	swap_x

TABLE 8-1 Instructions by Opcode Value *(Continued)*

dec	hex	mnemonic	dec	hex	mnemonic
18	12	bipush	65	41	sadd
19	13	sipush	66	42	iadd
20	14	iipush	67	43	ssub
21	15	aload	68	44	isub
22	16	sload	69	45	smul
23	17	iload	70	46	imul
24	18	aload_0	71	47	sdiv
25	19	aload_1	72	48	idiv
26	1A	aload_2	73	49	srem
27	1B	aload_3	74	4A	irem
28	1C	sload_0	75	4B	sneg
29	1D	sload_1	76	4C	ineg
30	1E	sload_2	77	4D	sshl
31	1F	sload_3	78	4E	ishl
32	20	iload_0	79	4F	sshr
33	21	iload_1	80	50	ishr
34	22	iload_2	81	51	sushr
35	23	iload_3	82	52	iushr
36	24	aaload	83	53	sand
37	25	baload	84	54	iand
38	26	saload	85	55	sor
39	27	iaload	86	56	ior
40	28	astore	87	57	sxor
41	29	sstore	88	58	ixor
42	2A	istore	89	59	sinc
43	2B	astore_0	90	5A	iinc
44	2C	astore_1	91	5B	s2b
45	2D	astore_2	92	5C	s2i
46	2E	astore_3	93	5D	i2b
94	5E	i2s	141	8D	invokestatic

TABLE 8-1 Instructions by Opcode Value *(Continued)*

dec	hex	mnemonic	dec	hex	mnemonic
95	5F	icmp	142	8E	invokeinterface
96	60	ifeq	143	8F	new
97	61	ifne	144	90	newarray
98	62	iflt	145	91	anewarray
99	63	ifge	146	92	arraylength
100	64	ifgt	147	93	athrow
101	65	ifle	148	94	checkcast
102	66	ifnull	149	95	instanceof
103	67	ifnonnull	150	96	sinc_w
104	68	if_acmpeq	151	97	iinc_w
105	69	if_acmpne	152	98	ifeq_w
106	6A	if_scmpeq	153	99	ifne_w
107	6B	if_scmpne	154	9A	iflt_w
108	6C	if_scmplt	155	9B	ifge_w
109	6D	if_scmpge	156	9C	ifgt_w
110	6E	if_scmpgt	157	9D	ifle_w
111	6F	if_scmple	158	9E	ifnull_w
112	70	goto	159	9F	ifnonnull_w
113	71	jsr	160	A0	if_acmpeq_w
114	72	ret	161	A1	if_acmpne_w
115	73	stableswitch	162	A2	if_scmpeq_w
116	74	itableswitch	163	A3	if_scmpne_w
117	75	slookupswitch	164	A4	if_scmplt_w
118	76	ilookupswitch	165	A5	if_scmpge_w
119	77	areturn	166	A6	if_scmpgt_w
120	78	sreturn	167	A7	if_scmple_w
121	79	ireturn	168	A8	goto_w
122	7A	return	169	A9	getfield_a_w
123	7B	getstatic_a	170	AA	getfield_b_w
124	7C	getstatic_b	171	AB	getfield_s_w

TABLE 8-1 Instructions by Opcode Value *(Continued)*

dec	hex	mnemonic	dec	hex	mnemonic
125	7D	getstatic_s	172	AC	getfield_i_w
126	7E	getstatic_i	173	AD	getfield_a_this
127	7F	putstatic_a	174	AE	getfield_b_this
128	80	putstatic_b	175	AF	getfield_s_this
129	81	putstatic_s	176	B0	getfield_i_this
130	82	putstatic_i	177	B1	putfield_a_w
131	83	getfield_a	178	B2	putfield_b_w
132	84	getfield_b	179	B3	putfield_s_w
133	85	getfield_s	180	B4	putfield_i_w
134	86	getfield_i	181	B5	putfield_a_this
135	87	putfield_a	182	B6	putfield_b_this
136	88	putfield_b	183	B7	putfield_s_this
137	89	putfield_s	184	B8	putfield_i_this
138	8A	putfield_i			…
139	8B	invokevirtual	254	FE	impdep1
140	8C	invokespecial	255	FF	impdep2

TABLE 8-2 Instructions by Opcode Mnemonic

mnemonic	dec	hex	mnemonic	dec	hex
aaload	36	24	iand	84	54
aastore	55	37	iastore	58	3A
aconst_null	1	01	icmp	95	5F
aload	21	15	iconst_0	10	0A
aload_0	24	18	iconst_1	11	0B
aload_1	25	19	iconst_2	12	0C
aload_2	26	1A	iconst_3	13	0D
aload_3	27	1B	iconst_4	14	0E
anewarray	145	91	iconst_5	15	0F
areturn	119	77	iconst_m1	9	09

TABLE 8-2 Instructions by Opcode Mnemonic *(Continued)*

mnemonic	dec	hex	mnemonic	dec	hex
arraylength	146	92	idiv	72	48
astore	40	28	if_acmpeq	104	68
astore_0	43	2B	if_acmpeq_w	160	A0
astore_1	44	2C	if_acmpne	105	69
astore_2	45	2D	if_acmpne_w	161	A1
astore_3	46	2E	if_scmpeq	106	6A
athrow	147	93	if_scmpeq_w	162	A2
baload	37	25	if_scmpge	109	6D
bastore	56	38	if_scmpge_w	165	A5
bipush	18	12	if_scmpgt	110	6E
bspush	16	10	if_scmpgt_w	166	A6
checkcast	148	94	if_scmple	111	6F
dup	61	3D	if_scmple_w	167	A7
dup_x	63	3F	if_scmplt	108	6C
dup2	62	3E	if_scmplt_w	164	A4
getfield_a	131	83	if_scmpne	107	6B
getfield_a_this	173	AD	if_scmpne_w	163	A3
getfield_a_w	169	A9	ifeq	96	60
getfield_b	132	84	ifeq_w	152	98
getfield_b_this	174	AE	ifge	99	63
getfield_b_w	170	AA	ifge_w	155	9B
getfield_i	134	86	ifgt	100	64
getfield_i_this	176	B0	ifgt_w	156	9C
getfield_i_w	172	AC	ifle	101	65
getfield_s	133	85	ifle_w	157	9D
getfield_s_this	175	AF	iflt	98	62
getfield_s_w	171	AB	iflt_w	154	9A
getstatic_a	123	7B	ifne	97	61
getstatic_b	124	7C	ifne_w	153	99
getstatic_i	126	7E	ifnonnull	103	67

TABLE 8-2 Instructions by Opcode Mnemonic *(Continued)*

mnemonic	dec	hex	mnemonic	dec	hex
getstatic_s	125	7D	ifnonnull_w	159	9F
goto	112	70	ifnull	102	66
goto_w	168	A8	ifnull_w	158	9E
i2b	93	5D	iinc	90	5A
i2s	94	5E	iinc_w	151	97
iadd	66	42	iipush	20	14
iaload	39	27	iload	23	17
iload_0	32	20	putstatic_s	129	81
iload_1	33	21	ret	114	72
iload_2	34	22	return	122	7A
iload_3	35	23	s2b	91	5B
ilookupswitch	118	76	s2i	92	5C
imul	70	46	sadd	65	41
ineg	76	4C	saload	38	26
instanceof	149	95	sand	83	53
invokeinterface	142	8E	sastore	57	39
invokespecial	140	8C	sconst_0	3	03
invokestatic	141	8D	sconst_1	4	04
invokevirtual	139	8B	sconst_2	5	05
ior	86	56	sconst_3	6	06
irem	74	4A	sconst_4	7	07
ireturn	121	79	sconst_5	8	08
ishl	78	4E	sconst_m1	2	02
ishr	80	50	sdiv	71	47
istore	42	2A	sinc	89	59
istore_0	51	33	sinc_w	150	96
istore_1	52	34	sipush	19	13
istore_2	53	35	sload	22	16
istore_3	54	36	sload_0	28	1C
isub	68	44	sload_1	29	1D

TABLE 8-2 Instructions by Opcode Mnemonic *(Continued)*

mnemonic	dec	hex	mnemonic	dec	hex
itableswitch	116	74	sload_2	30	1E
iushr	82	52	sload_3	31	1F
ixor	88	58	slookupswitch	117	75
jsr	113	71	smul	69	45
new	143	8F	sneg	75	4B
newarray	144	90	sor	85	55
nop	0	00	srem	73	49
pop	59	3B	sreturn	120	78
pop2	60	3C	sshl	77	4D
putfield_a	135	87	sshr	79	4F
putfield_a_this	181	B5	sspush	17	11
putfield_a_w	177	B1	sstore	41	29
putfield_b	136	88	sstore_0	47	2F
putfield_b_this	182	B6	sstore_1	48	30
putfield_b_w	178	B2	sstore_2	49	31
putfield_i	138	8A	sstore_3	50	32
putfield_i_this	184	B8	ssub	67	43
putfield_i_w	180	B4	stableswitch	115	73
putfield_s	137	89	sushr	81	51
putfield_s_this	183	B7	swap_x	64	40
putfield_s_w	179	B3	sxor	87	57
putstatic_a	127	7F			
putstatic_b	128	80			
putstatic_i	130	82			

Glossary

active applet instance an applet instance that is selected on at least one of the logical channels.

AID (application identifier) defined by ISO 7816, a string used to uniquely identify card applications and certain types of files in card file systems. An AID consists of two distinct pieces: a 5-byte RID (resource identifier) and a 0 to 11-byte PIX (proprietary identifier extension). The RID is a resource identifier assigned to companies by ISO. The PIX identifiers are assigned by companies.

A unique AID is assigned for each package. In addition, a unique AID is assigned for each applet in the package. The package AID and the default AID for each applet defined in the package are specified in the `CAP` file. They are supplied to the converter when the `CAP` file is generated.

APDU an acronym for Application Protocol Data Unit as defined in ISO 7816-4.

API an acronym for Application Programming Interface. The API defines calling conventions by which an application program accesses the operating system and other services.

applet within the context of this document, a Java Card applet, which is the basic unit of selection, context, functionality, and security in Java Card technology.

applet developer a person creating an applet using Java Card technology.

applet execution context context of a package that contains currently active applet.

applet firewall the mechanism that prevents unauthorized accesses to objects in contexts other than currently active context.

applet package see *library package*.

assigned logical channel the logical channel on which the applet instance is either the active applet instance or will become the active applet instance.

atomic operation an operation that either completes in its entirety or no part of the operation completes at all.

atomicity state in which a particular operation is atomic. Atomicity of data updates guarantee that data are not corrupted in case of power loss or card removal.

ATR an acronym for Answer to Reset. An ATR is a string of bytes sent by the Java Card platform after a reset condition.

basic logical channel logical channel 0, the only channel that is active at card reset. This channel is permanent and can never be closed.

big-endian a technique of storing multibyte data where the high-order bytes come first. For example, given an 8-bit data item stored in big-endian order, the first bit read is considered the high bit.

binary compatibility in a Java Card system, a change in a Java programming language package results in a new `CAP` file. A new `CAP` file is binary compatible with (equivalently, does not break compatibility with) a preexisting `CAP` file if another `CAP` file converted using the export file of the preexisting `CAP` file can link with the new `CAP` file without errors.

bytecode machine-independent code generated by the compiler and executed by the Java virtual machine.

CAD an acronym for Card Acceptance Device. The CAD is the device in which the card is inserted.

CAP file the `CAP` file is produced by the Converter and is the standard file format for the binary compatibility of the Java Card platform. A `CAP` file contains an executable binary representation of the classes of a Java programming language package. The `CAP` file also contains the `CAP` file components (see also *CAP file component*). The `CAP` files produced by the converter are contained in Java™ Archive (JAR) files.

CAP file component a Java Card platform `CAP` file consists of a set of components which represent a Java programming language package. Each component describes a set of elements in the Java programming language package, or an aspect of the `CAP` file. A complete `CAP` file must contain all of the required components: Header, Directory, Import, Constant Pool, Method, Static Field, and Reference Location

The following components are optional: the Applet, Export, and Debug. The Applet component is included only if one or more Applets are defined in the package. The Export component is included only if classes in other packages may import elements in the package defined. The Debug component is optional. It contains all of the data necessary for debugging a package.

card session a card session begins with the insertion of the card into the CAD. The card is then able to exchange streams of APDUs with the CAD. The card session ends when the card is removed from the CAD.

cast the explicit conversion from one data type to another.

constant pool the constant pool contains variable-length structures representing various string constants, class names, field names, and other constants referred to within the CAP file and the Export File structure. Each of the constant pool entries, including entry zero, is a variable-length structure whose format is indicated by its first tag byte. There are no ordering constraints on entries in the constant pool entries. One constant pool is associated with each package.

There are differences between the Java platform constant pool and the Java Card technology-based constant pool. For example, in the Java platform constant pool there is one constant type for method references, while in the Java Card constant pool, there are three constant types for method references. The additional information provided by a constant type in Java Card technologies simplifies resolution of references.

context protected object space associated with each applet package and Java Card RE. All objects owned by an applet belong to context of the applet's package.

context switch a change from one currently active context to another. For example, a context switch is caused by an attempt to access an object that belongs to an applet instance that resides in a different package. The result of a context switch is a new currently active context.

Converter a piece of software that preprocesses all of the Java programming language class files that make up a package, and converts the package to a CAP file. The Converter also produces an export file.

currently active context when an object instance method is invoked, an owning context of this object becomes the currently active context.

currently selected applet the Java Card RE keeps track of the currently selected Java Card applet. Upon receiving a SELECT FILE command with this applet's AID, the Java Card RE makes this applet the currently selected applet. The Java Card RE sends all APDU commands to the currently selected applet.

custom CAP file component a new component added to the CAP file. The new component must conform to the general component format. It is silently ignored by a Java Card virtual machine that does not recognize the component. The identifiers associated with the new component are recorded in the custom_component item of the CAP file's Directory component.

default applet an applet that is selected by default on a logical channel when it is opened. If an applet is designated the default applet on a particular logical channel on the Java Card platform, it becomes the active applet by default when that logical channel is opened using the basic channel.

EEPROM an acronym for Electrically Erasable, Programmable Read Only Memory.

entry point objects see Java Card RE entry point objects.

Export file a file produced by the Converter that represents the fields and methods of a package that can be imported by classes in other packages.

externally visible in the Java Card platform, any classes, interfaces, their constructors, methods, and fields that can be accessed from another package according to the Java programming language semantics, as defined by the *Java Language Specification*, and Java Card API package access control restrictions (see *Java Language Specification*, section 2.2.1.1).

Externally visible items may be represented in an export file. For a library package, all externally visible items are represented in an export file. For an applet package, only those externally visible items that are part of a shareable interface are represented in an export file.

finalization the process by which a Java virtual machine (VM) allows an unreferenced object instance to release non-memory resources (for example, close and open files) prior to reclaiming the object's memory. Finalization is only performed on an object when that object is ready to be garbage collected (meaning, there are no references to the object).

Finalization is not supported by the Java Card virtual machine. The method `finalize()` is not called automatically by the Java Card virtual machine.

firewall see *applet firewall*.

flash memory a type of persistent mutable memory. It is more efficient in space and power than EPROM. Flash memory can be read bit by bit but can be updated only as a block. Thus, flash memory is typically used for storing additional programs or large chunks of data that are updated as a whole.

framework the set of classes that implement the API. This includes core and extension packages. Responsibilities include applet selection, sending APDU bytes, and managing atomicity.

garbage collection the process by which dynamically allocated storage is automatically reclaimed during the execution of a program.

heap a common pool of free memory usable by a program. A part of the computer's memory used for dynamic memory allocation, in which blocks of memory are used in an arbitrary order. The Java Card virtual machine's heap is not required to be garbage collected. Objects allocated from the heap are not necessarily reclaimed.

installer the on-card mechanism to download and install `CAP` files. The installer receives executable binary from the off-card installation program, writes the binary into the smart card memory, links it with the other classes on the card, and creates and initializes any data structures used internally by the Java Card Runtime Environment.

installation program the off-card mechanism that employs a card acceptance device (CAD) to transmit the executable binary in a `CAP` file to the installer running on the card.

instance variables also known as non-static fields.

instantiation in object-oriented programming, to produce a particular object from its class template. This involves allocation of a data structure with the types specified by the template, and initialization of instance variables with either default values or those provided by the class's constructor function.

instruction a statement that indicates an operation for the computer to perform and any data to be used in performing the operation. An instruction can be in machine language or a programming language.

internally visible items that are not externally visible. These items are not described in a package's export file, but some such items use private tokens to represent internal references. See also *externally visible*.

JAR file an acronym for Java Archive file, which is a file format used for aggregating many files into one.

Java Card Platform Remote Method Invocation a subset of the Java Platform Remote Method Invocation (RMI) system. It provides a mechanism for a client application running on the CAD platform to invoke a method on a remote object on the card.

Java Card Runtime Environment (Java Card RE) consists of the Java Card virtual machine, the framework, and the associated native methods.

Java Card Virtual Machine (Java Card VM) a subset of the Java virtual machine, which is designed to be run on smart cards and other resource-constrained devices. The Java Card VM acts an engine that loads Java class files and executes them with a particular set of semantics.

Java Card RE entry point objects objects owned by the Java Card RE context that contain entry point methods. These methods can be invoked from any context and allow non-privileged users (applets) to request privileged Java Card RE system services. Java Card RE entry point objects can be either temporary or permanent:

temporary - references to temporary Java Card RE entry point objects cannot be stored in class variables, instance variables or array components. The Java Card RE detects and restricts attempts to store references to these objects as part of the firewall functionality to prevent unauthorized reuse. Examples of these objects are APDU objects and all Java Card RE-owned exception objects.

permanent - references to permanent Java Card RE entry point objects can be stored and freely reused. Examples of these objects are Java Card RE-owned AID instances.

JDK™ software an acronym for Java Development Kit. The JDK software is a Sun Microsystems, Inc. product that provides the environment required for software development in the Java programming language. The JDK software is available for a variety of operating systems, for example Sun Microsystems Solaris™ OS and Microsoft Windows.

library package a Java programming language package that does not contain any non-abstract classes that extend the class `javacard.framework.Applet`. An applet package contains one or more non-abstract classes that extend the `javacard.framework.Applet` class.

local variable a data item known within a block, but inaccessible to code outside the block. For example, any variable defined within a method is a local variable and cannot be used outside the method.

logical channel as seen at the card edge, works as a logical link to an application on the card. A logical channel establishes a communications session between a card applet and the terminal. Commands issued on a specific logical channel are forwarded to the active applet on that logical channel. For more information, see the *ISO/IEC 7816 Specification, Part 4*. (`http://www.iso.org`).

MAC an acronym for Message Authentication Code. MAC is an encryption of data for security purposes.

mask production (masking) refers to embedding the Java Card virtual machine, runtime environment, and applets in the read-only memory of a smart card during manufacture.

method a procedure or routine associated with one or more classes in object-oriented languages.

multiselectable applets implements the `javacard.framework.MultiSelectable` interface. Multiselectable applets can be selected on multiple logical channels at the same time. They can also accept other applets belonging to the same package being selected simultaneously.

multiselected applet an applet instance that is selected and, therefore, active on more than one logical channel simultaneously.

namespace a set of names in which all names are unique.

native method a method that is not implemented in the Java programming language, but in another language. The `CAP` file format does not support native methods.

nibble four bits.

object-oriented a programming methodology based on the concept of an *object*, which is a data structure encapsulated with a set of routines, called *methods*, which operate on the data.

object owner the applet instance within the currently active context when the object is instantiated. An object can be owned by an applet instance, or by the Java Card RE.

objects in object-oriented programming, unique instances of a data structure defined according to the template provided by its class. Each object has its own values for the variables belonging to its class and can respond to the messages (methods) defined by its class.

origin logical channel the logical channel on which an APDU command is issued.

owning context the context in which an object is instantiated or created.

package a namespace within the Java programming language that can have classes and interfaces.

PCD an acronym for Proximity Coupling Device. The PCD is a contactless card reader device.

persistent object persistent objects and their values persist from one CAD session to the next, indefinitely. Objects are persistent by default. Persistent object values are updated atomically using transactions. The term persistent does not mean there is an object-oriented database on the card or that objects are serialized and deserialized, just that the objects are not lost when the card loses power.

PIX see *AID*.

RAM (random access memory) temporary working space for storing and modifying data. RAM is non-persistent memory; that is, the information content is not preserved when power is removed from the memory cell. RAM can be accessed an unlimited number of times and none of the restrictions of EEPROM apply.

reference implementation a fully functional and compatible implementation of a given technology. It enables developers to build prototypes of applications based on the technology.

remote interface an interface which extends, directly or indirectly, the interface `java.rmi.Remote`.

Each method declaration in the remote interface or its super-interfaces includes the exception `java.rmi.RemoteException` (or one of its superclasses) in its `throws` clause.

In a remote method declaration, if a remote object is declared as a return type, it is declared as the remote interface, not the implementation class of that interface.

In addition, Java Card RMI imposes additional constraints on the definition of remote methods. These constraints are as a result of the Java Card platform language subset and other feature limitations.

remote methods the methods of a remote interface.

remote object an object whose remote methods can be invoked remotely from the CAD client. A remote object is described by one or more remote interfaces.

RFU acronym for Reserved for Future Use.

RID see *AID*.

RMI an acronym for Remote Method Invocation. RMI is a mechanism for invoking instance methods on objects located on remote virtual machines (meaning, a virtual machine other than that of the invoker).

ROM (read-only memory) memory used for storing the fixed program of the card. A smart card's ROM contains operating system routines as well as permanent data and user applications. No power is needed to hold data in this kind of memory. ROM cannot be written to after the card is manufactured. Writing a binary image to the ROM is called masking and occurs during the chip manufacturing process.

runtime environment see *Java Card Runtime Environment (Java Card RE)*.

shareable interface an interface that defines a set of shared methods. These interface methods can be invoked from an applet in one context when the object implementing them is owned by an applet in another context.

shareable interface object (SIO) an object that implements the shareable interface.

smart card a card that stores and processes information through the electronic circuits embedded in silicon in the substrate of its body. Unlike magnetic stripe cards, smart cards carry both processing power and information. They do not require access to remote databases at the time of a transaction.

terminal a Card Acceptance Device that is typically a computer in its own right and can integrate a card reader as one of its components. In addition to being a smart card reader, a terminal can process data exchanged between itself and the smart card.

thread the basic unit of program execution. A process can have several threads running concurrently each performing a different job, such as waiting for events or performing a time consuming job that the program doesn't need to complete before going on. When a thread has finished its job, it is suspended or destroyed.

The Java Card virtual machine can support only a single thread of execution. Java Card technology programs cannot use class `Thread` or any of the thread-related keywords in the Java programming language.

transaction an atomic operation in which the developer defines the extent of the operation by indicating in the program code the beginning and end of the transaction.

transient object the state of transient objects do not persist from one CAD session to the next, and are reset to a default state at specified intervals. Updates to the values of transient objects are not atomic and are not affected by transactions.

verification a process performed on a `CAP` file that ensures that the binary representation of the package is structurally correct.

word an abstract storage unit. A word is large enough to hold a value of type `byte`, `short`, `reference` or `returnAddress`. Two words are large enough to hold a value of `integer` type.

Index

A

aaload, 7-5
aastore, 7-6
abstract, 2-7
AbstractMethodError, 2-21
access control, 2-3
  remote interfaces, 2-12
aconst_null, 7-8
AID, 4-3
AID-based naming, 4-3
aload, 7-8
aload_, 7-9
anewarray, 7-10
applet, 1-3
  mulitselectable, 2-12
applet component, 6-12
applet firewall, 1-5
application identifier
  See AID
areturn, 7-10
ArithmeticException, 2-21
ArrayIndexOutOfBoundsException, 2-21
arraylength, 7-11
arrays, 2-7, 2-10, 5-1, 6-1
ArrayStoreException, 2-21
astore, 7-12
astore_<n>, 7-12
athrow, 7-13
attributes, 2-16, 5-15
  ConstantValue, 5-15

B

baload, 7-14
bastore, 7-15
big-endian, 3-4
binary compatibility, 4-11
binary file formats, 3-4
binary representation, 4-1
bipush, 7-16
bitfield structures, 6-1
boolean, 2-7, 3-1
break, 2-7
bspush, 7-16
byte, 2-7, 3-1
bytecode, 2-16

C

CAP, 1-3
CAP file, 4-1
CAP file format, 6-1
case, 2-7
catch, 2-7
char, 2-4
checkcast, 7-17
class, 2-7, 5-8
  in a package, 2-9
  initialization, 2-11
  initialization methods, 3-4
  instances, 2-10
  object, 2-7
  remote, 2-12
  throwable, 2-8

unsupported, 2-5
class, 2-7
class component, 6-19, 6-21
class file, 2-13, 4-1
class_debug_info, 6-58
class_descriptor_info, 6-50
class_info, 6-26
ClassCastException, 2-21
ClassCirculatoryError, 2-21
ClassFile, 2-15
ClassFormatError, 2-21
ClassNotFoundException, 2-20
classsystem, 2-5
CloneNotSupportedException, 2-20
cloning, 2-3
compatibility, 4-11
components, 6-2
applet, 6-12
class, 6-19, 6-21
constant pool, 6-14
debug, 6-57
defining, 6-4
descriptor, 6-49
directory, 6-9
export, 6-47
header, 6-6
import, 6-13
installation, 6-5
method, 6-35
reference location, 6-44
static field, 6-41
constant pool, 2-14, 2-15
constant pool component, 6-14
CONSTANT_Classref, 5-6, 6-16
CONSTANT_InstanceFieldref, 6-18
CONSTANT_Integer, 5-7
CONSTANT_Package, 5-5
constant_pool, 5-4
CONSTANT_StaticFieldref, 6-19
CONSTANT_StaticMethodref, 6-19
CONSTANT_SuperMethodref, 6-18
CONSTANT_Utf8 structure, 5-7
CONSTANT_VirtualMethodref, 6-18
ConstantValue attribute, 5-15
context, 3-2
current, 3-3
owning, 3-3
context switch, 3-3
continue, 2-7
converted applet
See CAP
current context, 3-3

D

data type, 3-1
integer, 2-8
debug component, 6-57
default, 2-7
deletion, 2-8
descriptor component, 6-49
directory component, 6-9
do, 2-7
double, 2-4
dup, 7-19
dup_x, 7-20
dup2, 7-21
dynamic class loading, 2-2
dynamic object creation, 2-5

E

else, 2-7
errors, 2-21, 7-2
exception_handler_info, 6-37
ExceptionInInitializerError, 2-21
exceptions, 2-6, 2-19, 3-4, 7-3
checked, 2-20
uncatchable, 2-19
uncaught, 2-19
export component, 6-47
export file, 1-4
conversion, 4-6
export file format, 4-2, 5-1
name, 5-2
ownership, 5-2
structure, 5-3
extends, 2-7
externally visible items, 4-5

F

field_debug_info, 6-61

field_descriptor_info, 6-52
fields, 2-14, 2-15, 5-11
  descriptors, 2-14, 2-15
  static, 2-10
file formats
  Java Card platform, 4-1
final, 2-7
finalization, 2-2
finally, 2-7
float, 2-4
for, 2-7
frames, 3-3

G

garbage collection, 3-2
getfield_<t>, 7-22
getfield_<t>_this, 7-23
getfield_<t>_w, 7-25
getstatic_<t>, 7-26
goto, 2-7, 7-27
goto_w, 7-28

H

header component, 6-6
heap, 3-2
high bit, 6-1

I

i2b, 7-29
i2s, 7-29
iadd, 7-30
iaload, 7-31
iand, 7-32
iastore, 7-32
icmp, 7-33
iconst_<i>, 7-34
idiv, 7-34
if, 2-7
if<cond>, 7-39
if<cond>_w, 7-40
if_acmp, 7-35
if_acmp<cond>_w, 7-36
if_scmp<cond>, 7-37
if_scmp<cond>_w, 7-38
ifnonnull, 7-41
ifnonnull_w, 7-41
ifnull, 7-42
ifnull_w, 7-43
iinc, 7-43
iinc_w, 7-44
iipush, 7-45
IllegalAccessError, 2-21
IllegalAccessException, 2-20
IllegalArgumentException, 2-21
IllegalMonitorStateException, 2-21
IllegalStateException, 2-21
IllegalThreadStateException, 2-21
iload, 7-46
iload_<n>, 7-46
ilookupswitch, 7-47
implements, 2-7
import, 2-7
import component, 6-13
imul, 7-49
IncompatibleClassChangeError, 2-21
IndexOutOfBoundsException, 2-21
ineg, 7-49
initialization, 3-3
  class, 2-11
installation, 4-7, 6-5
instance initialization methods, 3-3
instanceof, 2-7, 7-50
instances, 2-10
InstantiationError, 2-21
InstantiationException, 2-20
instruction set, 3-4, 7-1
instructions
  by opcode mnemonic, 8-4
  by opcode value, 8-1
int, 2-7, 3-1
integer data type, 2-8
interface, 2-7
interface initialization methods, 3-4
interface_info, 6-26
interfaces, 2-9, 5-8
  remote, 2-12
InternalError, 2-22

InterruptedException, 2-20
invokeinterface, 7-52
invokespecial, 7-54
invokestatic, 7-55
invokevirtual, 7-56
ior, 7-58
irem, 7-58
ireturn, 7-59
ishl, 7-60
ishr, 7-61
istore, 7-61
istore_<n>, 7-62
isub, 7-63
itableswitch, 7-63
items, 5-1, 6-1
iushr, 7-65
ixor, 7-66

J

JAR files, 4-2, 5-2, 6-3
Java Card applet
  See applet
Java Card Converter, 1-4
Java Card file formats, 4-1
Java Card RE context, 3-3
Java Card Remote Method Invocation
  See Java Card RMI
Java Card RMI, 2-12
  parameters, 2-13
  return values, 2-13
Java Card system, 1-2
Java Card Virtual Machine, 1-2, 3-5
  limitations, 2-8
Java Card Virtual Machine errors, 7-2
Java Card Virtual Machine instruction set, 7-1
Java programming language, 2-1
  unsupported features, 2-2
Java Virtual Machine, 2-1, 2-13
jsr, 7-66

K

keywords, 2-4, 2-7

L

ldc, 2-18
ldc_w, 2-18
library package, 1-4
LinkageError, 2-21
linking, 4-5, 4-7, 4-13
long, 2-4
lookupswitch, 2-18

M

method component, 6-35
method descriptors, 2-15
method_debug_info, 6-62
method_descriptor_info, 6-54
method_info, 6-38
methods, 2-10, 2-14, 2-16, 5-13
  static, 2-10
  virtual, 2-6, 4-9
mnemonic, 7-4
multiselectable applets, 2-12
must, 7-1

N

naming, 4-3
native, 2-4
NegativeArraySizeException, 2-21
new, 2-7, 7-67
newarray, 7-68
NoClassDefFoundError, 2-22
nop, 7-69
NoSuchFieldError, 2-22
NoSuchFieldException, 2-20
NoSuchMethodError, 2-22
NoSuchMethodException, 2-20
NullPointerException, 2-21
NumberFormatException, 2-21
numeric types, 3-1

O

object, 2-7
objects, 2-7, 2-10
  deletion mechanism, 2-8
  dynamic creation, 2-5
  representation of, 3-3

opcode mnemonic, 8-4
opcode value, 8-1
opcodes, 3-4
  reserved, 7-2
operands, 3-4
OutOfMemoryError, 2-22
owning context, 3-3

P

package, 2-5
  applet, 1-4
  classes, 2-9
  library, 1-4
  name, 2-9
  references, 2-9
  versions, 4-13
package, 2-7
parameters, 2-13
PIX, 4-3
pop, 7-69
pop2, 7-70
primitive types, 3-1
primitive values
  values
    primitive, 3-1
private, 2-7
proprietary identifier extension
  See PIX
protected, 2-7
public, 2-7
putfield_<t>, 7-70
putfield_<t>_this, 7-72
putfield_<t>_w, 7-74
putstatic_<t>, 7-75

R

reference location component, 6-44
reference types, 3-1
reference types, 3-1
reference values, 3-1
references
  external, 4-6
  internal, 4-6
remote classes, 2-12
remote interfaces, 2-12
  access control, 2-12
reserved opcodes, 7-2
resident image, 4-13
resource identifier
  See RID
ret, 7-77
return, 2-7, 7-78
return values, 2-13
RID, 4-3
RMI
  See Java Card RMI
runtime data areas, 3-2
runtime exceptions
  exceptions
    runtime, 2-21

S

s2b, 7-78
s2i, 7-79
sadd, 7-79
saload, 7-80
sand, 7-81
sastore, 7-81
sconst_<s>, 7-82
sdiv, 7-83
security, 1-4
  exceptions, 7-3
  manager, 2-2
SecurityException, 2-21
short, 2-7, 3-1
sinc, 7-83
sinc_w, 7-84
sipush, 7-85
sload, 7-85
sload_<n>, 7-86
slookupswitch, 7-87
smul, 7-88
sneg, 7-88
sor, 7-89
srem, 7-90
sreturn, 7-90
sshl, 7-91
sshr, 7-92
sspush, 7-93

sstore, 7-93
sstore_<n>, 7-94
ssub, 7-94
stableswitch, 7-95
StackOverflowError, 2-22
static, 2-7
static field component, 6-41
static field image, 6-41
static fields, 2-10
static methods, 2-10
strictfp, 2-4
StringIndexOutOfBoundsException, 2-21
super, 2-7
sushr, 7-96
swap_x, 7-97
switch, 2-7
switch statements, 2-11
sxor, 7-98
synchronized, 2-4
system, 2-5

T

tables, 5-1, 6-1
tableswitch, 2-19
this, 2-7
ThreadDeath, 2-22
threads, 2-2, 3-2
throw, 2-7
throwable, 2-8
throws, 2-7
token-based linking, 4-5
tokens
  assignment, 4-7
  class, 4-8
  details, 4-7
  instance field, 4-9
  interface methods, 4-10
  package, 4-8
  private, 4-5
  public, 4-5
  static field, 4-8
  static method, 4-8
  virtual method, 4-10
transient, 2-4
try, 2-7
type_descriptor, 6-23
type_descriptor_info, 6-56
types, 2-7, 3-5
  boolean, 3-1
  numeric, 3-1
  primitive, 3-1
  reference, 3-1
  reference, 3-1
  unsupported, 2-4

U

union notation, 6-1
UnknownError, 2-22
UnsatisfiedLinkError, 2-22

V

values
  reference, 3-1
VerifyError, 2-22
virtual methods, 2-6, 4-9
VirtualMachineError, 2-22
void, 2-7
volatile, 2-4

W

while, 2-7
wide, 2-19
words, 3-2